

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

# Quasi-stateful RNNs with truncated back-propagation through time

Tesi di Laurea Magistrale in
High Performance Computing Engineering - Ingegneria del Calcolo ad Alte Prestazioni

Author: **Valentina Moretti**

Student ID: 222914
Advisor: Prof. Manuel Roveri, Prof. Cesare Alippi
Co-advisors: Ivan Marisca, Andrea Cini
Academic Year: 2023-2024

Questa tesi fa parte del Double Master Degree in High Performance Computing Engineering presso il Politecnico di Milano e l'Università della Svizzera italiana

# Abstract

In the context of multivariate time series forecasting, this study addresses the challenge of efficiently training recurrent neural networks (RNNs) on long time series by proposing a novel methodology to initialize the state of the network from past observations. RNNs, with their recurrent connections, are designed to process sequences in order, retaining the hidden state from one batch to the next. This type of training is computationally expensive and prevents the input sequence from being divided into batches, of a size that fits the hardware, and that can be processed in parallel. The primary objective is to improve the computational efficiency and scalability of the training procedure by computing gradients only for a predefined, fixed-size window interval rather than the entire time series while still modeling longer-range dependencies. The proposed approach exploits a train-free recurrent encoder (a *reservoir*) to embed past observations into an initial representation. The resulting encoding is then further processed to initialize the state of the trainable RNN which refines and updates the representation by processing the remaining observations. In this way, the trainable components process only the last time steps while the past history is accounted for in the initialization encoding. Since the reservoir is training-free, it can process the whole time series ahead of training and the trainable RNN can be fitted efficiently by subsampling mini-batches of the input data and making the training procedure scalable and adjustable according to the hardware capacity. Our approach is validated against relevant baselines on both synthetic and real-world datasets from different domains. Empirical results show that the proposed model compares favorably against the baselines, often matching the performance of the reference architectures while being drastically more scalable.

**Keywords:** RNN, echo state networks, reservoir computing, time series forecasting, training efficiency

# Abstract in lingua italiana

Nel contesto della previsione di serie temporali multivariate, questo studio affronta la sfida di addestrare in modo efficiente le RNNs su serie temporali lunghe, proponendo una nuova metodologia per inizializzare lo stato della rete a partire dalle osservazioni passate. Le RNNs, con le loro connessioni ricorrenti, sono progettate per elaborare sequenze in ordine, mantenendo lo stato nascosto da un batch all'altro. Questo tipo di addestramento è computazionalmente costoso e impedisce di suddividere la sequenza di input in batch di una dimensione adatta all'hardware e che possano essere elaborate in parallelo. L'obiettivo principale è migliorare l'efficienza computazionale e la scalabilità della procedura di addestramento, calcolando i gradienti solo per una finestra temporale predefinita, di dimensioni fisse, anziché per l'intera serie temporale, mantenendo comunque la capacità di modellare dipendenze a lungo raggio. L'approccio proposto sfrutta un encoder ricorrente non addestrabile (un *reservoir*) per incorporare le osservazioni passate in una rappresentazione. L'encoding risultante viene quindi ulteriormente elaborato per inizializzare lo stato della RNN addestrabile, che affina e aggiorna la rappresentazione elaborando le osservazioni rimanenti. In questo modo, i componenti addestrabili elaborano solo gli ultimi passi temporali, mentre la storia passata viene considerata nell'encoding di inizializzazione. Poiché il *reservoir* non richiede addestramento, può elaborare l'intera serie temporale prima dell'addestramento e la RNN addestrabile può essere ottimizzata in modo efficiente campionando mini-batch dei dati di input, rendendo la procedura di addestramento scalabile e regolabile in base alla capacità hardware. Il nostro approccio è convalidato rispetto a baseline rilevanti sia su dataset sintetici che su dataset reali provenienti da diversi domini. I risultati empirici mostrano che il modello proposto offre prestazioni competitive rispetto alle baselines, spesso eguagliando le prestazioni delle architetture di riferimento, risultando però drasticamente più scalabile.

**Parole chiave:** Reti neurali ricorrenti, echo state networks, reservoir computing, predizione di serie temporali, efficienza del training

# Contents

**Abstract** **iii**

**Abstract in lingua italiana** **v**

**Contents** **vii**

**List of Figures** **xi**

**List of Tables** **xiii**

**Acronyms** **xv**

**1 Introduction** **1**

1.1 Thesis outline . . . 3

**2 Preliminaries** **5**

2.1 Time series . . . 5

2.1.1 Stationary and nonstationary process . . . 5

2.1.2 Classifications of time series and data collection methods . . . 6

2.2 Forecasting . . . 7

2.3 Time series detrending and decomposition . . . 7

2.3.1 Patterns . . . 8

2.3.2 Decomposition . . . 9

2.3.3 Differencing . . . 10

2.3.4 Detrending . . . 10

2.4 Methods for time series forecasting . . . 11

2.4.1 Regression models . . . 12

2.4.2 Least squares method . . . 13

2.4.3 Exponential smoothing . . . 14

2.5 Learning in a stationary environment: AR, MA, ARMA and ARIMA models 15

2.5.1 MA models . . . 15
2.5.2 AR models . . . 17
2.5.3 ARMA models . . . 19
2.5.4 ARIMA models . . . 20
2.5.5 Autoregressive nonlinear predictive models . . . 21
2.5.6 Multi-step prediction stategies . . . 24
2.5.7 Learning in a non-stationary environment . . . 25
2.6 Evaluation metrics . . . 26
2.6.1 Scale-dependent metrics . . . 26
2.6.2 Percentage errors . . . 27
2.6.3 Relative and absolute errors . . . 27
2.7 Challenges of time series forecasting . . . 28
2.7.1 Outliers . . . 28
2.7.2 Missing values . . . 28
2.7.3 The problem of long-term time series forecasting and possible approaches . . . 28

**3 Related work** **31**
3.1 Recurrent neural networks . . . 31
3.1.1 Architecture . . . 31
3.1.2 Training . . . 32
3.1.3 LSTM and GRU . . . 34
3.2 Reservoir computing . . . 36
3.2.1 Randomization in neural networks . . . 36
3.2.2 Deep randomized feed-forward neural networks . . . 37
3.2.3 Echo state networks . . . 38
3.2.4 Euler state networks . . . 44
3.2.5 Reservoir computing for multivariate time series . . . 45
3.3 Baseline models . . . 46
3.3.1 Linear models . . . 46
3.3.2 Nonpersistent state recurrent neural network (nonpersistent state RNN) . . . 49
3.3.3 Persistent state recurrent neural network (persistent state RNN) . . . 49
3.3.4 ESN . . . 50

**4 Hybrid learnable RNNs with reservoir state initializers** **55**
4.1 High-level motivation and introduction to the proposed approach . . . 55
4.2 Proposed approach . . . 57

4.2.1 Problem statement . . . . . 60
4.2.2 Solution overview . . . . . 61

**5 Experimental evaluation** **65**
5.1 Experimental setting . . . . . 65
5.1.1 First experiment: performance analysis on synthetic dataset . . . . 65
5.1.2 Second experiment: performance comparison on real-word data . . 68
5.1.3 Evaluation protocol and hyperparameters search . . . . . 70
5.2 Results . . . . . 71
5.3 Result analysis . . . . . 72
5.3.1 Multi step-ahead forecasting performances . . . . . 72
5.3.2 Performance at specific time steps of the horizon . . . . . 78
5.3.3 Hyperparameters . . . . . 79
5.3.4 Memory and computational efficiency . . . . . 84
5.3.5 Ablation study . . . . . 85

**6 Conclusions** **89**
6.1 Implication of the thesis . . . . . 89
6.2 Limitations . . . . . 89
6.3 Future works . . . . . 90

**Bibliography** **91**

**A Graph neural networks** **105**
A.1 A brief introduction to graph neural networks . . . . . 105
A.2 Spatio-temporal time series forecasting . . . . . 106
A.3 Research gap: scalable spatio-temporal RNN architecture . . . . . 109
A.3.1 Trainable decoding . . . . . 111

**B Echo state network (ESN) technical specifications** **113**
B.1 Lyapunov exponents . . . . . 113

**Acknowledgements** **115**

# List of Figures

2.1 Time series patterns. . . . . 8
2.2 STL decomposition of the time series. . . . . 9
2.3 MA model . . . . . 16
2.4 AR model . . . . . 18
2.5 ARMA model . . . . . 19
2.6 Sequence to sequence model. . . . . 22
2.7 Sliding window approach. . . . . 23

3.1 Comparison between FNNs and RNNs. . . . . 31
3.2 A schematic representation of a cell of an ERNN block. . . . . 32
3.3 Back propagation through time in RNNs. . . . . 33
3.4 A schematic representation of a cell of an LSTM block. . . . . 35
3.5 A schematic representation of a cell of a GRU block. . . . . 36
3.6 Echo state network. . . . . 39
3.7 Representation of the ESN architecture. . . . . 40
3.8 The representation of a DeepESN with $N_l$ stacked reservoir layers. . . . . 43
3.9 Overview of the models and what part of the time series they process. . . . 47
3.10 Overview of the linear model. . . . . 48
3.11 Overview of the nonpersistent state RNN. . . . . 49
3.12 A visual representation of the time steps in a batch. . . . . 51
3.13 Overview of the persistent state RNN. . . . . 51
3.14 Overview of the ESN. . . . . 53

4.1 Overview of the models and what part of the time series they process with a focus on the Reservoir recurrent neural network (ReRNN). . . . . 58
4.2 Overview of the ReRNN. . . . . 59

5.1 A view of eight random time series of the synthetic dataset. . . . . 67
5.2 Metrics on the synthetic dataset . . . . . 75
5.3 Metrics on the MetrLa dataset . . . . . 76
5.4 Metrics on the CEREn-SME dataset . . . . . 76

5.5 MAE on the synthetic dataset . . . . . 77
5.6 MAE on the MetrLa dataset . . . . . 77
5.7 MAE on the CEREn-SME dataset . . . . . 78
5.8 MAE over the orizon for the synthetic dataset . . . . . 80
5.9 MAE over the orizon for the MetrLa dataset . . . . . 80
5.10 MAE over the orizon for the CEREn-SME dataset . . . . . 81
5.11 Plot of the performances of the models on the synthetic dataset. . . . . 86
5.12 Ablation study: test MAE . . . . . 87
5.13 Ablation study: GPU usage and memory requirements. . . . . 88

A.1 Spatiotemporal Graph Neural Network. . . . . 108
A.2 Scalable spatiotemporal RNN. . . . . 110

# List of Tables

5.1 Datasets overview. . . . . 69
5.2 Performance metrics for the datasets. . . . . 75
5.3 Hyperparameters used in the experiments. . . . . 83
5.4 GPU usage, memory consumption, and batch per second (Batch/S) . . . . 85
5.5 Ablation study: GPU utilization and memory requirements. . . . . 86
5.6 Ablation study: GPU usage and memory consumption. . . . . 87

# Acronyms

**ESN** echo state network. ix, xi, 3, 31, 38–40, 43, 47, 50, 52, 53, 55–57, 63, 66, 68, 72, 74, 75, 81–86, 110, 111, 113

**FullBPTT RNN** full backpropagation through time recurrent neural network. 73–75, 78, 79, 83–85, 89, 90

**GNN** graph neural network. 68, 90, 105, 106

**iid** independent and identically distributed. 5, 18, 29, 109, 110

**MAE** mean absolute error. 26, 27, 61, 62, 68, 71, 72, 74–78, 80, 81

**MAPE** mean absolute percentage error. 27, 68, 71, 74–76

**MLP** multilayer perceptron. 47, 52, 56, 58, 59, 62, 68, 84–88, 108, 110

**MRE** mean relative error. 68, 71, 74–76

**MSE** mean square error. 26, 42, 68, 71, 72, 74–76

**nonpersistent state RNN** nonpersistent state recurrent neural network. viii, xi, 2, 3, 29, 47, 49, 56, 57, 60, 66, 68–70, 72, 74, 75, 78, 79, 83–85

**persistent state RNN** persistent state recurrent neural network. viii, xi, 2, 3, 29, 47, 49–51, 55–57, 66–75, 78, 79, 83–86

**RC** reservoir computing. 31, 45, 52

**ReRNN** reservoir recurrent neural network. xi, 2, 57–59, 65–75, 78, 79, 81, 83–89

**RNN** recurrent neural network. iii, v, viii, ix, xi, xii, 1–3, 21, 22, 29, 31–35, 38, 42, 47, 49–51, 55–64, 66, 67, 69, 70, 72, 73, 83–85, 89, 105, 106, 108–110

**STGNN** spatiotemporal graph neural network. 55, 106–110

# 1 | Introduction

Long-range time series forecasting is a subject of interest in various domains, including energy analytics [71, 117], public health management [88] and finance [3]. For instance, it is necessary to forecast the output of a wind power plant for integrating renewable energy into the power grid depending on the unpredictable wind patterns [117]. Furthermore, in the context of public health, it is necessary to predict the dynamics of diseases for effective decision-making by gathering data throughout various locations and time periods and looking for patterns such as seasonality, outbreaks, and anomalies [88]. In this domain, it is particularly needed the ability to capture dependencies with historical data from the distant past, as these can significantly impact the prediction of future dynamics of the disease. Similarly, in financial markets, investors need forecasting to predict stock prices, which is not trivial due to the complex nature of market dynamics [3]. All these applications have one common challenge: the need to capture both short-term and long-term patterns in time series.

These previous examples showcase the importance of long-term time series forecasting in different contexts and provide the reason for which we often need to consider long-range temporal dependencies. Moreover, when addressing this challenge, we often need to address the computational complexity of long-term forecasting models and the need to handle the memory-related issues that arise when dealing with long input history.

Among the machine learning models, RNNs are well-suited to analyze sequential data. RNNs, indeed, instead of treating each input independently, capture the relations in the input sequence thanks to their recurrent connection. This ability makes them particularly useful in applications such as time series analysis, where the order of the data matters. RNNs are a class of models particularly advantageous for time series forecasting since they have a recurrent state that allows them to remember the information of previous steps of the time sequence. The recurrent connections enable them to capture temporal dependencies, which is required to perform time series forecasting. However, RNN struggle with processing long and complex sequences because they compress the past history into a relatively small state, leading to the loss of important information. In addition to accuracy

issues, RNNs also faces computational efficiency problems. Indeed, since they maintain a state, they must process data in a sequential order, which prevents subsampling and parallel processing, making the training process less scalable and efficient. We will call this type of RNN, *persistent state RNN* since it truly maintains a linked state along the processing of the whole sequence without resetting it at any point. Indeed, the internal state is never reset, and the RNN keeps and uses the hidden state coming from the processing of the previous time steps.

On the other hand, an alternative type of RNN can be used, which is more efficient and allows subsampling, at the price of not keeping a history linked across batches. We will call this second alternative of RNNs, *nonpersistent state RNN* since the internal state is reset after the processing of each batch. This method is scalable and computationally efficient since it allows the batches to be shuffled and processed in parallel. However, since the state is reset at the beginning of each batch, it only accounts for the history within the time steps of the batch, breaking and losing the dependencies that exist across the different batches.

It is worth noting that the difference between persistent state RNN and nonpersistent state RNN lies in how the hidden state is initialized and managed. To blend the advantages of both approaches, namely the computational efficiency of nonpersistent state RNNs and the ability to capture long-term dependencies of the persistent state RNNs, we introduce a novel approach named ReRNN which stands for reservoir RNN.

In this approach, a randomized RNN (a reservoir) projects the dynamics of the time series in a high-dimension representation where the important dynamics and dependencies among the time steps of the time series are embedded in a rich feature space. The representation is then used to initialize a trainable RNN that will process the last time steps of the time series having, in this way, access to the history preceding its window. Since the reservoir is training-free, it can process the whole time series ahead of training and a RNN can be trained in an efficient way enabling subsampling similarly to the nonpersistent state RNN.

Since this approach has both trainable and training-free components, it may require less hard tuning of the hyperparameters than a completely training-free model. To assert the ability of our approach to outperform the capacity to memorize long-term dependencies of the state-of-art models, we collect the test metrics performance on both real-world and synthetic datasets. The results show that our model consistently outperforms the other baselines when using a training procedure that enables subsampling in batches and employs the same window size.

Our main contributions can be summarized as follows.

- We introduce a new approach for the initialization of RNNs's internal state by relying on reservoir computing.
- We address the problems of efficiency, scalability, computational complexity, and memory usage of the training procedure in the context of long-term forecasting of time series while keeping strong performance at inference time.
- We perform an empirical evaluation of the proposed model on both synthetic and real-world datasets from different domains.
- We compare the results of the proposed model to reference architectures, including nonpersistent state RNNs, persistent state RNNs, ESNs and linear models.

To validate our findings, we benchmark the proposed model and the baseline models on the task of long-range time series forecasting on synthetic datasets and real-world data from traffic and energy analytics. Then we analyze how different model behaves at different forecasting horizons and assess the contribution in terms of performance and efficiency of the proposed components.

Our work introduces a new framework that combines scalable and efficient training, which is crucial in some contexts like distributed sensor networks with hardware constraints. Furthermore, our hybrid architecture can be simpler to train resulting in improved performance accuracy in some scenarios.

## 1.1. Thesis outline

The thesis is structured in the following way:

- In Chapter 2 we introduce the two main topics of the thesis. We introduce the topic of time series analysis and forecasting, evaluation metrics, and forecasting methods, including classical regression models and machine learning techniques, and we present the challenges of long-term forecasting which is the main topic of the thesis.
- In Chapter 3 we introduce RNNs, reservoir Computing, which is at the basis of our model, ESNs and deep ESNs. Then, we describe the baseline models we will use in our experiments in detail.
- In Chapter 4, we formulate the problem we address in the thesis, and we propose our model, its architecture and training procedure.

- In Chapter 5, we present the experimental setup, the dataset and evaluation metrics and we report and analyze the results.
- Lastly, in Chapter 6, we discuss the implications of our work and we present possible future paths toward which our work could be extended.

# 2 | Preliminaries

In this chapter, we introduce the concept of time series and forecasting, which are the main topics of this thesis. We start by defining a time series and its properties, such as stationarity and nonstationarity, and we introduce the concept of forecasting. We also introduce the concept of time series decomposition and the main metrics used to evaluate the performance of a forecasting model. We present the main methods used for time series forecasting, such as regression models, ARIMA models, machine learning models and we discuss the main challenges of time series forecasting.

We take [58] as one of the main references for the whole section.

## 2.1. Time series

A time series is a sequence of observed data measured over time.

A time series process is a stochastic process that generates a sequence of random, time-dependent observed values [58]. Hence, a time series is the realization of the stochastic process. Modeling a time series means modeling a stochastic process that generates the observed values in a noisy framework. Indeed, the predicted values of a time series are always affected by noise.

Time series can be classified as stationary and non-stationary, univariate and multivariate, continuous and discrete time series.

### 2.1.1. Stationary and nonstationary process

When dealing with a time series, we cannot assume that the observations are independent, because the observations are recorded at different time intervals and the observed values are affected by the previous values. This is a fundamental concept because, when considering time series, we cannot make the assumption independent and identically distributed (iid) data. However, it can still be assumed that the observations are identically distributed, that is, they come from the same probability density function.

The process of generating the time series is stationary when the statistical properties of the time series do not change over time and it does not contain seasonalities. This means that the data generated from the stochastic process are realizations of a random variable whose distribution does not change over time. On the other hand, if one of the previous conditions is not met, the time series is nonstationary. For example, a time series coming from a drifting sensor is nonstationary because the expected value of the observed values changes over time due to the aging of the sensor. In this case, observed values are generated by a random variable whose distribution can change over time, i.e., a distribution dependent on the time index.

Moreover, time series are called weakly stationary, if only the first and second moments are constant over time. This means that the process has a constant mean and that the covariance between the values at any couple of time points, $t$ and $t-k$ depends only on the lag $k$ and not on the actual time $t$. In other words, the expected value and the covariance matrix of the random variable do not change over time but the distribution might [65].

If we are able to find a trend or seasonality in the time series, we should remove it with a transformation to the data, such as differencing or detrending, so that the training of the models can be performed on the stationary residual.

### 2.1.2. Classifications of time series and data collection methods

Here, we present the main categories of time series.

In the context of signal processing, a process is said to be time-invariant it does not explicitly depend on time. In other words, if an input $x(t)$ gives the output $y(t)$, the input at an elapsed time such as $x(t-t_0)$ will result in the output having the same delay $y(t-t_0)$ [92]. On the other hand, a process is time-varying when its outputs directly depend on time.

A discrete time series can be collected from a continuous physical process, where the observations are recorded at specific time intervals. The sampling rate and number of observations are two important aspects to consider to capture the information contained in the original continuous time series. Lastly, based on the regularity of the time intervals, a discrete time series can be further classified as equally spaced or unequally spaced.

A time series is multivariate if the observations are multi-dimensional where each dimension is associated with a different variable. For example, measuring different physical quantities in the air, such as temperature, pressure and humidity, would result in a mul-

tivariate time series. In this case, a vector can collect the observed values and each of its elements would represent a different physical quantity. On the other hand, if the time series is one-dimensional, it is univariate.

## 2.2. Forecasting

The goal of forecasting time series is to predict the future with accuracy [58] by capturing patterns and relationships in historical data, trying to avoid the simple replication of past events that may not occur again.

Based on some aspects, such as the nature of the application, the data available, the forecasting horizon and the predictability of the data, the forecasting approach can be a naïve one, as using the last observed value as the forecast, or one with high complexity, as using a deep learning model.

Forecasting can be classified into short-term forecasting, medium-term forecasting and long-term forecasting, based on the forecasting horizon [58]. Other factors that affect the primary variable to be forecasted can also be considered. For example, the model can include the effect of exogenous variables, which are not the primary variables that are forecast but they can affect the forecasting outcome. However, to integrate these variables into the forecasting model, it is necessary to know their values, especially for future time points, and to know their relationship with the primary variable that is forecast.

In a probabilistic forecast, the goal is to predict forecast distribution which is the set of possible future values that the primary random variable can take. However, we will focus on point forecast.

## 2.3. Time series detrending and decomposition

Various patterns exist in time series data such as trends, seasonalities or cycles. The performances of predictive models can be damaged by these patterns since they can hide the underlying dynamics of the stochastic process generating the time series and may cause the model to become biased toward learning these patterns. This problem can be addressed by removing the patterns, for example by applying differencing, detrending or other transformations to the data, and the model should be trained on the stationary residual. In the next sections, we will present some recurrent patterns, decomposition strategies, differencing and detrending transformations by following [58].

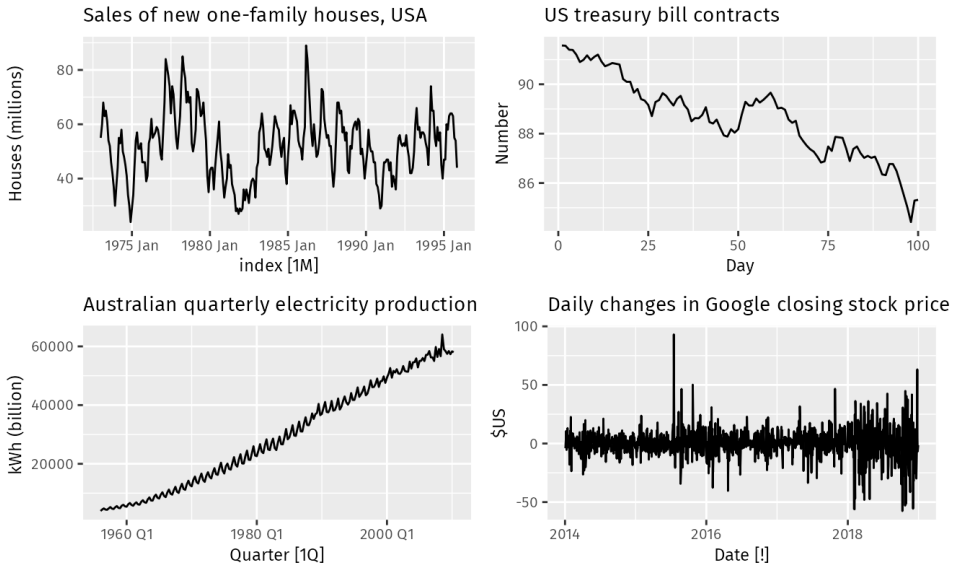



Figure 2.1: Time series patterns.
The first picture shows yearly seasonality and cyclic pattern every 6 years, the second picture in the first row shows a downward trend, the first picture in the second raw shows the trend and the seasonality and the last picture is a white noise.
*Source: [58]*

### 2.3.1. Patterns

A trend is a long-term increase or decrease in the data and can be linear or nonlinear. A seasonality is a pattern that repeats at fixed intervals, such as daily, weekly, monthly or yearly. A cycle is a pattern that repeats at non-fixed intervals and usually has longer periods than the seasonality periods [58] (see Figure 2.1).

Autocorrelation is the correlation between the values of a time series at different time points and it is used to identify the presence of patterns in the data. The correlation coefficient is called Autocorrelation Function (ACF) and it is a function of the lag $r_k$. Given a time series $y$, of length $T$, the autocorrelation at lag $k$ is defined as:

$$r_k = \frac{\sum_{t=k+1}^{T} (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^{T} (y_t - \bar{y})^2}$$

where $\bar{y}$ is the mean of the time series and $y_t$ is the value of the time series at time $t$ [58].

If the time series presents a trend, the autocorrelations for small lags are large and positive because observations close in time are similar to each other, while for large lags the autocorrelations decrease. If the time series presents a seasonality, the autocorrelations

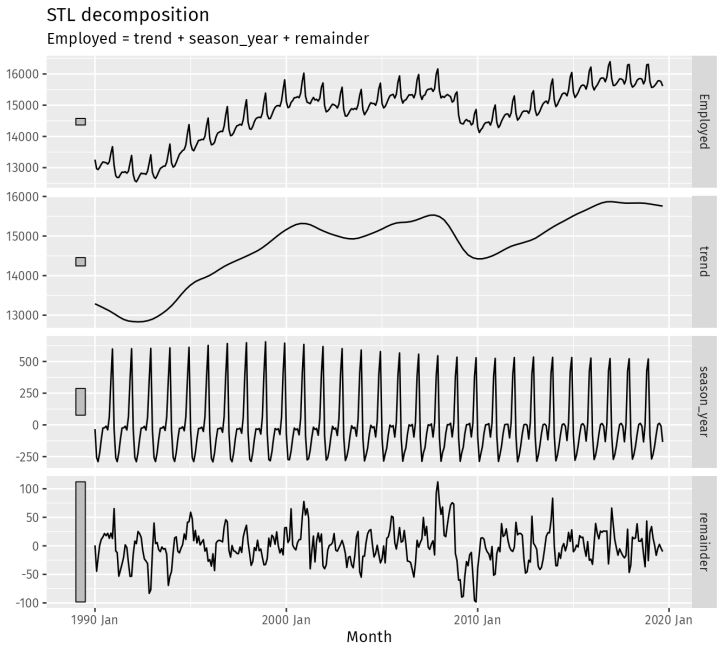



Figure 2.2: STL decomposition of the time series.
The first picture shows the original time series, the second picture shows the trend, the third picture shows the seasonality and the last picture shows the residuals.
*Source: [58]*

are larger for the multiples of the seasonal frequency, than for other lags.

If all patterns are removed, the resulting time series resembles noise [58].

### 2.3.2. Decomposition

Time series decomposition is the process of separating a time series into its components, such as trend, seasonality and the residual variation in the data. A time series can be decomposed into additive components or multiplicative components. In additive decomposition, the observed values are the sum of the trend $T$, the seasonality $S$ and the remainder $R$ (see Figure 2.2).

$$y_t = S_t + T_t + R_t$$

[58].

This is used when the magnitude of the seasonal or cyclic variation does not depend on the level of the time series. Otherwise, multiplicative decomposition is used where the observed values are the product of the trend, the seasonality and the remainder.

$$y_t = S_t \times T_t \times R_t$$

[58].

Alternatively, instead of applying a multiplicative decomposition, it can be applied an additive decomposition after having applied a log transformation to the data. Indeed, the previous equation becomes:

$$\log y_t = \log S_t + \log T_t + \log R_t$$

[58].

### 2.3.3. Differencing

The approach of differencing time series data is used to remove non-stationarity in time series.

It allows us to focus on relative changes in the data. After the model has processed the differenced data, the forecast can be transformed back. For this reason, transformations to remove non-stationarity must be invertible.

For instance, a time-dependent time series $y = t$ is non-stationary because the mean depends on time (time-variant), but if we take the difference between consecutive values, we obtain a stationary time series $y = 1$.

Note that removing $t-1$ may remove the trend, but the same lag doesn't always work for different time series. For example, a time series can have a growing trend and, at the same time, a cyclical behavior when observed with finer granularity (e.g., energy consumption may increase compared to the previous week rather than compared to the previous day) [58].

### 2.3.4. Detrending

A method to identify the trend is the moving average. Moving averages take windows of nearby data to decrease the influence of short-term fluctuations. It is employed to remove the trend from the time series. After computing the moving average, the original time series is subtracted from the moving average to obtain the residual component. A moving

average of order $m$ ($m-$MA) can be written as

$$\hat{T}_t = \frac{1}{m} \sum_{j=-k}^{k} y_{t+j},$$

where $m = 2k + 1$ [58]. It estimates the trend at time $t$ ($\hat{T}_t$) as the average of the observations in the window of fixed length $m$ centered at $t$. If $m$ is odd, the window is symmetric around $t$; if $m$ is even, the window is asymmetric. One way to address this asymmetry is to repeat another MA after the first one, but centering the window in the opposite direction (denoted with $2 \times m -$ MA) [58]. That is, if the first MA is centered more at the beginning of the window, the second MA is centered more at the end of the window. Moreover, the moving average will shrink the time series. To avoid this, the time series can be padded with the first or last value of the time series as many times as half of the window size.

After having calculated the trend with the moving average, the trend can be removed from the time series by subtracting the trend from the original time series, in the case of an additive decomposition, or by dividing the original time series by the trend, in the case of a multiplicative decomposition.

In order to remove the seasonality, instead, it is usually calculated the average of the detrended time series over the same period of the seasonality.

$$\hat{s}_t = \frac{1}{M} \sum_{h=1}^{M-1} y_{t+hT}$$

with $t = 1, \ldots, T$ and $MT < N$, where $M$ is the number of seasons in the time series and $T$ is the seasonal period [58].

For example, if the seasonality is monthly, the average of the detrended time series is taken over all the January, all the February, and so on. The values of the seasonalities can be removed from the detrended time series to obtain the residual [58].

## 2.4. Methods for time series forecasting

In this work, we will focus on point forecast, that is forecasting values, and not on probabilistic predictors that predict the distribution of the future values.

We introduce here the notion of predicted values and residuals [58], which are used to evaluate the performance of a forecasting model. The predicted value $\hat{y}_{t|t-1}$ is the value

predicted by the model based on the preceding time steps $y_1, \ldots, y_{t-1}$, while the residuals $e_t$ are the differences between the observed values $y_t$ and the predicted values $\hat{y}_t$, i.e. $e_t = y_t - \hat{y}_t$, in the training set. The residuals are used to evaluate the performance of the model. The residuals should be uncorrelated, have zero mean, have constant variance (homoscedasticity) and have a normal distribution. If the residuals are correlated, it means that the model has not captured all the information in the data and that means there is still some information left in the residuals that should be used to improve the forecast. If the residuals have a mean different from zero, it means that the model has a bias, and also in this case the model should be improved [58].

In the next sections, we will present regression models, the least square method and exponential smoothing by following [58].

### 2.4.1. Regression models

The linear regression models are used to model the linear relationship between the dependent variable we want to forecast $y$ and the independent variables $x$. In the case of a single predicted variable, the model is called simple linear regression and it is defined as

$$y_t = \beta_0 + \beta_1 x_t + \varepsilon_t$$

where $\beta_0$ is the intercept, $\beta_1$ is the slope, $x_t$ is the independent variable and $\varepsilon_t$ is the error term representing anything that affects the dependent variable other than the independent variable [58]. $\varepsilon_t$ is assumed to have mean zero i.e. $\sum_{t=1}^{T} \varepsilon_t = 0$, not to be autocorrelated with the error term at other time points i.e. $\sum_{t=1}^{T} \varepsilon_t \varepsilon_{t-j} = 0 \quad$ for all $j \neq 0$ and to be uncorrelated with the independent variable i.e. $\sum_{t=1}^{T} x_{k,t} \varepsilon_t = 0 \quad$ for all $k$, otherwise the model would be missing some important information. However, since we are dealing with time series, the residuals are likely to be autocorrelated since the observed values are affected by the previous values.

In the case of multiple independent variables, the model is called multiple linear regression and it is defined as

$$y_t = \beta_0 + \beta_1 x_{1,t} + \beta_2 x_{2,t} + \cdots + \beta_k x_{k,t} + \varepsilon_t$$

where $x_{1,t}, x_{2,t}, \ldots, x_{k,t}$ are the independent (numerical) variables and $\beta_0, \beta_1, \ldots, \beta_k$ are the coefficients measuring the marginal effects of the independent variables on the dependent variable [58]. The marginal effect is the change in the dependent variable for a one-unit

change in the independent variable, keeping all other variables constant.

In matrix notation, the model can be written as

$$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

where $\boldsymbol{y} = (y_1, \ldots, y_T)', \boldsymbol{\varepsilon} = (\varepsilon_1, \ldots, \varepsilon_T)', \boldsymbol{\beta} = (\beta_0, \ldots, \beta_k)'$ and

$$\boldsymbol{X} = \begin{bmatrix} 1 & x_{1,1} & x_{2,1} & \ldots & x_{k,1} \\ 1 & x_{1,2} & x_{2,2} & \ldots & x_{k,2} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{1,T} & x_{2,T} & \ldots & x_{k,T} \end{bmatrix}.$$

where $\varepsilon$ has mean $\mathbf{0}$ and variance $\sigma^2\boldsymbol{I}$, $\boldsymbol{I}$ is the identity matrix and $\boldsymbol{X}$ lyes in $\mathbb{R}^{T\times(k+1)}$.

Finding the best values for $\beta_0, \beta_1, \ldots, \beta_k$ is the task of the regression model.

### 2.4.2. Least squares method

The least squares method [73] minimizes the sum of the squared differences between the observed values and the predicted values to find the value of the coefficients, i.e.

$$\sum_{t=1}^{T} \varepsilon_t^2 = \sum_{t=1}^{T} (y_t - \beta_0 - \beta_1 x_{1,t} - \beta_2 x_{2,t} - \cdots - \beta_k x_{k,t})^2$$

[58].

Once the fitted coefficients $\hat{\beta}_0, \hat{\beta}_1, \ldots, \hat{\beta}_k$ are found, the model can be used to make predictions on the data. The fitted value $\hat{y}_t$ is predicted by setting the error $\varepsilon_t$ to zero, i.e.

$$\hat{y}_t = \hat{\beta}_0 + \hat{\beta}_1 x_{1,t} + \hat{\beta}_2 x_{2,t} + \cdots + \hat{\beta}_k x_{k,t}$$

with $t = 1, \ldots, T$ [58].

In matrix form, the least squares method can be written as

$$\varepsilon'\varepsilon = (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta})'(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta})$$

and the solution is given by

$$\hat{\boldsymbol{\beta}} = (\boldsymbol{X}'\boldsymbol{X})^{-1}\boldsymbol{X}'\boldsymbol{y}$$

where $\hat{\boldsymbol{\beta}} = \left(\hat{\beta}_0, \hat{\beta}_1, \ldots, \hat{\beta}_k\right)'$.

Lastly, the fitted values can be computed as

$$\hat{\boldsymbol{y}} = \boldsymbol{X}\hat{\boldsymbol{\beta}} = \boldsymbol{X}\left(\boldsymbol{X}'\boldsymbol{X}\right)^{-1}\boldsymbol{X}'\boldsymbol{y} = \boldsymbol{H}\boldsymbol{y}$$

where $\boldsymbol{H}$ is the hat matrix. For example, if $\boldsymbol{x}^*$ is a new observation of the independent variables, the forecast value is given by

$$\hat{y} = \boldsymbol{x}^*\hat{\boldsymbol{\beta}} = \boldsymbol{x}^*\left(\boldsymbol{X}'\boldsymbol{X}\right)^{-1}\boldsymbol{X}'\boldsymbol{y}$$

[58].

The fitted values obtained from the previous formulations, however, cannot be used to predict the future values of the time series. In the case of ex-ante forecast, we need to predict the future values of the independent variables and then use the predicted values to forecast the future values of the dependent variable. However, if predicting the future values of the independent variables is not possible, we can consider the scenario-based forecast or use as predictor variables the lagged values. This means that if we want to do a $h$-step ahead forecast, we use the predictor variables that are $h$ steps behind the forecast value $y_{t+h}$.

$$y_{t+h} = \beta_0 + \beta_1 x_{1,t} + \cdots + \beta_k x_{k,t} + \varepsilon_{t+h}$$

The fitted coefficients can be used to make predictions $h$ steps ahead.

### 2.4.3. Exponential smoothing

The simple exponential smoothing [17], [57], [120], is a method used to forecast time series when no clear trend or seasonality can be identified. It is a way in between the naïve forecast and the average forecast. The naïve forecast takes as the forecast value the last observed value,

$$\hat{y}_{T+h|T} = y_T$$

while the average forecast takes as the forecast value the average of the observed values

$$\hat{y}_{T+h|T} = \frac{1}{T}\sum_{t=1}^{T} y_t$$

[58].

The exponential smoothing takes as the forecast value a weighted average of the observed values with weights that decrease exponentially as the observations get older. In this way, a greater weight is given to the most recent observations.

$$\hat{y}_{T+1|T} = \alpha y_T + \alpha(1-\alpha)y_{T-1} + \alpha(1-\alpha)^2 y_{T-2} + \cdots$$

where $\alpha$ is the smoothing parameter and it is between 0 and 1 [58]. It requires the choice of the smoothing parameter $\alpha$ and the initial value of the forecast $l_0$.

There are some extensions of the simple exponential smoothing to include the trend and the seasonality. We will not go into the details of these methods.

## 2.5. Learning in a stationary environment: AR, MA, ARMA and ARIMA models

Linear time invariant (LTI) models are used to forecast time series data in a stationary environment. These models are worthwhile since they allow us to find the optimal predictor for stationary time series data. We introduce MA, AR, ARMA, ARIMA and autoregressive nonlinear models taking as main reference [58].

### 2.5.1. MA models

The current value of the series in a moving average (MA) model is assumed to be a linear combination of the past white noise terms. An MA model of order $q$ can be written as

$$y_t = \varepsilon_t + \theta_1\varepsilon_{t-1} + \theta_2\varepsilon_{t-2} + \ldots + \theta_q\varepsilon_{t-q}$$

where $\varepsilon_t$ is the white noise term and $\theta_1, \theta_2, \ldots, \theta_q$ are the coefficients of the model [58]. The output of the MA model is the linear combination of the last $q$ values of the white noise term plus the white noise term at the same time point $\varepsilon_t$ which is normally distributed.

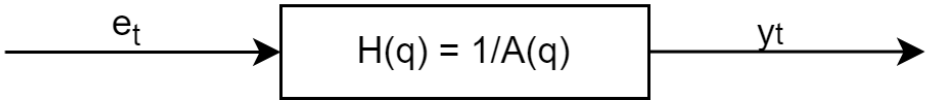



Figure 2.3: MA model

We introduce the term $q^{-k}$ as the backward shift operator, which is defined as follows:

$$q^{-k}x_t = x_{t-k}$$

Using the delay operator, the MA model can be written as

$$y_t = \varepsilon_t + \theta_1 q^{-1}\varepsilon_t + \theta_2 q^{-2}\varepsilon_t + \ldots + \theta_q q^{-q}\varepsilon_t$$

and collecting the $\varepsilon_t$ terms, we have

$$y_t = \varepsilon_t(1 + \theta_1 q^{-1} + \theta_2 q^{-2} + \ldots + \theta_q q^{-q})$$

We obtain

$$y_t = C(q)\varepsilon_t$$

where $y_t$ is the response, $\varepsilon_t$ the input and $C(q)$ is the MA operator defined as follows:

$$C(q) = 1 + \theta_1 q^{-1} + \theta_2 q^{-2} + \ldots + \theta_q q^{-q}$$

(see Figure 2.3).

The transfer function is

$$H(q) = C(q).$$

It has $n$ zeros and all poles at the origin.

The zeros of the transfer function are the roots of the equation $H(q) = 0$, while the poles are the roots of the denominator of the transfer function. In this case, the zeros are the roots of the equation $C(q) = 0$. For a system to be Bounded Input Bounded Output (BIBO) stable, the poles of the transfer function must lie within the unit circle. BIBO stable means that the output of the system is bounded for a bounded input. Since the MA model has all poles at the origin, it is BIBO stable.

It must be noted that the MA model is for forecasting future values, while the MA smoothing cited in the previous sections (subsection 2.3.4) is for estimating the trend of

the past values of the time series.

### 2.5.2. AR models

In the autoregressive (AR) model of order p, the current value of the time series is linearly dependent on its past $n$ values and is summed to a white noise term. An AR model of order $n$ can be written as

$$y_t + a_1 y_{t-1} + a_2 y_{t-2} + \ldots + a_n y_{t-n} = \varepsilon_t$$

where 1 is the intercept, $a_1, a_2, \ldots, a_n$ are the coefficients of the model and $\varepsilon_t$ is the white noise term [58]. The output $y_t$ is, therefore, the linear combination of the last $n$ values of the input $y_t$ plus $\varepsilon_t$ at the same time point.

The AR model is a dynamic response system with a white noise input.

We use the delay operator to analyze the dynamic behavior of the system and to formulate the transfer function. In this way, the AR model can be written as

$$y_t + a_1 q^{-1} y_t + a_2 q^{-2} y_t + \ldots + a_n q^{-n} y_t = \varepsilon_t$$

and collecting the $y_t$ terms, we have

$$y_t(1 + a_1 q^{-1} + a_2 q^{-2} + \ldots + a_n q^{-n}) = \varepsilon_t$$

which can be written as

$$y_t = \frac{\varepsilon_t}{(1 + a_1 q^{-1} + a_2 q^{-2} + \ldots + a_n q^{-n})}$$

where the denominator is the $A(q)$ polynomial. We obtain

$$y_t = \frac{1}{A(q)} \varepsilon_t$$

where $A(q)$ is the AR operator, $y_t$ is the response and $\varepsilon_t$ is the input.

We call $H(q)$ the transfer function of the system, which is defined as:

$$H(q) = \frac{1}{A(q)}$$

(see Figure 2.4).

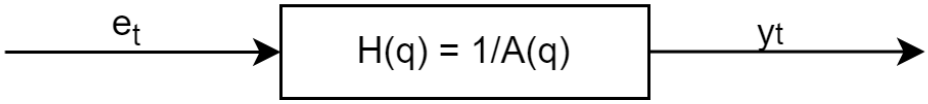



Figure 2.4: AR model

The stability of the discrete-time transfer function depends on the poles of the system. The poles of the transfer function are the roots of the characteristic polynomial of the AR model, which are the roots of the equation $A(q) = 0$. Consequently, the AR model is stable if all the poles lie within the unit circle.

To estimate the parameters of the AR model the maximum likelihood estimation (MLE) is used. The assumptions of the MLE are that the stochastic process $Y$ generating the data is Gaussian and the noise terms are iid Gaussians. It also assumes that data are centered. The parameter that maximizes the likelihood function is

$$\hat{\theta}_{MLE} = \arg\max_{\theta} \log L(\theta|Y) = \arg\max_{\theta} f_{\theta}(Y_1, Y_2, \ldots, Y_T)$$

where $L(\theta|Y)$ is the likelihood function, $f_{\theta}(Y_1, Y_2, \ldots, Y_T)$ is the joint probability density function of the data and $\theta$ are the parameters of the model [58].

The vector autoregression (VAR) model is an AR model used for the multivariate case. In the VAR model, all the variables are treated as endogenous and influence each other. In the case of $m$ variables time series, each variable now becomes a vector of length $m$ $Y_t = (y_{1,t}, y_{2,t}, \ldots, y_{m,t})$, and the VAR model can be written as

$$Y_t + A_1 Y_{t-1} + A_2 Y_{t-2} + \ldots + A_n Y_{t-n} = \boldsymbol{\varepsilon}_t$$

where $I, A_1, A_2, \ldots, A_n$ in $\mathbb{R}^{m \times m}$ are the coefficients of the model and $\boldsymbol{\varepsilon}_t = (\varepsilon_{1,t}, \varepsilon_{2,t}, \ldots, \varepsilon_{m,t})$ is the white noise term.

Similarly to the AR model, the VAR model can be written in matrix form as

$$\boldsymbol{A}(q) Y_t = E_t$$

where $\boldsymbol{A}(q)$ is the AR operator, which is:

$$\boldsymbol{A}(q) = 1 + \boldsymbol{A}_1 q^{-1} + \boldsymbol{A}_2 q^{-2} + \ldots + \boldsymbol{A}_n q^{-n}.$$

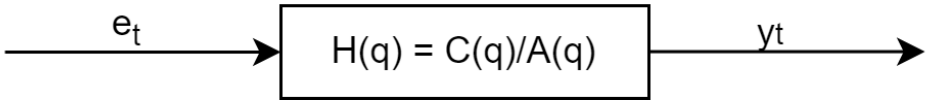



Figure 2.5: ARMA model

### 2.5.3. ARMA models

The autoregressive moving average (ARMA) model is the linear combination of the AR and MA models used to capture both serial correlation and white noise. The stability of the ARMA model primarily depends on the AR component.

The ARMA model of order $p, q$ can be written as

$$y_t + a_1 y_{t-1} + a_2 y_{t-2} + \ldots + a_p y_{t-p} = \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \ldots + \theta_q \varepsilon_{t-q}$$

where $a_1, a_2, \ldots, a_p$ are the coefficients of the AR model, $\theta_1, \theta_2, \ldots, \theta_q$ are the coefficients of the MA model and $\varepsilon_t$ is the white noise term. The AR part is of order $p$ and the MA part is of order $q$. If the signals are expressed at the same time instant using the delay operator, the ARMA model can be written as

$$y_t + a_1 q^{-1} y_t + a_2 q^{-2} y_t + \ldots + a_p q^{-p} y_t = \varepsilon_t + \theta_1 q^{-1} \varepsilon_t + \theta_2 q^{-2} \varepsilon_t + \ldots + \theta_q q^{-q} \varepsilon_t$$

and collecting $y_t$ and $\varepsilon_t$, we have

$$y_t(1 + a_1 q^{-1} + a_2 q^{-2} + \ldots + a_p q^{-p}) = \varepsilon_t(1 + \theta_1 q^{-1} + \theta_2 q^{-2} + \ldots + \theta_q q^{-q})$$

which can be written as

$$y_t = \varepsilon_t(1 + \theta_1 q^{-1} + \theta_2 q^{-2} + \ldots + \theta_q q^{-q})(1 + a_1 q^{-1} + a_2 q^{-2} + \ldots + a_p q^{-p})^{-1}$$

where the numerator is the C(q) polynomial and the denominator is the A(q) polynomial. We obtain

$$y_t = \frac{C(q)}{A(q)} \varepsilon_t$$

where $A(q)$ is the AR operator and $C(q)$ is the MA operator. The operator $C(q)/A(q)$ is called the discrete-time transfer function of the ARMA model (see Figure 2.5).

The zeroes of the transfer function are the roots of the equation $C(q) = 0$ and the poles are the roots of the equation $A(q) = 0$.

If we include in the model an exogenous variable, we obtain the ARMAX model.

### 2.5.4. ARIMA models

The ARIMA model extends the ARMA framework by using differencing to handle non-stationary data. The stationarity is obtained by differentiating the series, and the ARMA models can be used to forecast. If we call $y'_t$, the differenced time series, the ARIMA model can be written as

$$y'_t = a_1 y'_{t-1} + a_2 y'_{t-2} + \ldots + a_p y'_{t-p} + \theta_0 \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \ldots + \theta_q \varepsilon_{t-q}$$

where $a_1, a_2, \ldots, a_p$ are the coefficients of the AR model, $\theta_0, \theta_1, \theta_2, \ldots, \theta_q$ are the coefficients of the MA model and $\varepsilon_t$ is the white noise term [58]. The term $y'_t$ may be the result of multiple differencing. $p$ is the order of the AR model, $q$ is the order of the MA model and $d$ is the order of differencing.

Using the delay operator, the ARIMA model can be written as

$$(1 - a_1 B - a_2 B^2 - \ldots - a_p B^p)(1 - B)^d y_t = (1 + \theta_1 B + \theta_2 B^2 + \ldots + \theta_q B^q)\varepsilon_t$$

where $B$ is the backward shift operator and $1 - B$ is the differencing operator. If we apply the backshift operator to the time series and isolate $y_t$, we obtain a linear combination of the past values of the time series and the white noise term. Using the estimates of the ARIMA model, the future values of the time series for the time step $T + 1$ can be forecast. Then, if we repeat the process feeding the forecast values back to the model, we can forecast the future values for the time step $T + 2$ and so on.

If we want to include some additional terms, the ARIMA model can include also the seasonal component, and it is called the SARIMA model.

The Box-Jenkins approach finds a suitable ARIMA model for the data through an iterative method of three steps [59]. It is advantageous since it is assumed that a time series can always be modeled through an ARIMA or ARMA model depending on its stationarity. The first step is the model selection which aims to find how many parameters should have its model. The second step is parameter estimation in which numerical methods are used to find the best parameters for the model. Lastly, the third step consists of checking whether the model is inadequate, and if this is the case, the procedure is repeated starting from the first iteration.

### 2.5.5. Autoregressive nonlinear predictive models

Artificial neural networks are employed to capture the non-linear relationship between the dependent variable and the independent variables [58]. The simplest case is the neural network autoregression (NNAR) model, which is a feedforward neural network with one hidden layer.

We present here some models used to forecast time series.

The feedforward neural network (FNN) is defined as

$$a_l = \boldsymbol{W}_l^T \boldsymbol{h}_{l-1} + \boldsymbol{b}_l \quad \text{for} \quad l = 1, \dots, L \ , \ h_l = \phi_l(a_l)$$

where $\boldsymbol{h}_0 = x_t \in \mathbb{R}^{n_T}$, $\hat{\boldsymbol{y}}_t = \boldsymbol{h}_L \in \mathbb{R}^{n_O}$, $\phi_l$ is the activation function of the $l$-th layer, $\boldsymbol{W}_l$ is the weight matrix of the $l$-th layer $\in \mathbb{R}^{n_{H,l-1} \times n_{H,l}}$, $n_{H,l}$ is the number of hidden neurons in the $l$-th layer, $\boldsymbol{h}_l$ is the hidden vector of the $l$-th layer $\in \mathbb{R}^{n_l}$ , $\boldsymbol{b}_l \in \mathbb{R}^{n_{H,l}}$ is the bias vector of the $l$-th layer and $L$ is the number of layers, $n_T$ and $n_O$ are the number of input and output neurons.

The objective is to identify the parameters $\boldsymbol{\theta} = \{\boldsymbol{W}_l, \boldsymbol{b}_l\}_{l=1}^L$ to minimize the loss function

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$$

where $\mathcal{L}(\boldsymbol{\theta})$ measures the mean squared error between the predicted values $\hat{\boldsymbol{y}}_t = f(\boldsymbol{x}_t, \boldsymbol{\theta})$ and the true values $\boldsymbol{y}_t$:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{t=1}^{N} (\boldsymbol{y}_t - f(\boldsymbol{x_t}, \boldsymbol{\theta}))^2$$

where $N$ is the number of observations. To prevent overfitting, $\Omega$ can be added to the loss function:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{t=1}^{N} (\boldsymbol{y}_t - f(\boldsymbol{x_t}, \boldsymbol{\theta}))^2 + \Omega(\boldsymbol{\theta})$$

[45]. For example, L2 norm regularization penalty is defined as $\Omega(\boldsymbol{\theta}) = \lambda ||\boldsymbol{\theta}||_2^2$, with $\lambda$ controlling the strength of the regularization.

Other models are the RNN [33], the Long Short-Term memory (LSTM) network [56] and Gated Recurrent Unit (GRU) [22] which are models for sequence learning. These models, differently from FNN, have their own memory. We will go into the details of these models in the next sections.

When dealing with time series, convolutional neural networks (CNN), that exploit the principle of locality, are used with RNNs to learn local temporal features [5], [82].

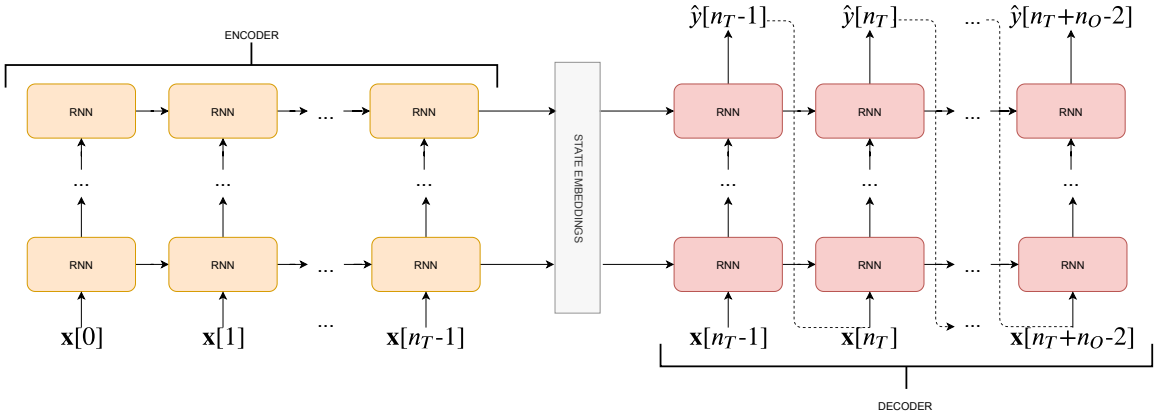



Figure 2.6: Sequence to sequence model.
A seq2seq architecture where the components (the encoder and the decoder) are made of RNNs. In the decoder, during the teacher-forcing training process, the training data follow the solid lines, while, during prediction, the data follow the dotted lines.
*Source: [45]*

Sequence to sequence models are used to predict a sequence of values given of arbitrary length. It is composted by a (deep) neural network (as an RNN) that maps the input history into a state embedding and another (deep) neural network mapping back the state embedding into the output sequence. The encoder is fed with a window of the time series and the decoder is trained to perform a multiple-step ahead forecast. The encoder output is discarded and only the hidden state is recovered to be used as a context vector for the decoder. The decoder is trained to predict the next value of the sequence given the context vector and the previous values of the target sequence. This is a teacher-forcing approach because the decoder is trained to generate the target sequence into itself but a one-time step shifted into the future. The encoder reads the input sequence and outputs a fixed-length context vector representing the input sequence.

$$c = f_{\theta_f}(x_t)$$

where $f_{\theta_f}$ is the last hidden state of the encoder and $c$ is the context vector. The decoder reads the context vector and outputs the predicted sequence.

$$\hat{y}_t = g_{\theta_g}(c)$$

(see Figure 2.6).

Lastly, the transformers are sequence-to-sequence models that are based on a self-attention

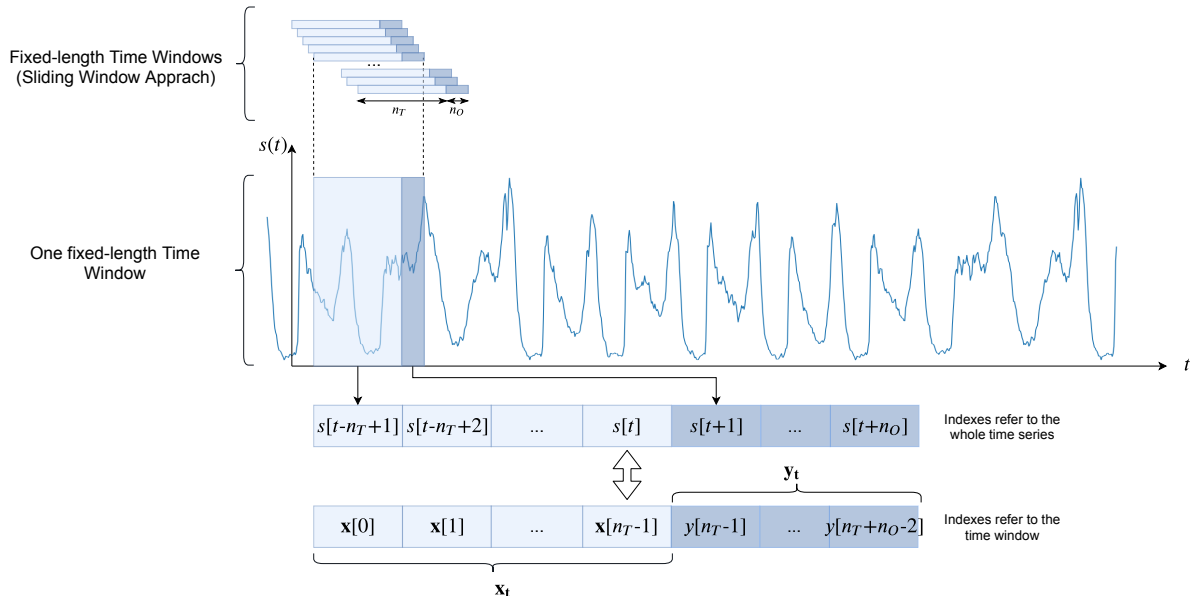



Figure 2.7: Sliding window approach.
The image shows a sliding window on a time series. In this way, we create from the target many *"input-output pairs* $(x_t, y_t) \forall t \in \{nT, nT+1, \ldots, T-nO\}$ *"*, where $nT$ is the number of time steps used to predict the next time step and $nO$ is the number of time steps to predict. This approach enables the use of a supervised machine learning model to learn how to predict future outcomes based on past data.

*Source: [45]*

mechanism and are used for time series forecasting. The input sequence is embedded into a dense vector and is added to the positional encoding. Then it is fed to the encoder which is composed of a stack of identical layers composed by multi-head self-attention mechanism and feedforward neural networks. The decoder has the same structure as the encoder but uses a masked self-attention mechanism to prevent the model from looking into the future and with an additional multi-head self-attention mechanism to take into account the encoder output. Thanks to the multi-head self-attention mechanism, the model can focus on relevant parts of the input sequence in parallel and thanks to the positional encoding, the model can take into account the order of the sequence.

To train a model, the data is divided into a training set and a validation set. In the sliding window approach, input data are presented as windows of length $n_T$ which slide over the time series. The model tries to predict the next $n_O$ value of the time series based on the previous $n_T$ values [45] (see Figure 2.7).

### 2.5.6. Multi-step prediction stategies

Multi-step time series forecasting can be defined with the following. Given a window of $W \geq 1$ past values $X_{t-W:t} = [X_{t-W}, \ldots, X_{t-1}]$, the aim is to predict the $H \geq 1$ future observations $X_{t+h}$ with $h = 1, \ldots, H$ [26].

We present here three methods to forecast multiple step-ahead for time series [108].

The first method is the Recursive strategy, where the model is trained to perform a one-step-ahead forecast. The forecast value is then, recursively fed back to the model to forecast the next value assuming it is the true value. The resulting output is built concatenating the sequence of $n_O$ forecast values.

The second method is the Direct strategy, where $n_O$ models $f_k, k = 1, \ldots, n_O$ are trained independently to forecast the next $t + k$-th value. Each model shares the same input and outputs just one value. Again, the output is built by concatenating the sequence of $n_O$ forecast values.

The third method is the multiple input multiple output (MIMO) strategy, where one single model is trained to forecast multiple steps ahead in one shot. The output will hence be a sequence of $n_O$ forecast values.

When doing multi-step time series forecasting, there are different ways to transform back the data after differencing: it can be done in a teacher-forcing way or in a recursive way. When the predicted values are brought back to the original scale, if the horizon is multi-step, the teacher-forcing approach implies summing to each step the previous time step in the target sequence, while the recursive approach implies summing the previous time step in the predicted sequence. The teacher-forcing is an unbiased way to train the model, but it cannot be used during testing because we don't have access to the ground truth labels, which can be used just to measure the error, not to predict the future. However, this can lead to a mismatch between the model's behavior during training and its behavior during inference because the model during testing has no more access to the true values. The recursive approach, on the other hand, can accumulate errors, especially if there is strong non-stationarity and many time steps need to be predicted. Otherwise, it can be used in a hybrid approach, where the model is trained with teacher-forcing on some data and then gradually transitioned to the recursive approach, which is then used for testing.

### 2.5.7. Learning in a non-stationary environment

In most real-world scenarios, time series data are non-stationary. In this case, we need to update our stream-based application to track changes in the process. If the distribution changes over time, we need to adapt the model to the new distribution or retrain the model from scratch. In some cases, we don't need to drastically change the family of models but it is enough to update the parameters of the model.

We present here three methods to update the model in a non-stationary environment: instance selection, instance weighting and multiple models ensemble.

The instance selection method is used to select the most relevant instances to update the model. That means that we select the most recent data to update the model. The length of the window can be fixed or can be adaptive to the data with a heuristic approach.

In the weighting method [79], instead of removing the old data, we assign a weight to each instance based on its relevance. The weight can be chosen based on the recency or relevance of the data.

Lastly, in the multiple models ensemble method [91], we train multiple models on different windows of the data. The outputs of an ensemble of models are combined to obtain the final forecast through a weighted average or a majority vote. It includes dynamical systems to integrate new models and remove obsolete ones.

Since retraining the whole model or its subpart can be computationally expensive, it can be decided to employ an incremental approach. This approach consists of updating the model depending on the change in dynamics. There are two types of incremental learning: the passive learning and the pro-active learning.

In passive learning, the model continuously updates the parameters based on the prediction done and then processes the new data. We call online learning the case where the model is updated in real-time as each sample of data point arrives. We call batch learning if instead of updating the model at each new data point, the model is updated at the end of each batch of data points.

In proactive learning, the model is updated only when the probability density function of the data changes. This change is detected by a feature-based check or by a statistical test.

In a broader view, when learning in a non-stationary environment, we can aim at a lifelong learning approach. The DARPA lifelong learning machines program [111] aims to develop algorithms that can learn and adapt continuously from a stream of data and that can

apply the learned skills and knowledge to new tasks.

## 2.6. Evaluation metrics

The forecast error is measured as the difference between the observed value and the predicted value in the test set:

$$e_{T+h} = y_{T+h} - \hat{y}_{T+h|T}$$

, where $T$ is the last time point in the training set and $h$ is the step ahead [58]. In the next sections, we present the categorization and definitions of some metrics following as main reference [58].

### 2.6.1. Scale-dependent metrics

Scale-dependent metrics are metrics that depend on the scale of the data, such as the mean absolute error (MAE), the mean square error (MSE) and the root mean squared error (RMSE). The MAE is the average of the absolute values of the forecast errors, the MSE is the average of the squared forecast errors and the RMSE is the square root of the MSE.

We report here the definition of the MSE and MAE:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

where $y_i$ is the true value and $\hat{y}_i$ is the predicted value [58]. In a general case, since the MSE makes big errors more significant and small errors less significant, it is more sensitive to outliers than the MAE. Consequently, MSE can be used when we want to penalize a lot of big errors. In our experiment, MAE will be used as the loss function to minimize and both MSE and MAE will be used as metrics to evaluate the performance on the test set.

### 2.6.2. Percentage errors

While the scale-dependent metrics are on the same scale of the data, the percentage errors can be used to make a comparison between the errors on different datasets. Two percentage errors are the mean absolute percentage error (MAPE) and the Symmetric Mean Absolute Percentage Error (SMAPE). The MAPE is the average of the absolute values of the percentage errors, i.e.

$$\text{MAPE} = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i} * 100\right|$$

which is equal to

$$\text{MAPE} = \text{mean}\left(|p_t|\right)$$

with $p_t = 100e_t/y_t$ . The MAPE is undefined when the true value is zero.

The Symmetric Mean Absolute Percentage Error is also the average of the absolute values of the percentage errors, but the absolute values are divided by the sum of the true values and the predicted values, i.e.

$$\text{SMAPE} = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{\left(y_i + \hat{y}_i\right)/2} * 100\right|$$

[58]. It was proposed by [101] to address the issue of the MAPE to put a higher penalty on negative errors.

### 2.6.3. Relative and absolute errors

The Mean Absolute Scaled Error (MASE) measures the performance of the model compared to the naive forecast that uses the last value of the time series. The MASE is defined as

$$\text{MASE} = \frac{\frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|}{\frac{1}{n-1}\sum_{i=2}^{n}|y_i - y_{i-1}|}$$

[58]. The MASE could, hence, be defined as the ratio between the MAE of the model and the MAE of the naive forecast.

The Mean Relative Error (MRE) is the ratio between the MAE of the model and the

absolute value of the true value. It is defined as

$$\text{MRE} = \sum_{i=1}^{n} \frac{|y_i - \hat{y}_i|}{|y_i|}.$$

## 2.7. Challenges of time series forecasting

We present here some of the challenges of time series forecasting, including the one of forecasting long-term time series data, which is the main focus of the thesis.

### 2.7.1. Outliers

Outliers are values in the dataset that are considerably different from the other values in the time series [58]. They can be caused by errors in the data collection process, or by the process itself, or can reflect a rare event. If we are able to understand the cause of the outliers, and they are caused by errors, they can be removed from the data and can be replaced by missing values. In this way, the model is likely to perform better than having outliers. Another method implies replacing them with an estimate closer to the other values in the time series.

### 2.7.2. Missing values

A time series can have missing values due to errors in the data collection, in the data transfer, in the hardware or software, or for irregular samplings, or even because sensors were activated in different moments [26]. As well as for outliers, missing values can be treated and substituted using different methods according to their distribution in the time series and according to the specific cases [58].

### 2.7.3. The problem of long-term time series forecasting and possible approaches

When we need to forecast many steps into the future, we encounter additional challenges.

Statistical methods such as AR, ARMIA and ES, are not able to model complex non-linear relationships between the variables when performing long-term multivariate time series forecasting.

If we are using Machine Learning models, the performance will likely decrease as we forecast further into the future.

RNNs are able to model sequences and non-fixed-length inputs and for this reason are suited for time series forecasting [80], [51], [68], [127]. One challenge with RNNs is their memory management, particularly in scenarios with varying dynamics, where in some cases, we need to remember the past for a long time and in other cases for a short time.

RNN can be used to forecast multiple steps ahead, however, they are computationally expensive if the input sequence is long. Indeed the RNN needs to process the entire input sequence in order before making a prediction.

When we use nonpersistent state RNN in a sliding-window approach, we implicitly assume that the whole information we need to make a prediction is inside the window of the input sequence. Thus, it follows that the windows of the input sequence can be sampled as they were iid and the model uses only the time steps in the input sequence of each window to make predictions. This approach enables subsampling reducing computational complexity.

On the other hand, when we use persistent state RNN, we cannot work with only a single window of the input sequence at a time, but we need to process the whole sequence because the hidden state must be passed from one batch to the next.

The idea behind the thesis, which will be expanded in chapter 4, is to use a reservoir that encodes the past sequences without weighing on the training of the model, followed by an RNN that is trained only on the most recent data. This approach enables subsampling reducing computational complexity and allows us to overcome some limitations related to GPU memory capabilities.

Transformers can be used for time series forecasting since they can capture long-term dependencies [55]. Indeed, the self-attention mechanism enables the model to access historical observations at any time step in the past so that the network can focus on parts of the inputs that are important for the current output step. However, they calculate the dot product self-attention, which is computationally expensive and can be a bottleneck when forecasting long-term time series data. A solution to this problem is sparse versions of self-attention, which can reduce memory consumption and improve the efficiency [2] [113] [69].

Some studies claim shown that transformers have poorer performances than simple linear-centric models in long-term multivariate time series forecasting [90] [78] [124]. Indeed, as claimed in [124], transformers use self-attention which is *"permutation invariant and 'anti-order'"*. Indeed, even if the positional encoding can introduce a sort of order in the sequence, the self-attention mechanism will cause some loss of temporal information. Transformers indeed perform well in learning the semantic correlations among the ele-

ments in a sequence, but in the context of time series forecasting, we are interested rather in learning the temporal dependencies in an ordered time series [124].

Linear-centric models have a single linear layer and it is combined with non-linear preprocessing and postprocessing. However, linear models, given their simplicity, are not able to capture complex inter-dependencies among various time steps in multivariate time series and they don't exploit the temporal features present in the sequences, including patterns such as seasonality and trends [55]. Examples of linear-centric models are the DLinear and NLinear models and we will describe them in the next sections subsection 3.3.1.

A solution might be to use ensemble models which have already shown good performance in time series forecasting tasks such as traffic forecasting [32], financial forecasting [2], energy management [105] [44], and medical applications [1] [43].

We will present our solution to the problem of long-term time series forecasting in chapter 4.

# 3 | Related work

In this section, we will introduce the concept of reservoir computing (RC) which is an important component of the new framework introduced in the next section. Since the RC is a type of RNN, we will start by introducing the concept of RNNs. Then, we will introduce the concept of ESNs and we will explain the advantages of its training procedure. Lastly, we will extend the concept of ESNs to ESNs.

## 3.1. Recurrent neural networks

There are two main types of neural networks: feedforward neural networks (FNNs) and RNNs (RNNs). In FNNs, the information flows in one direction, from the input layer to the output layer, and there are no cycles in the network (see Figure 3.1).

### 3.1.1. Architecture

RNNs are dynamical and non-linear systems that have a state vector and might have some hidden layers that are modified according to the current state and inputs [99]. RNN can handle varying-size input sizes by processing it sequentially and retaining the state between inputs. RNNs allows previous outputs to be used as inputs while having hidden

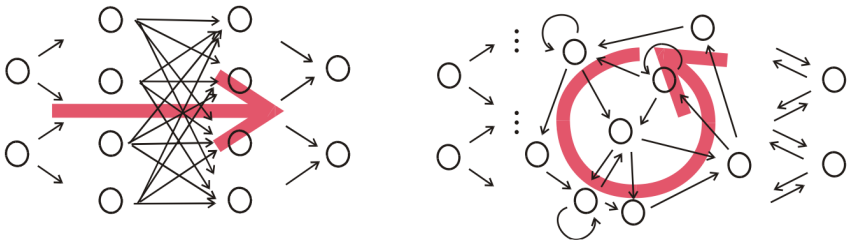


Figure 3.1: Comparison between FNNs and RNNs.
On the left, there is a FNN where the information flows in one direction, and on the right a RNN which contains cycles.

*Source: [62]*

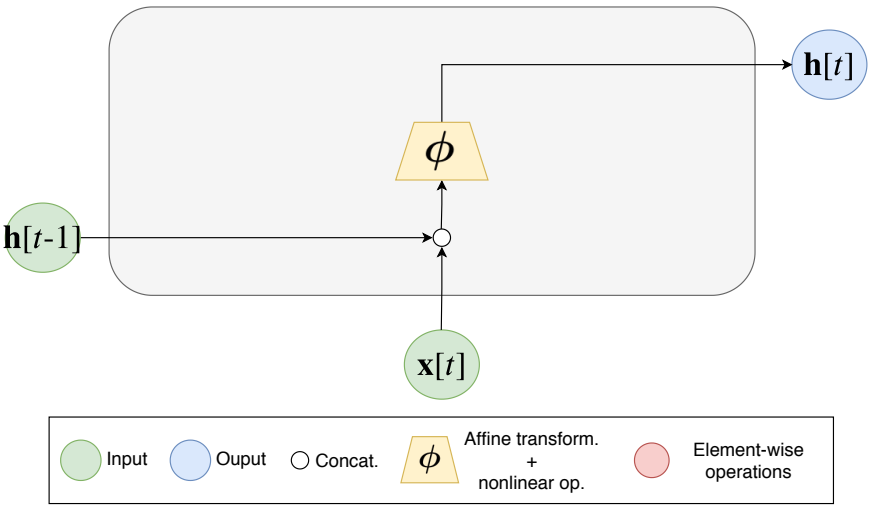



Figure 3.2: A schematic representation of a cell of an ERNN block.
*Source: [45]*

states that can store information about the past. memory in RNNs is implemented by feedback connections.

Elman RNN implements the following equation:

$$\mathbf{a}[t] = \mathbf{W}^T\mathbf{h}[t-1] + \mathbf{U}^T\mathbf{x}[t]$$
$$\mathbf{h}[t] = \phi(\mathbf{a}[t])$$
$$\mathbf{y}[t] = \psi\left(\mathbf{V}^T\mathbf{h}[t]\right)$$

where $\mathbf{x}[t]$ is the input at time t, $\mathbf{h}[t]$ is the hidden state at time t, $\mathbf{y}[t]$ is the output at time t, $\mathbf{W} \in \mathbb{R}^{n_H \times n_H}$ is the weight matrix of the internal connections, $\mathbf{U} \in \mathbb{R}^{d \times n_H}$ is the weight matrix of the input to hidden connections, $\mathbf{V} \in \mathbb{R}^{n_H \times n_O}$ is the weight matrix of the hidden to output connections, $\Phi()$ is the activation function of the internal units, and $\Psi()$ is the activation function of the output units, generally linear [45] (see Figure 3.2).

### 3.1.2. Training

In our experiments, we will use supervised learning to train the RNNs. The supervised learning approach consists of using teacher data to make the model lean [62]. The training is done by minimizing a loss function that measures the difference between the predicted output and the target output. The goal is to make the network learn to generalize new data without overfitting the training data.

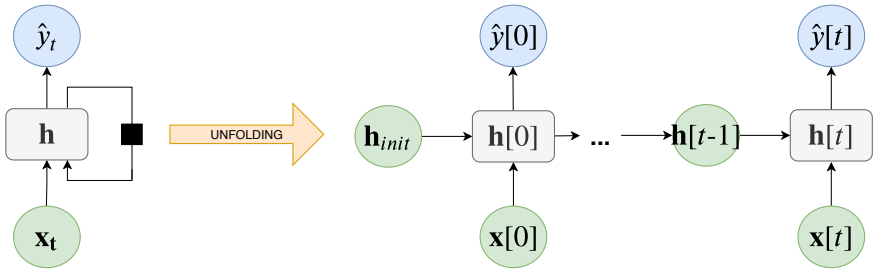



Figure 3.3: Back Propagation Through Time in RNNs.
An illustration of BPTT obtained as an unfolding of a standard RNN. On the left is an RNN with a single input where the delay operator is represented by a black box. On the right is the network obtained by unfolding the RNN, resembling a deep feedforward neural network where each layer has the same weights. The initial state of the network, $h_{init}$, is usually initialized to zero.

*Source: [45]*

The loss function $\mathcal{L}$ is computed as the sum of the loss at each time step:

$$\mathcal{L}(\hat{y}, y) = \sum_{k=1}^{T} \mathcal{L}(\hat{y}[k], y[k]).$$

The Backpropagation Trough time (BPTT) [118], [98] is an extension of the backpropagation algorithm that is used to train RNNs.

Intuitively, BPTT is equivalent to applying the Backpropagation to the RNN unfolded in time obtained by stacking identical copies of the network, that share the same weights, one for each time step [45]. The unfolded network is a feedforward neural network and the backpropagation algorithm can be applied to train the network (see Figure 3.3).

RNN suffers from the vanishing and exploding gradient problem [8],[94]. This is due to the fact that the gradient of the loss function is computed by backpropagating the error through time which is done by recursively applying the chain rule to compute the gradients at each time step. To address this issue, the Truncated Back Propagation Through Time (TBPTT) [119] truncates the backpropagation at a fixed number of time steps preventing the gradients from being computed over a long sequence of time steps. Moreover, exploding gradients can be handled by gradient clipping, which consists of putting a ceiling on the maximum value for the gradient.

### 3.1.3. LSTM and GRU

LSTM and GRU models are extensions of RNNs that address the problem of vanishing gradient and short-term memory when processing long sequences [45].

An LSTM layer contains various LSTM cells that are used to control the information flow. One LSTM cell is made of one input gate, one output gate and one forget gate. The input gate controls if the state is updated with external input, the output gate controls if the state is observable from the outside, and the forget gate allows the resetting of the state. The cell state is the memory of the LSTM and the hidden state is the output of the LSTM. The cell state addresses the short-term memory problem of the RNNs by allowing the network to remember the information for a longer period of time and the gates regulate what information should be remembered and what information should be forgotten by the cell state. The cell state is updated with a pointwise multiplication with the output of the forget gate and with an addition with the output of the input gate which updates the cell state with the relevant information. If the forget vector has a value near zero, the cell state is reset. The hidden state is the result of the output gate applied to the cell state and to the previous hidden state. Since the gate controls gradient flow by potentially keeping the memory unaltered for many time steps, the LSTMs solve the vanishing gradient problem. The cell state $\mathbf{c}[t]$ and the network's output $\mathbf{h}[t]$ at time step $t$ are obtained from the forget gate $\mathbf{f}[t]$, the input gate $\mathbf{i}[t]$ and the output gate $\mathbf{o}[t]$ follwing the equations:

$$
\begin{aligned}
\mathbf{i}[\mathbf{t}] &= \psi\left(\mathbf{W_i}\mathbf{h}[t-1] + \mathbf{U_i}\mathbf{x}[t]\right) \\
\mathbf{f}[\mathbf{t}] &= \psi\left(\mathbf{W_f}\mathbf{h}[t-1] + \mathbf{U_f}\mathbf{x}[t]\right) \\
\mathbf{o}[\mathbf{t}] &= \psi\left(\mathbf{W_o}\mathbf{h}[t-1] + \mathbf{U_o}\mathbf{x}[t]\right) \\
\widetilde{\mathbf{c}}[\mathbf{t}] &= \phi\left(\mathbf{W_c}\mathbf{h}[t-1] + \mathbf{U_c}\mathbf{x}[t]\right) \\
\mathbf{c}[\mathbf{t}] &= \mathbf{f}[t] \odot \mathbf{c}[t-1] + \mathbf{i}[t] \odot \widetilde{\mathbf{c}}[t] \\
\mathbf{h}[\mathbf{t}] &= \mathbf{o}[t] \odot \phi(\mathbf{c}[t])
\end{aligned}
$$

where $\mathbf{W_f}, \mathbf{W_i}, \mathbf{W_o}, \mathbf{W_c} \in \mathbb{R}^{n_H \times n_H}$, $\mathbf{U_f}, \mathbf{U_i}, \mathbf{U_o}, \mathbf{U_c} \in \mathbb{R}^{n_H \times d}$ are trainable parameters, $d$ is the number of features of the input, $\odot$ is the Hadamard product, $\psi(\cdot)$ is generally a sigmoid activation while $\phi(\cdot)$ can be any non-linear one [45] (see Figure 3.4).

GRUs do not have a cell state and they have a reset gate and an update gate only. The reset gate decides how much of the past information to forget and the update gate chooses what to forget and what new information to add. The update $\mathbf{u}[t]$ and reset $\mathbf{r}[t]$ gates are

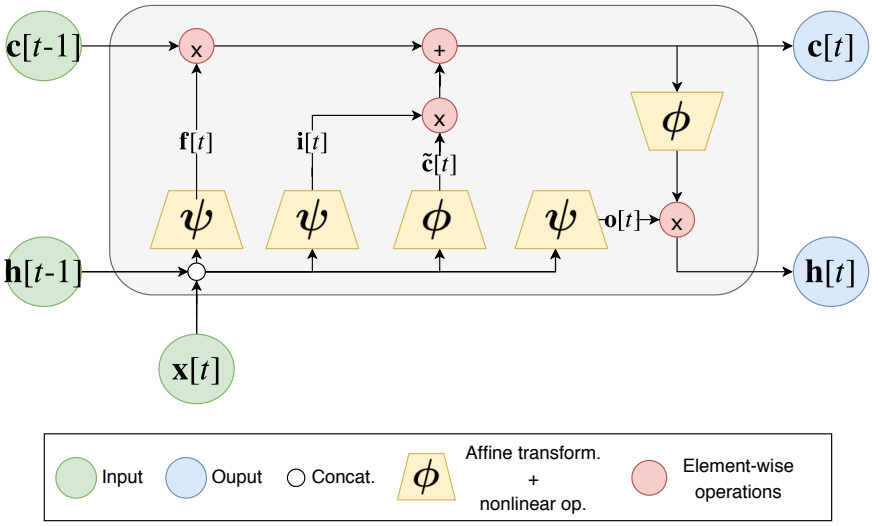



Figure 3.4: A schematic representation of a cell of an LSTM block.
*Source: [45]*

used in this way to update the hidden state $\mathbf{h}[t]$:

$$\begin{aligned}
\mathbf{u}[\mathbf{t}] &= \psi\left(\mathbf{W_u}\mathbf{h}[t-1] + \mathbf{U_u}\mathbf{x}[t]\right) \\
\mathbf{r}[\mathbf{t}] &= \psi\left(\mathbf{W_r}\mathbf{h}[t-1] + \mathbf{U_r}\mathbf{x}[t]\right) \\
\widetilde{\mathbf{h}}[\mathbf{t}] &= \phi\left(\mathbf{W_c}(\mathbf{r}[t] \odot \mathbf{h}[t-1]) + \mathbf{U_c}\mathbf{x}[t]\right) \\
\mathbf{h}[\mathbf{t}] &= \mathbf{u}[\mathbf{t}] \odot \mathbf{h}[t-1] + (1 - \mathbf{u}[t]) \odot \widetilde{\mathbf{h}}[t]
\end{aligned}$$

where $\mathbf{W_u}, \mathbf{W_r}, \mathbf{W_c} \in \mathbb{R}^{n_H \times n_H}$, $\mathbf{U_u}, \mathbf{U_r}, \mathbf{U_c} \in \mathbb{R}^{n_H \times d}$ are the trainable parameters, $\psi(\cdot)$ is the activation function while $\phi(\cdot)$ is a non-linear function [45] (see Figure 3.5).

The LSTM, having an additional gate, the forget gate, and hence more parameters, is more prone to overfitting than the GRU and is generally slower to train with respect to the GRU.

We also mention that Bidirectional LSTM is used when the output sample depends on the entire sequence and not just from the past frames. The input, in this case, is fed into two identical parallel LSTMs, one in forward order and the other in backward order.

Deep RNNs, obtained by stacking multiple RNNs [100], are usually able to capture better temporal hierarchy in time series data. The architecture of a deep RNN is given by:

$$\mathbf{h}_\ell[t] = f\left(\mathbf{h}_\ell[t-1], \mathbf{h}_{\ell-1}[t]; \mathbf{\Theta}\right) \quad \ell = 1, 2, \ldots, L$$

where $\mathbf{h}_\ell[t]$ is the hidden state of the $l$-th layer at time $t$, $f()$ is the activation function of

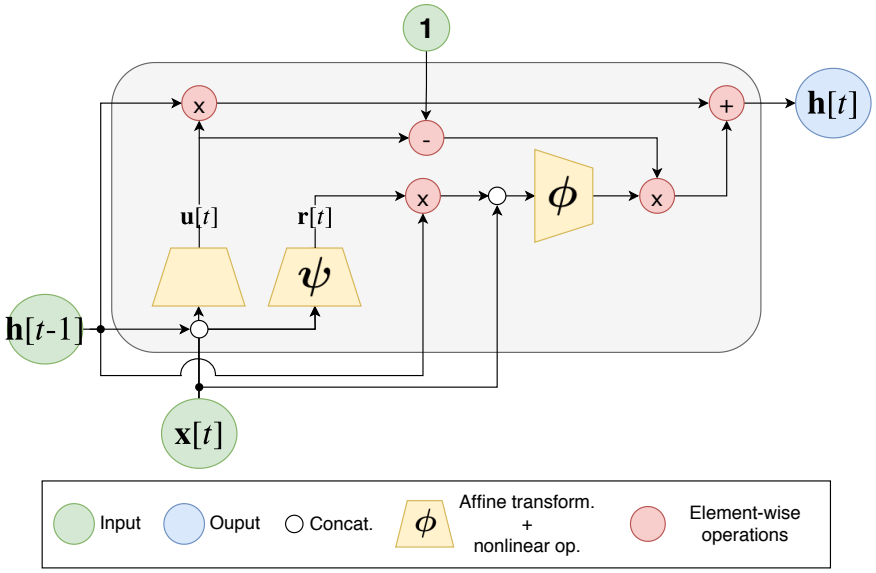



Figure 3.5: A schematic representation of a cell of a GRU block.
*Source: [45]*

the internal units and $\boldsymbol{\Theta}$ is the set of parameters to be learned.

## 3.2. Reservoir computing

### 3.2.1. Randomization in neural networks

In the context of learning, accuracy may not always be the only objective [36]. Indeed, in some contexts, there may be limitations in the hardware devices, or the task may have more stringent time and efficiency requirements. For example, it may be the case for distributed intelligence applications on low-powered edge devices. In such cases, fully trainable Deep Neural Networks may not be the most optimal choice.

Deep learning has two fundamental components: architectural bias and training algorithms [36]. In the context of Deep Neural Networks, there is the issue of over-parametrization of the models, which is a waste of efficiency and resources. Randomized networks can be a solution to this problem. Indeed, since randomized neural networks mainly rely on architectural biases, it can be avoided much of the complexity given by back-propagation training of fully trainable DNNs. Indeed, as demonstrated by [97], it is possible to fix up to 90% of the parameters in a deep convolutional neural network and train only the remaining 10%, and only an insignificant drop in accuracy will be measured in different scenarios.

Making use of the architectural bias only, randomized neural networks can outperform the accuracy of standard machine learning algorithms [42]. Indeed, traditional machine learning algorithms often optimize parameters that may not necessarily need fine-tuning.

A significant difference between the reservoir approach and the trainable system is that in the former case, the system must be designed to capture the desired dynamics since it doesn't learn autonomously [36].

### 3.2.2. Deep randomized feed-forward neural networks

Adding layers to a fully trainable neural network doesn't change the nature of the optimization problem. However, it is not trivial to extend this concept for the randomized neural networks [39].

A kernel function is uniquely linked to the "skeleton" of the network, which is the feed-forward network's topological structure without taking the weights into account [31]. These kernel functions can generally be approximated by sampling weights from a Gaussian distribution [30]. Hence, even with random weights, the network structure could capture and represent the underlying kernel [39].

As shown in [47], for DNN with random Gaussian weights, at the beginning of the training the state vector for different classes lies very close to each other and during training, they move further away. The representation function creates embeddings in which angles associated with points in the same cluster are reduced more than points in the distinct clusters. Given this observation, as stated in [47], for certain applications, randomized networks can be used in the initial layers to separate points with distinguishable angles so that it becomes easier to classify them. Similarly, trained weights in the deeper layers can further discriminate between the remaining points. This aspect applies to networks with random Gaussian weights and ReLU or analogous activation functions that have the ability to make the network sensitive to the angles between inputs [39].

Even if, post-training, a significant percentage of weights in a deep network is considered useless and can be removed [34], pruning the architecture before training and training only a small network remains challenging. The lottery ticket hypothesis [34] suggests that only small subsets of weights (tickets) of deep networks are responsible for good performance and the larger the network, the more likely effective weights are found. However, a clear connection with fully randomized networks has not been found.

A potential gap in research, identified in [39], concerns the lack of a comprehensive understanding of the accuracy and performance trade-offs of deep RVFL [93] networks for

large-scale problems. RVFL are feedforward neural networks characterized by a single, non-trainable hidden layer with random input weights, and training is applied only to the output layer. Given that this paradigm represents the simplest form of a randomized network, this underscores a gap not only in the RVFL domain but also in the broader reservoir Computing (RC) framework.

Moreover, there exists also hybrid forms of neural networks that combine fully trained and randomized components, known as semi-random neural networks,

### 3.2.3. Echo state networks

Here, we offer a concise overview of related works in reservoir Computing that support our approach and the direction we are moving towards. Most references are taken from [36] and [39].

Reservoir computing (RC) [83], [40] is founded on RNNs and employs the dynamics of a fixed, non-linear system named a reservoir to project input signals into higher dimensional spaces[109]. The reservoir is made by a certain number of recurrent hidden layers that are left untrained after being initialized. The dynamical system is initialized under certain proprieties, such as the Echo State Property [63] which ensures that the system is stable, non-chaotic asymptotically. The training process is then confined to the readout part of the network. Its recurrent connections allow for a memory effect, which is a key feature for processing time series data. The reservoir typically has a substantial number of non-linear neurons, and its role is essentially to expand the input history into a high-dimensional rich feature space. In this space, the original learning problem can be more easily approached. Indeed, the internal dynamics of the reservoir create a richer representation of the input data through the values of the neurons' activations which forms a state vector. The state vector is a high-dimensional representation of the input signal which encodes information about the input data and the dynamics of the reservoir.

This approach accelerates the training process achieving great speedup. Indeed, training RNNs, particularly due to back-propagation through time, can be slow and resource-intensive, often requiring high-performance computing support.

reservoir computing is used either as priors due to architectural biases or as generic feature extractors. However, is studied whether they can be used as generic learning models.

ESNs is a type of reservoir computing. The ESN is composed of the reservoir followed by a Readout. The Readout is composed of a set of weights that are trained to map the reservoir's state to the output. An ESN is a discrete-time neural network composed by

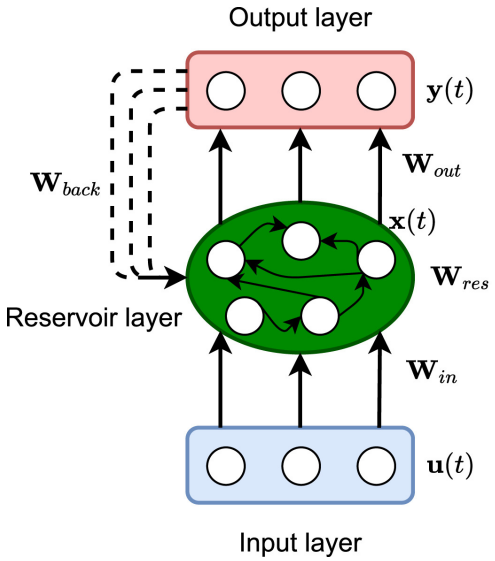



Figure 3.6: Echo State Network.
An ESN of one reservoir layer with the corresponding input-to-reservoir, reservoir-to-reservoir, reservoir-to-output connections and optional output-to-reservoir connections.
*Source: [77]*

$N_r$ nodes having a non-linear activation function $f()$, an input at time k $\mathbf{x}[\mathbf{k}]$ of size $N_i$, and an output $\mathbf{y}[\mathbf{k}]$ of size $N_o$ (see Figure 3.6).

We denote the input-to-reservoir weight matrix as $\boldsymbol{W}_i^r \in R^{N_r \times N_i}$ the reservoir internal connections as $\boldsymbol{W}_r^r \in R^{N_r \times N_r}$, the output-to-reservoir weight matrix as $\boldsymbol{W}_o^r \in R^{N_r \times N_o}$, reservoir-to-output weight matrix as $\boldsymbol{W}_r^o \in R^{N_o \times N_r}$ and input-to-output weight matrix as $\boldsymbol{W}_i^o \in R^{N_o \times N_i}$.

The output-to-reservoir weight matrix is used to connect back the output to the reservoir to the internal units, allowing the reservoir to have a feedback loop. The weight matrices are initialized with random values usually from a uniform distribution or from a Gaussian distribution in the range $[-1, 1]$ [12].

The activation scheme of the internal nodes at the time step k is defined by the following equation:

$$\begin{aligned} \mathbf{h}[k] &= f\left(\mathbf{W}_r^r \mathbf{h}[k-1] + \mathbf{W}_i^r \mathbf{x}[k] + \mathbf{W}_o^r \mathbf{y}[k-1]\right) \\ \mathbf{y}[k] &= g\left(\mathbf{W}_r^o \mathbf{h}[k] + \mathbf{W}_i^o \mathbf{x}[k]\right) \end{aligned} \tag{3.1}$$

[12]. The activation function of the internal units $f()$ is typically a sigmoidal function and the output function of the output units $g()$ is typically the identity function. The readout network is made by the weights of the connections between the reservoir and the output layer and between the input and the output layer and is usually trained to solve

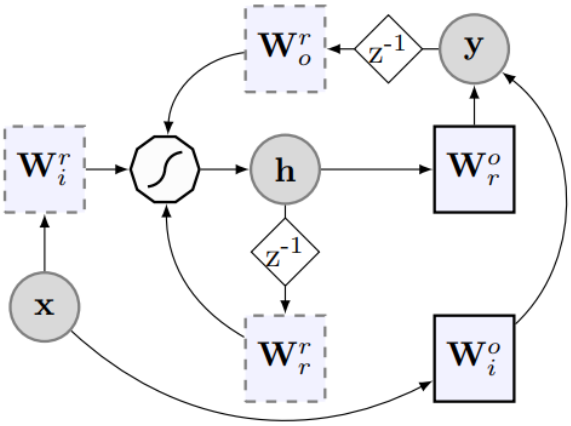



Figure 3.7: Representation of the ESN architecture.
An overview of the ESN architecture where $x$ is the input, $h$ the hidden state, $y$ the output, $\boldsymbol{W}_r^r$, $\boldsymbol{W}_r^0$ and $\boldsymbol{W}_r^i$ are train-less components, $\boldsymbol{W}_o^r$ and $\boldsymbol{W}_o^i$ the trainable components of the readout, $z^{-1}$ is the unitary shift operator and the remaining symbol is the sigmoidal function.

*Source: [12]*

a least square problem (see Figure 3.7) [12].

The generalization capabilities of the reservoir mainly depend on the high number of processing units in the recurrent layer, the sparsity of the recurrent connections, and the spectral radius of the recurrent weights matrix [75], [10].

Given the randomness of the reservoir, the same input sequence will produce different reservoir states if the reservoir is initialized with different random weights. During our experiments, we will fix some seeds to keep into account this aspect.

## Echo state property

During the initialization of reservoir computing neural networks, matrices are randomly initialized. The hidden layer is sparsely connected to make computations faster. To ensure global asymptotic stability, we must impose the "Echo State Property" [123], [60] [87] to the state transition function.

The echo state property guarantees that the trajectories of any two distinct initial states will converge if they come from a compact collection [12]. We report here the definition

of echo state property given in [62]:

**Theorem 1.** *(Echo states) Assume an untrained network with weights* $\boldsymbol{W}^{in}, \boldsymbol{W}$ *and* $\boldsymbol{W}^{back}$ *is driven by teacher input* $\boldsymbol{u}(n)$ *and teacher-forced by teacher output* $\boldsymbol{y}(n)$ *from compact intervals* $U$ *and* $Y$*, if for every left-infinite input/output sequence* $(\boldsymbol{u}(n), \boldsymbol{y}(n-1))$*, where* $n = \ldots, -2, -1, 0$*, and for all state sequences* $\boldsymbol{x}(n), \boldsymbol{x}'(n)$ *compatible with the teacher sequence, e.g. with*

$$x(n+1) = f\left(W^{in}\, u(n+1) + W x(n) + W^{back}\, y(n)\right)$$
$$x'(n+1) = f\left(W^{in}\, u(n+1) + W x'(n) + W^{back}\, y(n)\right)$$

*it holds that* $\boldsymbol{x}(n) = \boldsymbol{x}'(n)$ *for all* $n \leq 0$*.*

This means that the network state, after enough time, becomes independent of the initial state and depends only on the input and output history.

Even if the weight matrices are initialized with random values, their values can be manipulated and scaled to obtain a desired property. The spectral radius of the reservoir weight matrix is often used to control the stability of the reservoir and the memory of the system. A sufficient condition for the non-existence of the echo state property is that the spectral radius of the reservoir weight matrix is bigger than 1. Although it may happen that echo state property is obtained also violating this condition or that is not obtained under this condition (even if it doesn't occur in practice) [83]. Practically, to obtain this condition, the spectral radius of the recurrent weight matrix is often rescaled so that it remains smaller than 1.

We report here the proposition given in [60], which is a sufficient condition for the non-existence of the echo state:

**Proposition 3.1.** *Let the weight matrix have a spectral radius* $|\lambda_{\max}| > 1$*, where* $\lambda_{\max}$ *is an eigenvalue of* $\mathbf{W}$ *with the largest absolute value. Then the network has an asymptotically unstable null state. This implies that it has no echo states for any input set* $U$ *containing* $\mathbf{0}$ *and admissible state set* $A = [-1, 1]^N$

where $\mathbf{W}$ is the reservoir weight matrix (see $\mathbf{W}_r^r$ in Equation 3.1). The other two matrices of Equation 3.1 are not required to have a spectral radius smaller than 1 since are not involved in establishing the echo state property.

However, for certain tasks, the best performance of some systems is obtained when their dynamics are on the edge of chaos [74]. This justifies the use of reservoirs with a spectral radius greater or equal to 1, which are on the edge of stability, where the system can be

chaotic and unstable.

Another condition, sufficient and more stringent, is ensured if the largest singular value of the recurrent weight matrix is smaller than one [83].

## Training procedure

RC models have the same architecture as Elman RNNs but a different training approach. RC has the advantage to allow to arrange the training as a linear regression, which can be solved in a closed form with the least-squares algorithms.

[60] reports a step-by-step training procedure. First, we need to build the reservoir weights $\mathbf{W_r^r}$ with echo state property following, for example, the method of the spectral radius reported in the previous section. Then, we randomly generate $\mathbf{W_r^i}$ and $\mathbf{W_o^r}$. For the training phase, we initialize with an arbitrary state $\mathbf{x}(0)$ and we feed the input sequence $\mathbf{u}(0), \mathbf{u}(1), \ldots, \mathbf{u}(T)$ to the reservoir. We update the k-th state of the reservoir with the equation:

$$\mathbf{x}(k) = f\left(\mathbf{W}_r^r \mathbf{x}(k-1) + \mathbf{W}_r^i \mathbf{u}(k) + \mathbf{W}_o^r \mathbf{y}(k-1)\right)$$

where $f()$ is the activation function of the internal units.

To compute the output weights that minimize the training error at the n-th step, we collect the states of the reservoir in $\mathbf{x}(n)$ and the teacher signal in $\mathbf{y_{teach}}(n)$. We take the output of the model $y(n)$ as:

$$\mathbf{y}(n) = f^{out}(\sum_{i=1}^{N} \mathbf{w}_i^{out} \mathbf{x}_i(n))$$

where $w^{out}$ are the output weights of the i-th layer and $f^{out}$ is $tanh$ so that is invertible. We obtain

$$(f^{out})^{-1}\mathbf{y}(n) = \sum_{i=1}^{N} \mathbf{w}_i^{out} e_i(\ldots, \mathbf{u}(n-1), \mathbf{u}(n))$$

where $e_i$ are the echo functions. We define the error:

$$\epsilon_{train}(n) = (f^{out})^{-1}\mathbf{y}_{teach}(n) - (f^{out})^{-1}\mathbf{y}(n).$$

The goal is to find the weights $\mathbf{w}^{out}$ that minimize the MSE:

$$\text{MSE}_{\text{ train}} = \frac{1}{(n_{\max} - n_{\min})} \sum_{i=n_{\min},\ldots,n_{\max}} \epsilon^2_{\text{train}}(n)$$

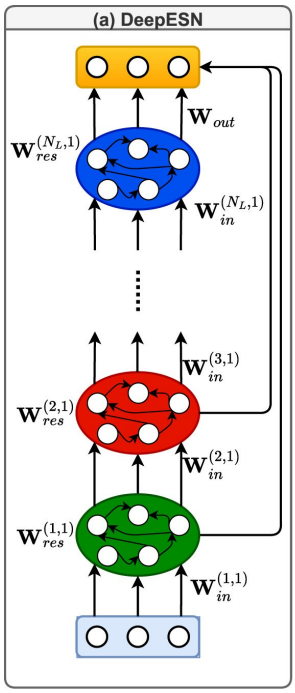



Figure 3.8: The representation of a DeepESN with $N_l$ stacked reservoir layers. *Source: [77]*

[60].

## Deep learning for reservoir computing

A deep-reservoir model is built by stacking a certain number of reservoirs together (see (Figure 3.8)) and connecting them to the readout layer using distinct weights for each layer. Once again, training is confined to the connections towards the readout. The system can represent various time scales and frequencies thanks to the presence of many layers. It's worth noting that there are alternative approaches to introduce depth in reservoir Computing, such as adding layers between the reservoir and the readout layer [13] or between the input and the reservoir. However, our focus here is limited to the additional layers between one reservoir and another.

The dynamic is richer if the scaling parameters, particularly the weights between connections of consecutive reservoir layers, are sufficiently large.

There are two primary configurations for the readout computation. In an all-layer setup, all reservoir layers are connected to the readout [39]. On the other hand, in a last-layer setup, only the final layer is connected to the readout.

The layers in the reservoir allow to represent different timescales. This is confirmed by

the fact that the impact of changing the input lasts longer in the higher layers of the deep reservoir architecture. This observation led to the development of an automated algorithm to decide the depth in untrained deep recurrent neural networks. The algorithm consists of adding layers until the impact on the reservoir performance becomes negligible [42].

A frequent question in the community is: *Can we find a reservoir structure that outperforms pure randomness?*. Multiple literature sources provide strategies to enhance reservoir organization. For example, in [96], [106] all reservoir units are interconnected to create a ring.

### 3.2.4. Euler state networks

In this section, we introduce a method to improve the stability and dissipativity of the reservoir through the Euler State Networks (EuSN) introduced by [37]. This approach wants to address the limits of the Echo State Property (ESP) which brings stability and robustness to the input perturbations but makes the memory of the reservoir more fading. This behavior disables the reservoir to propagate the history many time-steps ahead. The property of stability is derived by considering the recurrent system modeled by the following ODE and imposing the real part of the eigenvalues of its Jacobian to be less or equal to zero. The ODE is:

$$\mathbf{h}'(t) = f(\mathbf{h}(t), \mathbf{x}(t)) = \tanh\left(\mathbf{W}_h\mathbf{h}(t) + \mathbf{W}_x\mathbf{x}(t) + \mathbf{b}\right)$$

and the Jacobian is:

$$\mathbf{J}_f(\mathbf{h}(t), \mathbf{x}(t)) = \frac{\partial f(\mathbf{h}(t), \mathbf{x}(t))}{\partial \mathbf{h}(t)} = \mathbf{D}(t)\mathbf{W}_h$$

where $\mathbf{W_h}$ is the recurrent reservoir weight matrix, $\mathbf{D}(t)$ is a diagonal matrix with elements $1-\tilde{\mathbf{h}}_1^2(t), 1-\tilde{\mathbf{h}}_2^2(t), \ldots, 1-\tilde{\mathbf{h}}_N^2(t)$, with $\tilde{\mathbf{h}}_i(t)$ being the $i$-th element of $\tanh\left(\mathbf{W}_h\mathbf{h}(t) + \mathbf{W}_x\mathbf{x}(t) + \mathbf{b}\right)$ [37]. At the same time, if the eigenvalues are all negative and far from zero, the system is dissipative. Consequently, it is advantageous to impose the eigenvalues to assume the real part of the eigenvalues near zero and this is obtained by imposing the antisymmetry of the reservoir internal weights matrix. The antisymmetry of the matrix is satisfied if:

$$\mathbf{W}_h = -\mathbf{W}_h^T$$

This condition implies that all the eigenvalues of the Jacobian are purely imaginary (if $\mathbf{D}(t)$ is invertible) and allow us to obtain the desired property on the Jacobian.

The ODE is solved using the forward Euler method:

$$\begin{aligned}\mathbf{h}(t) =& F(\mathbf{h}(t-1), \mathbf{x}(t)) \\ =& \mathbf{h}(t-1) + \\ & \varepsilon \tanh\left(\left(\mathbf{W}_h - \gamma \mathbf{I}\right)\mathbf{h}(t-1) + \mathbf{W}_x \mathbf{x}(t) + \mathbf{b}\right)\end{aligned}$$

where $\varepsilon$ is the step size and $\gamma$ is a parameter that controls the stability of the reservoir [37].

Experiments [37] demonstrate that the EuSN outperforms the ability of long-term memorization of the RC models. We will use this approach in our experiments since it is expected to improve the representation of long-range dependencies in the data.

Analogously to the procedure analyzed in subsubsection 3.2.3, we randomly initialize a matrix $\mathbf{W} \in \mathbf{R}^{\mathbf{N}\times\mathbf{N}}$. Then, to follow the requirements of antisymmetry, the matrix $\mathbf{W}_h$ of the above formula is set to be equal to $\mathbf{W} - \mathbf{W}^T$. As underlined in [37], the stability of the reservoir is given by the antisymmetry of the matrix $\mathbf{W}_h$, and the spectral radius does not need to be controlled.

### 3.2.5. Reservoir computing for multivariate time series

In this overview, we present reservoir Computing as a technique for handling multivariate time series, i.e. the architecture is able to take multiple variables as inputs. The primary reference for this section is the source [14].

#### Reservoir computing for multivariate time series classification

MTS classifiers that use a basic RC architecture often cannot be as accurate as fully trainable neural networks. In the paper [14] the "reservoir model space" addresses this problem. Here, the reservoir state is projected into a lower dimensional representation through a trainable linear readout and with the learned parameters, the network assigns the input MTS to a class. In this study, the MTS is processed sequentially with a RC and the classification is performed using the sequence of states generated over time.

Although the primary focus of the paper is not classification, we highlight it because a similar preprocessing of the input will be used in our approach.

### Possible approaches for the representation of the input

Starting from the sequence of reservoir states, the input representation can be generated in different ways.

In some approaches, the last or the mean of the reservoir states are used as the description of the MTS [104], [85] and then a classification algorithm is employed. In this way, however, often, it cannot achieve the same accuracy as fully-trained neural networks.

It has been proposed an alternative approach named "output model space" by [48], [115], [19] in which the MTS is described through the parameters of a model trained to map the reservoir state into the next MTS in input. As observed in [14], according to the prediction step only some of the dynamics are useful to predict the next input. However, features considered unimportant for predicting future values in a multivariate time series might still encapsulate valuable information about the MTS characteristics. Consequently, discarding such features can introduce bias into the representation.

The "reservoir model space" proposed in [14] is the third possible approach designed to overcome this last limitation. It is an *"unsupervised procedure"* in which a one-step-ahead predictor encoder tries to predict the next state of the reservoir corresponding to an input sequence. For this task, is required to take into account all the dynamics of the reservoir. This approach avoids the exclusion of features that might be useful for characterizing the multivariate time series. The decoder, instead, takes the encoder representation and calculates the posterior probability of the output.

## 3.3. Baseline models

In this section, we introduce the baseline models used for the experiments and we provide a brief overview of the models (see Figure 3.9).

### 3.3.1. Linear models

Despite appearing less effective than transformers, which exploit self-attention mechanisms, linear models currently establish a benchmark for time series forecasting accuracy.

The research conducted by [124] claims that, in some settings, the transformer model is ineffective for time series forecasting due to the temporal information loss caused by the self-attention mechanism. To support this claim, they introduce a series of one-layer linear models that outperform the transformer benchmarks across various experiments. Among these models, there are DLinear and NLinear models (see Figure 3.10). As reported in the

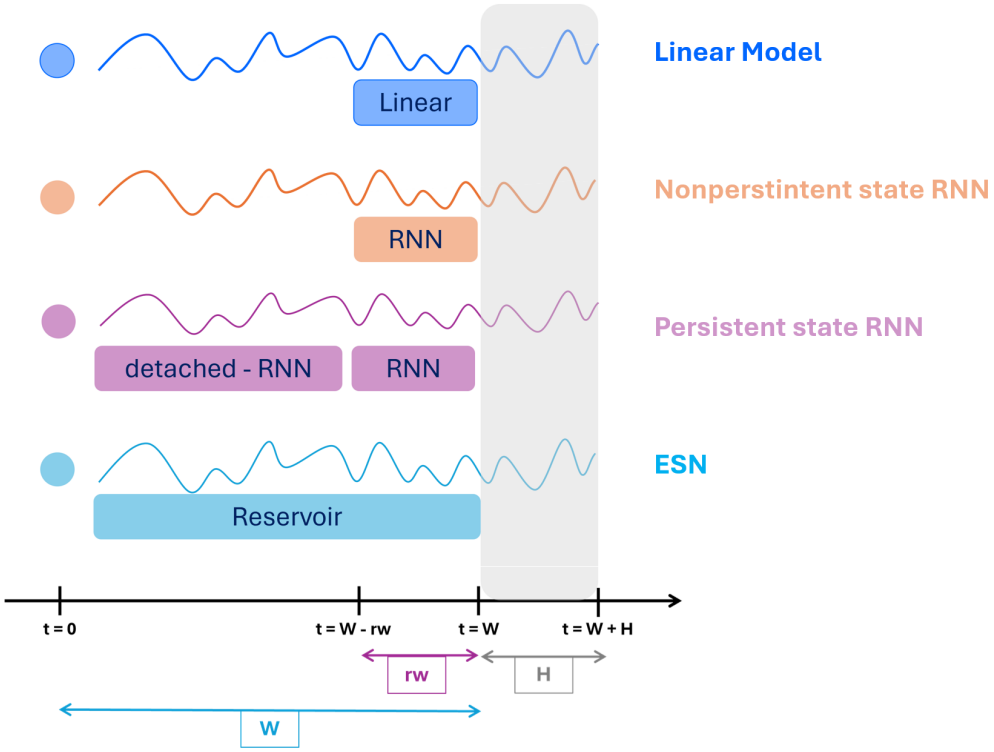



Figure 3.9: Overview of the models and what part of the time series they process. Here, we give as an example an entire time series of length $W + H$, where $W$ is the length of the input and $H$ is the length of the output. The linear models process the last $rw$ time steps of the window and project them into the final prediction. The nonpersistent state RNN processes the last part of the window of length $rw$. The persistent state RNN uses the detached RNN to process the first part of the window and produce a hidden state and the same RNN processes the last $rw$ time steps using the hidden state produced by the detached RNN. The ESN processes the whole window $W$ with the reservoir and a linear layer (or an multilayer perceptron (MLP)) uses the last hidden state produced by the reservoir to make the final prediction.

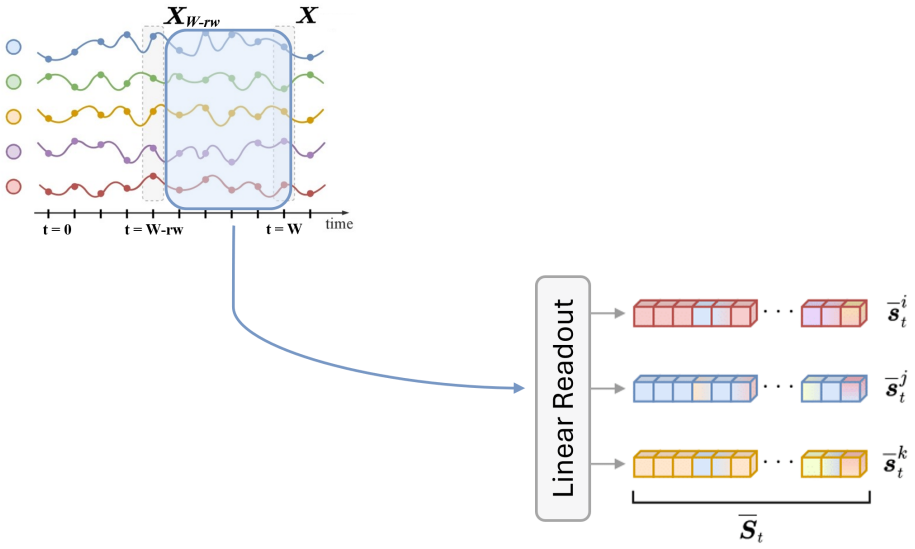



Figure 3.10: Overview of the Lineae Model.
The linear model processes the last $rw$ time steps of the window and projects it into predictions of length $H$.
*This image was created using elements from the original image by: [24]*

paper, DLinear calculates the moving average on the input data and subtracts it from the time series. Then, the moving average and the remainder are processed by two separate linear layers and the corresponding outputs are summed together to build the prediction. This approach is suitable for data with evident trends [124]. On the other hand, NLinear, before processing the data using the linear layer, subtracts the input batch by the last value of the sequence and sums it back before calculating the error on the prediction [124]. This kind of normalization allows to center of the sequence, mitigating potential biases introduced by varying levels across sequences. In this way, the model becomes more robust to variations in the distribution of individual sequences over time.

In the experiments, the authors designed the Nlinear model with one linear layer dedicated to each feature. This design choice was taken to prevent information loss since the model consists of only one layer, which is the linear layer. However, we will not use this separation for the other models used in our experiments.

During our study, we considered and used DLinear and NLinear models. However, since they do not outperform a simple linear model on the datasets, and to maintain generality, we decided to use a standard linear layer in the experiments reported.

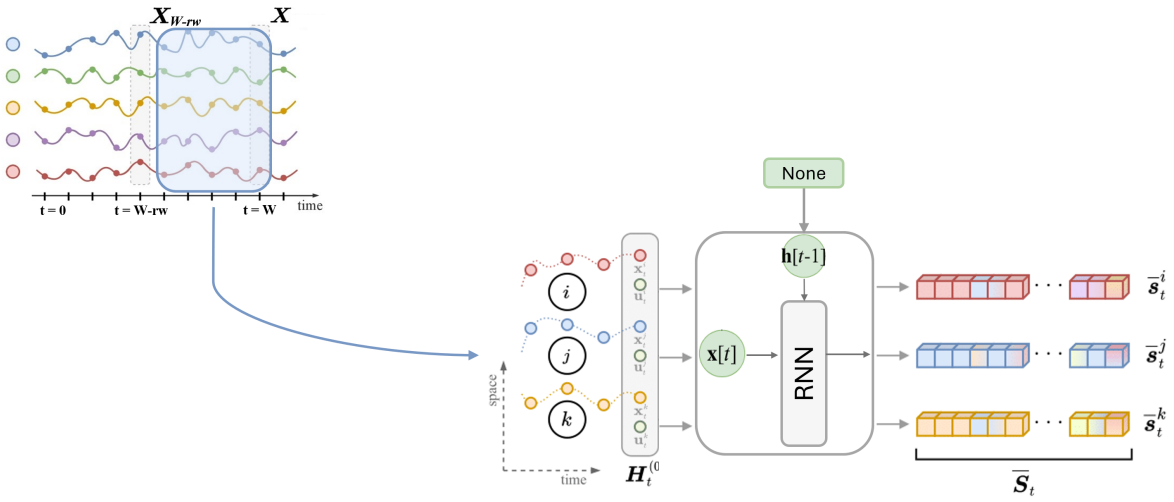



Figure 3.11: Overview of the nonpersistent state RNN.
The nonpersistent state RNN processes only the last $rw$ time steps of the window. The hidden state is reset after each batch with the value *None*, which is internally handled with a zeroes initialization.
*This image was created using elements from the original image by: [24]*

### 3.3.2. Nonpersistent state RNN

Nonpersistent state RNNs are taken as baselines since they are computationally more efficient than persistent state RNNs and we aim to keep that efficiency in our model without losing the advantages of persistent state RNNs.

Nonpersistent state RNNs process each batch independently, without considering the previous state of the model (see Figure 3.11). Consequently, batches can be shuffled or processed in parallel, making the training faster and more efficient.

**Managing input and covariates in the implementation** Here, we distinguish two cases based on two different implementations. If we are considering a dataset made of multiple time series of restricted length, the RNN processes only the last $rw$ time steps of the window. On the other hand, if we are considering a dataset made of a single long-time series, we use a sliding window to create the batches. In this case, a window of size $rw$ is slid over the time series to create the batches. Both these approaches are used in our experiments depending on the dataset. The same holds for the linear models.

### 3.3.3. Persistent state RNN

Persistent state RNNs are taken as baselines since they can capture long-term dependencies in the data.

Our design approach is a simplified version of TBBTT for persistent state RNNs. We handle batches with a larger number of time steps (see Figure 3.12), allowing us to initially process a portion of the batch (let's say from 0 to $W-rw$, with $W$ the total number of time steps in the batch and $rw$ the number of time steps to be processed by the trainable RNN) without considering any initial state and detaching the gradient from the computation graph. Subsequently, the same RNN processes the remaining portion of the batch having as initial state the one calculated using the previous portion (see Figure 3.13). Given the error that it makes predicting the output when being trained on the last $rw$ time steps, the RNN adjusts its weights to learn to extract a useful hidden state from the preceding window. In the following iteration, it uses those adjusted weights to extract a new hidden state to be used and the hidden state is again used in the training of the RNN on the last $rw$ time steps. In this way, the RNN learns how to extract a useful hidden state from the preceding window and uses it to make the final prediction.

This strategy eliminates the need to keep batches in strict time order because the state is computed based on the first time steps within each batch.

**Managing input and covariates in the implementation** Also here, we distinguish two cases. If the sliding window approach is used, window size $W$ must be fixed and the window is slid over the time series to create the batches. This window size is bigger than $rw$ to allow the RNN to process the first $W - rw$ time steps with a detached RNN to produce the hidden state. This approach brings a double advantage: it allows subsample batches in parallel and helps the RNN to extract a useful hidden state. Indeed, the RNN can struggle to extract a useful hidden state from a very long sequence.

Otherwise, if the sliding window approach is not used, the detached RNN processes the whole time series and the hidden state produced is used as the initial state for the RNN processing the last $rw$ time steps.

### 3.3.4. ESN

ESNs are taken as baselines since they are computationally efficient and can capture long-term dependencies in the data.

In our framework, we use the ESN to extract another dataset from the initial one. The reservoir is initialized with random weights uniformly distributed in the range $[-1, 1]$ and scaled by the spectral radius, which is the maximum absolute value of the eigenvalues of the reservoir matrix. The recurrent weights are pruned to create sparse connectivity if the specified density is less than 1, randomly setting to zero a fraction of the weights.

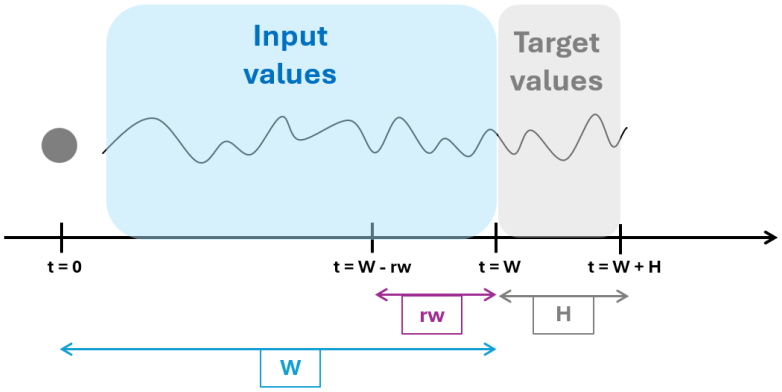



Figure 3.12: A visual representation of the time steps in a batch.
The input is of length $W$ and the last $rw$ time steps are those processed by the trainable RNNs. The predictions are of length $H$.

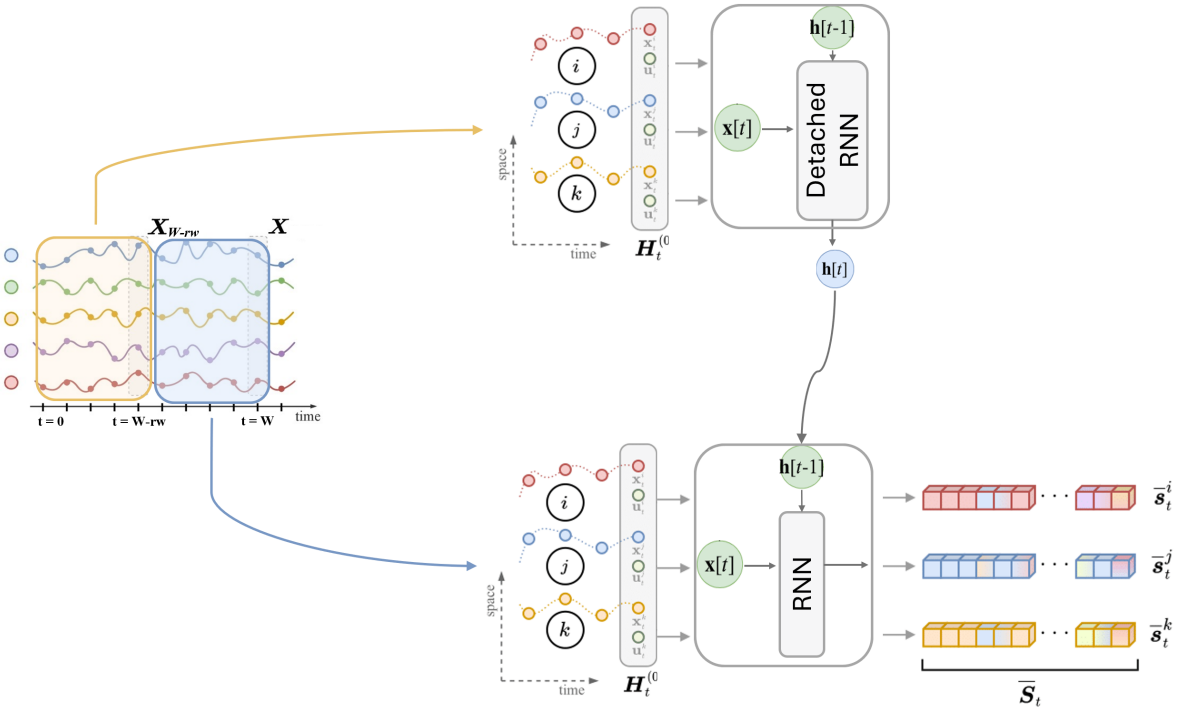



Figure 3.13: Overview of the persistent state RNN.
The persistent state RNN processes the first $W - rw$ time steps of the window with a detached RNN and the last $rw$ time steps with the same RNN using the hidden state produced by the detached RNN. The hidden state produced with the first $W - rw$ time steps is used as the initial state for the RNN processing the last $rw$ time steps.
*This image was created using elements from the original image by: [24]*

The reservoir is fed sequentially with each step of the time series and the states of the reservoir itself. It projects an input in a higher-dimensional space, which is then used as input for the MLP (or linear layer). Consequently, the dataset fed to the reservoir is transformed into a dataset with the same number of time steps as the original one and with a higher number of features, given by the hidden size of the reservoir. Since we use staked ESN, we can add layers to the reservoir and it will increase further the number of features extracted from the time series.

The output time series at each time step is a high-dimensional representation of the input history up to that point in time and is created by stacking the states of the reservoir at each time step.

The resulting dataset is given as a covariate to the dataset handler. In this way, it will produce batches with input and target taken from the initial dataset and the covariate taken from the reservoir keeping a correspondence in terms of time steps among the two datasets.

Therefore, the RC model pre-computes ahead of training, in an unsupervised manner, the encodings which are high-dimensional state representations that capture multi-scale temporal dynamics. This means that the system can learn and understand the historical patterns of the input time series efficiently before entering the training phase.

The reservoir implements the following equation:

$$\boldsymbol{h}(t) = (1-\alpha)\boldsymbol{h}(t-1) + \alpha f(\boldsymbol{W}_r^r \boldsymbol{u}(t) + \boldsymbol{W}_i^r \boldsymbol{h}(t-1))$$

where $\boldsymbol{h}(t)$ is the state of the reservoir at time $t$, $\boldsymbol{u}(t)$ is the input at time $t$, $\boldsymbol{W}_r^r$ and $\boldsymbol{W}_i^r$ are the recurrent and input weights of the reservoir, respectively, and $\alpha$ is the leaking rate. The new hidden state $h(t)$ is the linear combination of the previous hidden states $h(t-1)$ and the new input $u$ using the weights $W_r^r$ and $W_i^r$. Additionally, the leaking rate $\alpha$ controls the amount of information from the previous hidden state that is retained in the new hidden state.

The second component of the ESN, which is the readout, is used to make the final prediction. Differently from the reservoir, the computation of the readout is not detached from the computation graph and the gradient is backpropagated through it. It takes as input the last hidden state of the reservoir and maps it into predictions which are compared with the target variable of the original dataset (see Figure 3.14).

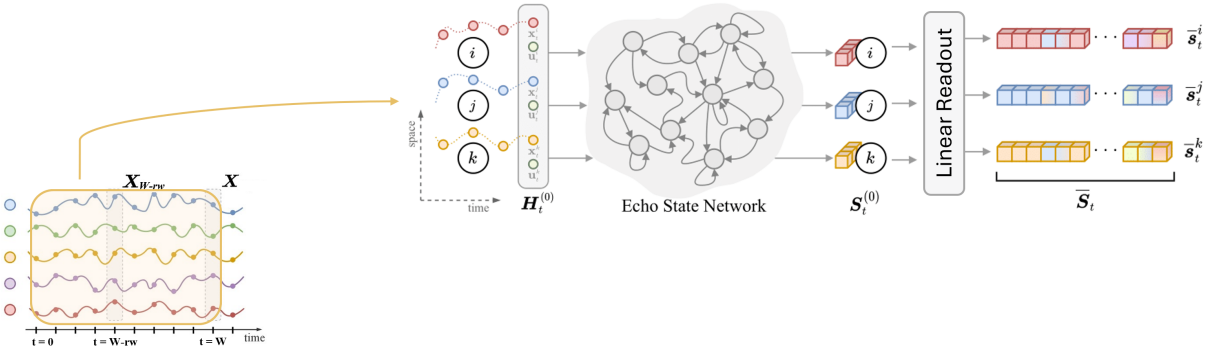



Figure 3.14: Overview of the ESN.
The whole window $W$ is processed by the ESN. The last hidden state produced by the ESN is used as the input for the linear layer that makes the final prediction.
*This image was created using elements from the original image by: [24]*

**Managing input and covariates in the implementation** Also here, we distinguish two cases. If the sliding window approach is not used, and the entire time series is into the batch, the reservoir processes the whole time series and then only its last hidden state is kept and used to make the final prediction. Otherwise, if the sliding window approach is used, the reservoir processes the whole window and the whole sequence of hidden states is kept. A way to handle the sequence of hidden states is to assign it to the batches as a covariate variable, keeping a correspondence in terms of time steps among the two datasets. Then, inside each batch, only the last hidden state is used to make the final prediction. In this way, the last hidden states of the reservoir in each batch contain the history seen by the reservoir from the beginning of the dataset up to the point of time corresponding to the last time step in input to the batch. An alternative approach to the previous one is to use a linear regression in place of the readout layer to make the final prediction, when possible. This approach can be faster than training the readout layer with the backpropagation algorithm if the dataset is not too large and it returns a closed-form solution.

# 4 | Hybrid learnable RNNs with reservoir state initializers

In this section, we present our approach for long-term time series forecasting based on RNNs. We start by providing a high-level motivation for the proposed approach and then discuss the model in detail. We're targeting a dual challenge. On one hand, we address the scalability and computational efficiency of the training procedure proper of recurrent models for time series forecasting. On the other hand, we aim to improve the accuracy in the long-term forecasting task, with design choices directed at revealing long-range dependencies.

As stated in section A.2, the training of long time series is computationally expensive, especially when dealing with multiple time series. Approaches that require to process the whole time series in sequence, are difficult to train on an entire time series. We will follow a similar approach to [24] which overcomes the problem of scalability of spatiotemporal graph neural networks (STGNNs) and enables the parallelization of the training procedure by sampling in batches the input data. This approach, allowing the training in mini-batches, allows the model to be trained in an efficient way while taking into account the temporal relations along the time series.

## 4.1. High-level motivation and introduction to the proposed approach

In this section, we provide a high-level overview of the model.

**ESN trade-off between training convenience, hyperparameters tuning and representation size** In our approach, we aim to get the advantages of the training of ESN, while mitigating its disadvantages and enhancing the performances. While persistent state RNNs are computationally expensive and require a long time to train for long sequences, the ESN model is efficient in terms of computation. Indeed, being composed of a train-free

reservoir, the training can be confined to the readout only. However, since it is not fully trainable, finding the right hyperparameters for the reservoir can be challenging. While there are benefits in not having to train the system, relying on the architectural bias only requires a significantly larger hidden state [36]. This larger representation is crucial because the system does not autonomously learn and adapt to the specific data dynamics. Since the reservoir does not learn, having a large representation is essential to cover a broader range of patterns and features. The model is less flexible when faced with data dynamics that did not appear during the design phase because it depends entirely on the predetermined architecture. Consequently, since the model will not learn, the representation must be large enough to capture all potential variations, making hyper-parameters a critical factor. In essence, there is a trade-off between the convenience of avoiding training and the necessity for a more expansive representation, which heavily depends on hyperparameter tuning. This makes the reservoir model advantageous if we can find a representation large enough. The large hidden state of the ESN, therefore, allows the network to keep track of the past dependencies. However, mapping the representation created by the reservoir to the target using a linear layer (or an MLP) might be difficult due to the complexity of the task of long-range time series forecasting. For this task, the model must take into account the dependencies across time steps, which will influence the outcome over the horizon. Therefore, in our approach, we introduce an RNN that should better keep track of the temporal dependencies in the last part of the time series, while keeping knowledge of the history thanks to its hidden state. This is achieved by initializing the hidden state with an encoding of the reservoir hidden state.

**Comparison between nonpersistent and persistent state** In our approach, we aim to keep the velocity and computational efficiency of nonpersistent state RNNs while having the ability to capture long-term dependencies of the persistent state RNNs.

The difference between nonpersistent state RNN and persistent state RNN lies in how they handle the hidden state. In the case of nonpersistent state RNN, the internal state is reset after each batch, hence, the internal state learned from the previous batches is not used. On the other hand, in the case of persistent state RNN, the internal state is never reset, and, therefore, we need to process the whole time series sequentially so that the RNN can use the state coming from the processing of the previous time steps. On one hand, the persistent state RNN is computationally expensive because it requires processing the whole sequence in order. On the other hand, the nonpersistent state RNN, which is computationally more lightweight, is not able to capture the long-term dependencies in the data.

**Idea behind the proposed approach** The idea behind our approach is to employ a reservoir that processes the whole time series ahead of training and a trainable RNN which initializes its hidden state with an encoding of the reservoir state. The reservoir transforms the input data into a high-dimensional space dataset [] representing the dynamics of the system up to that point in time. The trainable RNN uses an encoding of the reservoir states to initialize its hidden states when processing each input batch, having access, in this way, to the whole history of the system while being trained as a nonpersistent state RNN. In this way, there is no need, during training, to process the input time series sequentially from the beginning to the end, but the time series can be divided into batches, that can also be shuffled or processed in parallel, and that are individually processed by the RNN. We will refer to this new model as ReRNN.

Our approach can be particularly useful, especially in the case of edge devices, where the computational resources are limited and it is needed to process data as fast as possible and in an efficient way. For example, in the case of distributed traffic sensors, the computational resources represent a limitation for the training of models.

## 4.2. Proposed approach

In this section, we introduce the proposed model, which combines the advantages of reservoir computing and fully trainable RNNs for time series forecasting.

We propose a relatively simple modification to the RNN architecture by modifying only the way in which the history of observation is managed while keeping the rest of the architecture unchanged. In order to make the model effective for temporal dynamics, our idea is to initialize the hidden state of the RNN using the dynamics encoded by the reservoir instead of using the RNN itself.

**Components of the ReRNN** A reservoir with the same structure as the one in the reservoir model subsection 3.3.4 is used to extract covariates to be added to the batches as described in subsection 3.3.4. Differently from the ESN case, we do not use the representation corresponding to the last time step in the batch, but the one corresponding to the time step preceding the ones in the batch (that will be processed by the RNN). Indeed, we are now using the reservoir representation as the initial state for the RNN and not to directly forecast the future time steps. This is done because we are aiming to use the reservoir output as a representation of the history preceding the time steps processed by the RNN, instead of using the hidden state computed with the detached RNN as in the persistent state RNNs subsection 3.3.3.

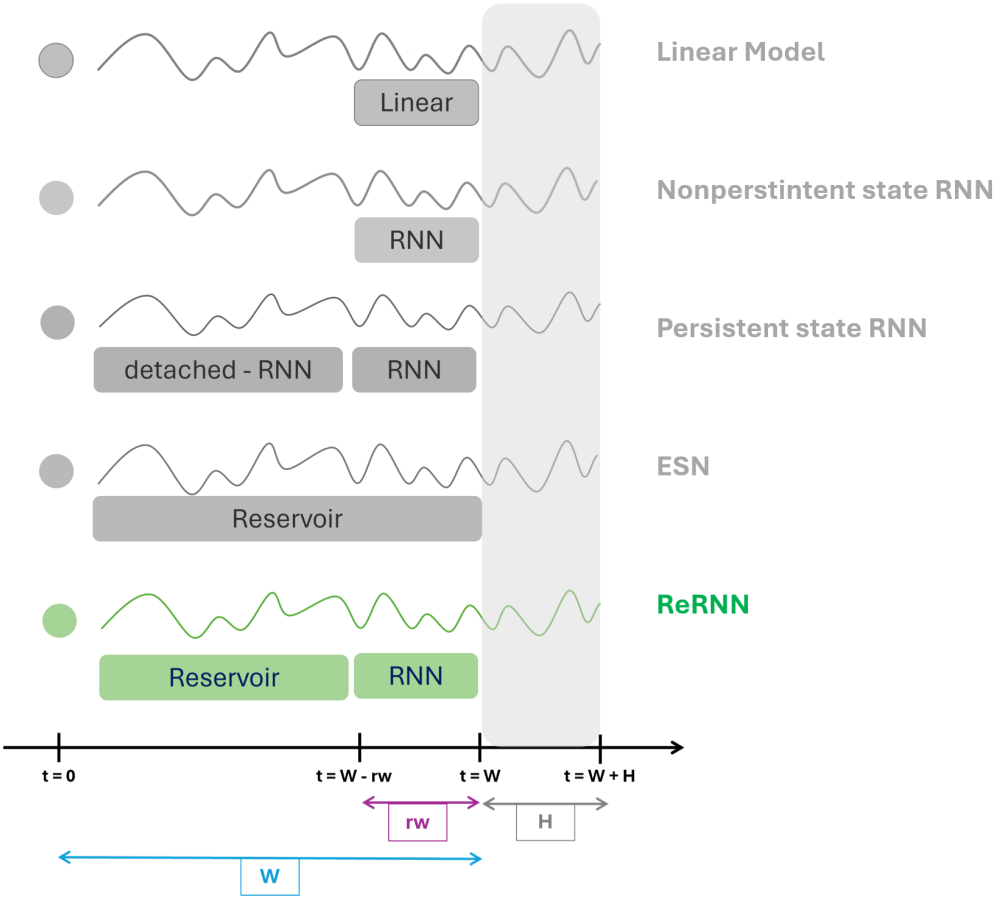



Figure 4.1: Overview of the models and what part of the time series they process with a focus on the ReRNN.

The proposed model (ReRNN) uses the reservoir to process the first $W - rw$ time steps and a linear layer (or an MLP) to map the hidden state produced by the reservoir into the initial state of the RNN. The RNN processes the last $rw$ time steps using as a hidden state the encoding produced by the linear layer (or MLP). This figure completes the one reported in the previous section Figure 3.9.

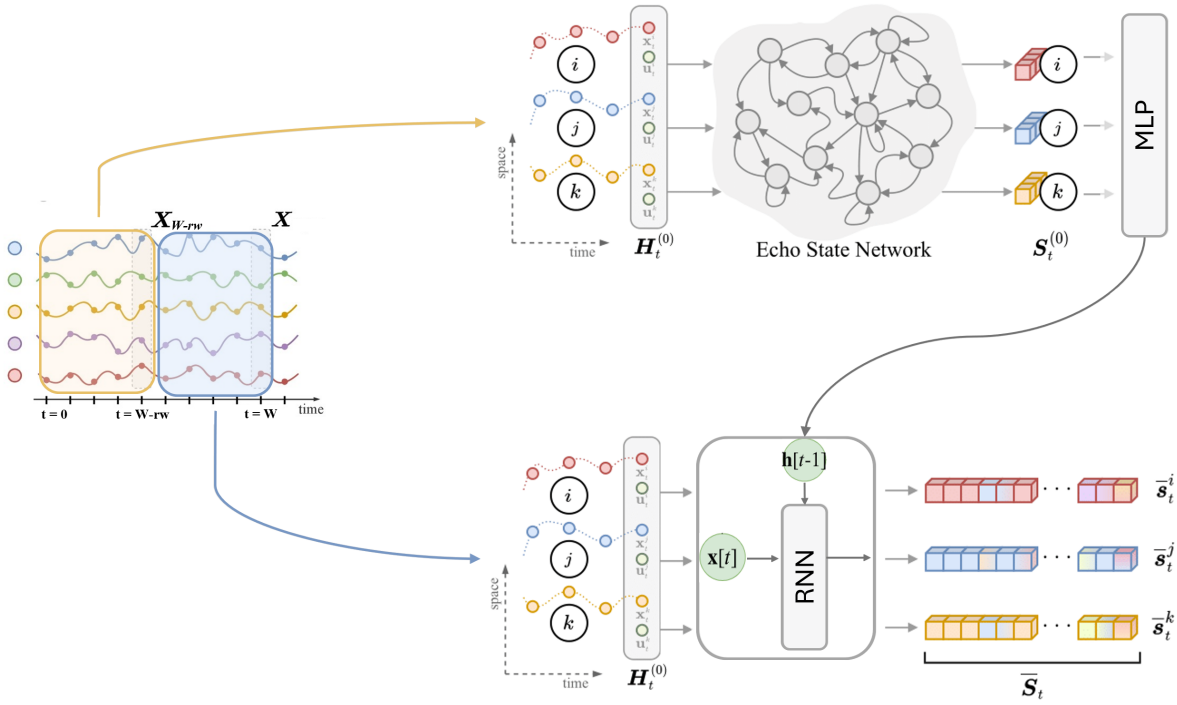



Figure 4.2: Overview of the ReRNN.
The first part of the window (up to $W - rw$) is processed by the reservoir. The hidden state produced by the reservoir is processed by the linear (or MLP) layer that creates an encoding for the initialization of the hidden state of the RNN. The RNN processes the last $rw$ time steps of the window. Finally, a linear layer maps the RNN hidden state into the prediction.

*This image was created using elements from the original image by Cini et al. [24]*

Doing so, the RNN employed, as the nonpersistent state RNN, does not need to pass the hidden state among different batches. Hence, each batch can be processed independently in a parallel and non-sequential way gaining the computational efficiency that we aim for our model during the training procedure. In this way, the RNN can access the whole history of the system up to that point in time while being trained as a nonpersistent state RNN.

An encoder is employed in order to learn how to map the reservoir's state to the RNN hidden state. The encoder can be easily implemented using a linear layer. In this way, the reservoir can be used to initialize the state of an RNN having a different hidden size than the reservoir and the encoder can learn how to handle the reservoir representation to effectively initialize the RNN.

We choose to use GRU [22] as the RNN for our experiments.

### 4.2.1. Problem statement

**Data structure and conventions** We take a set of $K$ time series, where each time series is composed of a sequence of $W$ vectors having the same dimension $N_x$. Each vector of observation of the $i$-th time series can be denoted with $\boldsymbol{x}_t^i \in \mathbb{R}^{N_x}$ where $t$ is the time step and $N_x$ the number of features ($1 \leq t \leq W$, $1 \leq i \leq K$). We define with $\mathbf{X} \in \mathbb{R}^{W \times K \times N_x}$ the set of $K$ time series, of length $W$, and feature dimension $N_x$. We define $\mathbf{X}_t \in \mathbb{R}^{K \times N_x}$ the set of $K$ observations taken at time step $t$. We denote with $\mathbf{X}_{t:t+T} \in \mathbb{R}^{T \times K \times N_x}$ the set of $K$ sequences in the interval $[t, t+T)$ and with $\mathbf{X}_{<t}$ the observations up to time step $t$ (with t excluded). In our settings, we consider a time-invariant stochastic process generating the time series for which:

$$\boldsymbol{x}_t^i \sim p^i(\boldsymbol{x}_t^i \mid \boldsymbol{x}_{<t}^i) \quad \forall i = 1, \ldots, K. \tag{4.1}$$

Hence, we assume that the process generating data produces new data depending on the previous ones, according to unknown criteria. For instance, we could have a dataset composed of a set of time series collecting the traffic information around the city considering different measures, such as velocity and number of vehicles. We can denote $K$ the number of time series, that may be given by the number of sensors there are on the roads. The number of features $N_x$, in this case, is two, since we have velocity and number of vehicles.

**Task description and objective function** Our task is to perform multi-step ahead time series forecasting (see subsection 2.5.6). Given a time step $t$, we aim to predict the horizon for $H$ steps in the future using all the past input information $\boldsymbol{X}_{<t}$. We define

the target $\mathbf{X}_{t:t+H}$. We aim to train a model $p_{\boldsymbol{\theta}}$ to find the best set of parameters $\boldsymbol{\theta}$ to approximate the stochastic process $p^i$:

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}^i_{t+h} \mid \boldsymbol{x}^i_{<t}) \approx p^i(\boldsymbol{x}^i_{t+h} \mid \boldsymbol{x}^i_{<t}) \quad \forall h \in [0, H) \quad \forall i = 1, \dots, K. \tag{4.2}$$

Hence, we aim to approximate the function generating data in which past values influence future ones. We train the models' family $\mathcal{F}(\cdot; \boldsymbol{\theta})$ that generates the predictions over the horizon $\widehat{\boldsymbol{X}}_{t:t+H}$ that should approximate the expected value of the target value $\boldsymbol{X}_{t:t+H}$. More formally:

$$\widehat{\boldsymbol{x}}^i_{t:t+H} = \mathcal{F}\left(\boldsymbol{x}^i_{<t}; \boldsymbol{\theta}\right) \quad \text{s.t.} \quad \widehat{\boldsymbol{x}}^i_{t:t+H} \approx \mathbb{E}_p\left[\boldsymbol{x}^i_{t:t+H}\right] \quad \forall i = 1, \dots, K. \tag{4.3}$$

To learn the parameters $\boldsymbol{\theta}$, it is minimized the cost function $\ell(\cdot)$ on training data such as

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \frac{1}{K} \sum_{i=1}^{K} \ell\left(\widehat{\boldsymbol{x}}^i_{t:t+H}, \boldsymbol{x}^i_{t:t+H}\right). \tag{4.4}$$

We use as cost function the MAE:

$$\ell\left(\widehat{\boldsymbol{x}}^i_{t:t+H}, \boldsymbol{x}^i_{t:t+H}\right) = \frac{1}{H} \sum_{h=0}^{H-1} \left\|\widehat{\boldsymbol{x}}^i_{t+h} - \boldsymbol{x}^i_{t+h}\right\|_1 \quad \forall i = 1, \dots, K. \tag{4.5}$$

### 4.2.2. Solution overview

**Proposed solution** We are addressing the problem of learning the past information we need to predict the future. As previously explained, training an RNN on the whole sequence propagating the hidden state across all the time steps is inefficient. The solution we propose is to train a model $p_{\boldsymbol{\theta}}$ using a recent window of data $\boldsymbol{x}^i_{t-W:t}$ and an encoding of the past history $\boldsymbol{z}^i_{<t-W}$, rather than the whole time steps of the input $\boldsymbol{x}^i_{<t}$ (as in equation 4.2). Therefore, we define the model to train $p_{\boldsymbol{\theta}}$ as:

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}^i_{t+h} \mid \boldsymbol{x}^i_{t-W:t}, \boldsymbol{z}^i_{t-W}) \approx p^i(\boldsymbol{x}^i_{t+h} \mid \boldsymbol{x}^i_{<t}) \quad \forall h \in [0, H) \quad \forall i = 1, \dots, K. \tag{4.6}$$

where $\boldsymbol{z}^i_{t-W}$ is an encoding of the history of the system obtained as $g(\boldsymbol{x}^i_{<t-W})$ where $g$ is a function that encodes the whole past in a vector. Hence, the model makes use of the input window $\boldsymbol{x}^i_{t-W:t}$ and of an encoding $g(\boldsymbol{x}^i_{<t-W})$ that takes into account the whole history preceding the window.

Similarly to the the previous section, we aim to learn the parameters $\boldsymbol{\theta}$, by minimizing the cost function $\ell(\cdot)$ on training data using as cost function the MAE.

**Detailed review of the proposed approach** Now we review the approach we propose in detail. We are given an input sequence $\mathbf{X} \in \mathbb{R}^{W \times K \times N_x}$, where:

- $W$ is the number of time steps of the series in input and $rw$ is the window on which we train the RNN, with $rw < W$,
- $K$ is the number of time series,
- $N_x$ is the number of features per time series.

The reservoir processes the input $\mathbf{X}_{0:W-rw}$ from time step 0 to time step $W - rw - 1$ and produces an output sequence $\mathbf{Z}$ with the same number of time series and of the same length $(W - rw)$ but on a richer feature space:

$$\boldsymbol{z}^i_{0:(W-rw)} = \text{RESERVOIR}\left(\boldsymbol{x}^i_{0:W-rw}\right) \quad \forall i = 1, \ldots, K. \tag{4.7}$$

The number of features in each time series produced by the reservoir corresponds to the hidden state size $N_r$ of the reservoir itself, hence the output is $\mathbf{Z}_{0:(W-rw)} \in \mathbb{R}^{(W-rw) \times K \times N_r}$.

Only the last reservoir's state after processing the input sequence $\mathbf{Z}_T \in \mathbb{R}^{K \times N_r}$ $(T = W - rw - 1)$ is kept and passed through an MLP that learn how to create an encoding suitable for initializing the GRU. Hence, the MLP outputs a representation with features dimension equal to the dimension of the hidden state of the GRU $(N_h)$, i.e.,

$$\boldsymbol{h}^{i\prime}_T = \text{MLP}\left(\boldsymbol{z}^i_T\right) \in \mathbb{R}^{N_h} \quad \forall i = 1, \ldots, K. \tag{4.8}$$

The GRU is initialized with $\boldsymbol{h}^{i\prime}_T \in \mathbb{R}^{N_h}$ and it processes the remaining part of the input sequence $\boldsymbol{x}^i_{\text{W}-\text{rw}:W} \in \mathbb{R}^{rw \times N_h}$:

$$\boldsymbol{h}^{i\prime}_W = \text{GRU}\left(\boldsymbol{x}^i_{\text{W}-\text{rw}:\text{W}}, \boldsymbol{h}^{i\prime}_T\right) \in \mathbb{R}^{N_h} \quad \forall i = 1, \ldots, K. \tag{4.9}$$

The GRU outputs a state $\boldsymbol{h}^{i\prime}_W \in \mathbb{R}^{N_h}$ which is then mapped to prediction through a linear layer (READOUT):

$$\hat{\boldsymbol{x}}^i_{W:W+H} = \text{READOUT}(\boldsymbol{h}^{i\prime}_W) \in \mathbb{R}^{H \times N_x} \quad \forall i = 1, \ldots, K. \tag{4.10}$$

$\hat{\boldsymbol{x}}^i_{W:W+H} \in \mathbb{R}^{H \times N_x}$ is the predicted output sequence, where $H$ is the horizon length (see

Figure 4.1).

**Detailed review of the proposed approach: sliding window case** Let's now consider the specific case in which the sliding window approach is employed. We take again the input sequence $\mathbf{X} \in \mathbb{R}^{W \times K \times N_x}$, and $rw$ as window size. The collection of reservoir state vectors at each time step is, again, $\mathbf{Z} \in \mathbb{R}^{(W-rw) \times K \times N_r}$ and are obtained as:

$$\boldsymbol{z}^i_{0:W-rw} = \text{RESERVOIR}\left(\boldsymbol{x}^i_{0:W-rw}\right) \in \mathbb{R}^{(W-rw) \times N_r} \quad \forall i = 1, \ldots, K. \tag{4.11}$$

In this case, the entire output of the reservoir is kept and associated as a covariate to the batches. Specifically, each batch associated with time steps from $t$ to $t + rw$, with $t$ being any time step in the input $0 \leq t < (W - rw)$, uses a covariate vector $\boldsymbol{z}^i_{t-1} \in \mathbb{R}^{N_r}$ that is the reservoir's representation $\boldsymbol{z}^i$ at the time step preceding $t$. Therefore, the MLP creates an encoding $\boldsymbol{h}^{i\prime}_{t-1}$ of the reservoir representation $\boldsymbol{z}^i_{t-1}$:

$$\boldsymbol{h}^{i\prime}_{t-1} = \text{MLP}\left(\boldsymbol{z}^i_{t-1}\right) \in \mathbb{R}^{N_h} \quad \forall i = 1, \ldots, K. \tag{4.12}$$

and the GRU, initialized with the calculated hidden state, processes the input window $\boldsymbol{x}^i_{\text{t:}(t+rw)} \in \mathbb{R}^{rw \times N_h}$

$$\boldsymbol{h}^{i\prime}_{t+rw} = \text{GRU}\left(\boldsymbol{x}^i_{\text{t:t+rw}}, \boldsymbol{h}^{i\prime}_{t-1}\right) \in \mathbb{R}^{N_h} \quad \forall i = 1, \ldots, K. \tag{4.13}$$

The resulting state is processed by a linear readout which outputs the final predictions:

$$\hat{\boldsymbol{x}}^i_{(t+rw):(t+rw+H)} = \text{READOUT}(\boldsymbol{h}^{i\prime}_{t+rw}) \in \mathbb{R}^{H \times N_x} \quad \forall i = 1, \ldots, K. \tag{4.14}$$

$\hat{\boldsymbol{x}}^i_{(t+rw):(t+rw+H)} \in \mathbb{R}^{H \times N_x}$ is the predicted output sequence, where $H$ is the horizon length.

**Managing input and covariates in the implementation** Also here, as for the ESN, we can distinguish two cases based on the dataset used. If we are using a sliding window approach, we process the whole dataset with the reservoir and we keep the whole sequence of hidden states. We assign the sequence of hidden states to the batches as a covariate variable. Each batch has access to the hidden states of the reservoir corresponding to the history of the system up to the time step preceding the first time step in the input of the batch. This is performed by aligning the dataset produced by the reservoir with the original one shifted one-time step to the right. In case we are not using the sliding window approach, instead, we process the time series up to $W - rw - 1$ time steps and

we keep the last hidden state of the reservoir only. The hidden state encodes the history of the dataset from the start to the point preceding the last $rw$ time steps of the dataset, which will be processed by the trainable RNN (see Figure 4.1).

# 5 | Experimental evaluation

In this chapter, we empirically show the performance of our approach and compare it against baseline models. In the following, we describe the datasets and the experimental settings used for the experiments. More specifically, we show results for two experiments:

1. **Performance analysis on synthetic dataset**: In this experiment, the performances of the ReRNN are compared on a synthetic dataset.
2. **Performance comparison on two real-world datasets**: In this experiment, the performances of the ReRNN are compared on one traffic dataset and one electricity dataset.

## 5.1. Experimental setting

### 5.1.1. First experiment: performance analysis on synthetic dataset

A synthetic dataset is created to assert if the model can capture the long-range dependencies in the data and if our model can outperform the baseline models in a setting where understanding both the slow and fast dynamics of time series is needed to make accurate predictions.

#### synthetic dataset

The dataset contains 15000 time series each created in such a way that time steps far back in time are needed to make accurate predictions Table 5.1. Each time series is 2400 time steps long and is a sum of sine waves with different frequencies and amplitudes. We sample at random a number $n_{\text{long}}$ and $n_{\text{short}} \sim \mathcal{U}(3, 6)$ representing the number of long-period sine waves and we do the same for short-period sine waves. The sine waves with long periods are sampled from a uniform distribution with periods between $\lfloor \frac{T}{4} \rfloor$ and $\lfloor \frac{T}{2} \rfloor$, with $T$ being the window size, and amplitudes between 1 and 2. The sine waves with short periods are sampled at random with a period between $\lfloor \frac{T}{32} \rfloor$ and $\lfloor \frac{T}{8} \rfloor$ and amplitudes

between 1 and 2. Finally, they are taken with random phases. More formally, we have:

$$\mathbf{X}_i(t) = \sum_{j=1}^{n_{\text{long}}} A_j^{\text{long}} \sin\left(\frac{2\pi t}{P_j^{\text{long}}} + \phi_j^{\text{long}}\right) + \sum_{k=1}^{n_{\text{short}}} A_k^{\text{short}} \sin\left(\frac{2\pi t}{P_k^{\text{short}}} + \phi_k^{\text{short}}\right)$$

where:

- $n_{\text{long}}$ and $n_{\text{short}}$ are a randomly chosen between 3 and 6
- $A_k^{\text{long}}$ and $A_k^{\text{short}}$ are the amplitudes of long and short periods sine waves and are randomly chosen between 1 and 2.
- $P_k^{\text{short}}$ is the period of short-period sine waves and is randomly chosen between $\frac{window}{32}$ and $\frac{window}{8}$.
- $P_k^{\text{long}}$ is the period of long-period sine waves and is randomly chosen between $\frac{window}{4}$ and $\frac{window}{2}$.
- $\phi_k^{\text{short}}$ and $\phi_k^{\text{long}}$ are the random phases of the long and short periods sine waves and are randomly chosen between 0 and $2\pi$.
- $t$ is the time index ranging from 0 to 2400.
- $i$ is the number of the time series in the dataset ranging from 0 to 15000.

Each time series is divided by the difference between the maximum and the minimum value of the sum of the sine waves composing it. A visual representation of the dataset is shown in Figure 5.1. The dataset is divided into training, validation, and test sets taking 70%, 10%, and 20% of the dataset respectively.

**Input and target configuration** The 2400 time steps of the time series are divided into input and target variables as follows. For the persistent state RNN we use a long input window of 2000 time steps, while for the other models, we use a shorter input window of 100 time steps. The target, instead, is 400 time steps long. In the case of linear or nonpersistent state RNN, the first 1900 time steps are not used, while the following 100 time steps are used as input (see Figure 3.9). In the case of persistent state RNN, the first 1900 time steps are used to initialize the hidden state of the RNN. In the case of ESN model, all the 2000 time steps are processed by the reservoir and the last representation is given to the readout layer. In the case of the ReRNN, the first 1900 time steps are processed by the reservoir and the last representation is used to create an encoding to initialize the hidden state of the RNN layer that processes the last 100 time steps. Since the length of the dataset is relatively short and the number of time series composing the

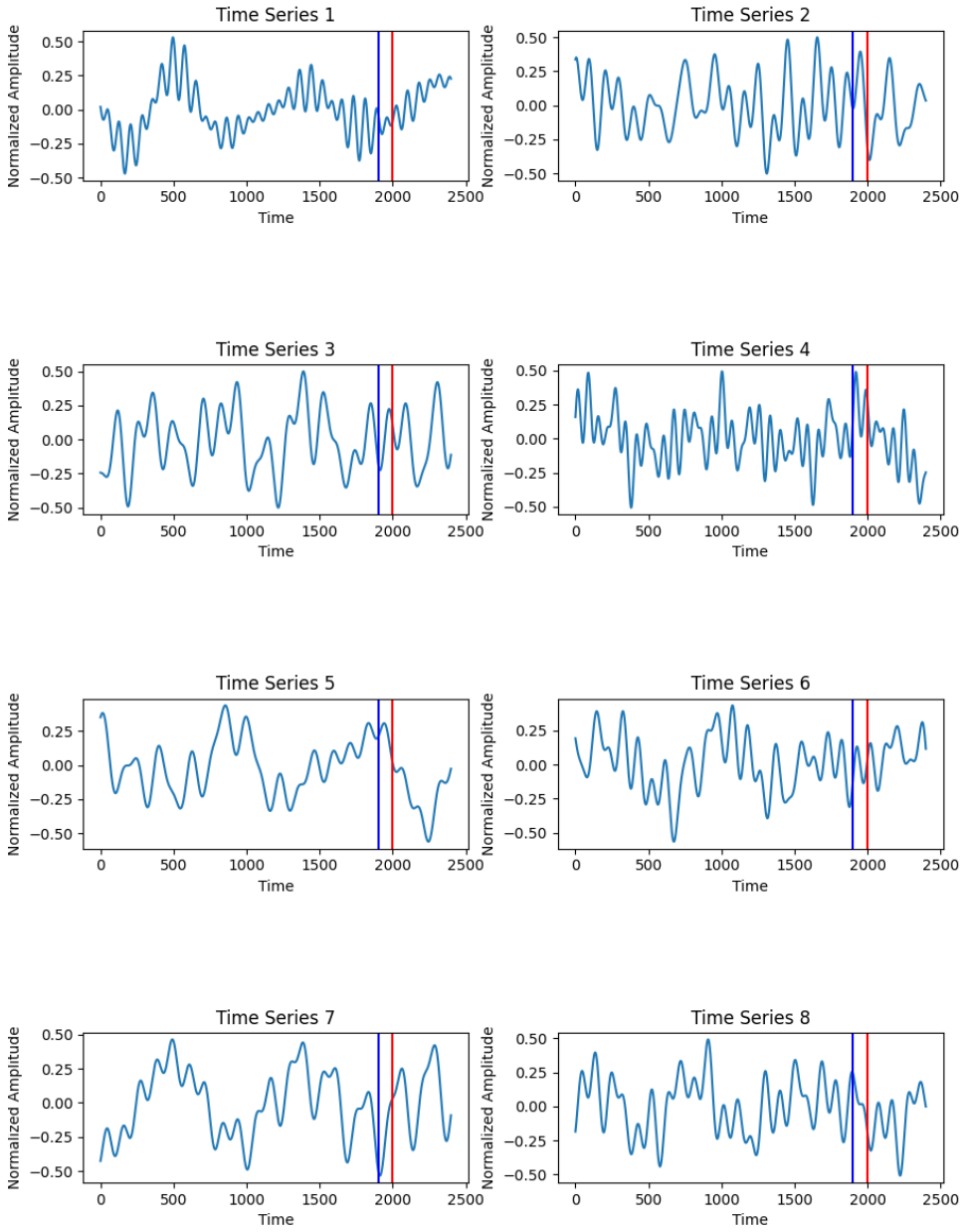



Figure 5.1: A view of eight random time series of the synthetic dataset.
The input data from the start to the blue line is used to find the hidden state in the persistent state RNN and in the ReRNN. The window from the blue line to the red line is used for the training of the RNNs and of the linear model. The target is from the red line to the end of the time series.

dataset is high, we do not need to use the sliding window approach to create the batches.

**Models comparison** We compare the ReRNN with the baseline models described in section 3.3.

We measure the performances of the ReRNN to assert whether it performs better than the baseline models on the synthetic dataset. The persistent state RNN is expected to perform better than the nonpersistent state RNN since it embeds the first part of the time series in its hidden state, while the nonpersistent state RNN does not use the first part of the time series. The same holds for the ReRNN, which also embeds the first part of the time series but uses the reservoir.

The performances in terms of GPU utilization and memory consumption of the ESN and persistent state RNN must be measured to assert if the ReRNN is more efficient. In particular, the persistent state RNN has a delay due to the processing of the first part of the time series to initialize the hidden state, while the ReRNN can process that part ahead of training. On the other hand, the ReRNN has an additional linear (or MLP) layer used to embed the representation given by the reservoir, which is large and could hence bring a delay in the training phase.

**Metrics** The MAE is used to calculate the loss to train the models The metrics used are MAE, MSE, MAPE and mean relative error (MRE) section 2.6.

**Hardware specifications** The experiments were run on a cluster of four NVIDIA RTX A5000 GPUs with 24 GB of memory each and with PCIe Gen4 @ x16.

**Software specifications** The code is written in Python and the libraries used are PyTorch, PyTorch Lightning, Numpy and, since we plan to extend our work to include graph neural networks (GNNs) in the future, Torch Spatiotemporal [23].

### 5.1.2. Second experiment: performance comparison on real-word data

#### Metr-La dataset

Metr-La dataset is a *"real-world large-scale"* traffic dataset collecting traffic speed measurements along the *"highway of Los Angeles County"* by 207 sensors during four moths [76]. The traffic speed is aggregated into 5-minute windows, and it is normalized with the Z-Score normalization [76]. The length of the Metr-La time series is 34272 time steps

| Dataset | # Steps | # Time series | Granularity | Window length | Horizon length |
|---|---|---|---|---|---|
| SYNTHETIC-DATASET | 2400 | 15000 | - | 100 | 400 |
| METR-LA | 34272 | 207 | 5 minutes | 96 (eight hours) | 96 |
| CEREn-SME | 25727 | 485 | 30 minutes | 96 (two days) | 96 |

Table 5.1: Number of steps and time series for each dataset, the granularity of the sampling, the window length (with the corresponding meaning in hours or days) and the horizon length.

Table 5.1. Since the dataset is aggregated with a 5-minute granularity, we use 96 time steps for the window and target, which correspond to eight hours.

**Sliding window approach, input and target configuration** The dataset is divided sequentially into training, validation, and test sets. The training set is 70% of the dataset, the validation set is 10% of the dataset, and the test set is 20% of the dataset. The dataset has a granularity of 30 minutes. The sliding window approach is used to extract the input and target for the batches with a stride between them equal to one. The number of time steps of the input in each batch is equal to 96 when training the linear model, the nonpersistent state RNN and the ReRNN. Otherwise, if we are training the persistent state RNN, the number of time steps of the input is equal to 288, which corresponds to the number of time steps in 24 hours. We need to increase the window for the persistent state RNN because the first 192 time steps, corresponding to the number of time steps in 16 hours, are used to initialize the hidden state of the RNN, while the last 96 time steps, corresponding to eight hours, are processed by the RNN. The horizon is also fixed to 96 time steps, which corresponds to eight hours.

**Metrics** We use the same metrics that were used in the first experiment subsection 5.1.1.

**Models comparison** We compare the ReRNN with the baseline models described in section 3.3.

**Hardware specifications** The same as it was used for the first experiment subsection 5.1.1.

## CEREn-SME dataset

The CEREn-SME dataset is a real-world electricity dataset. The dataset was collected by the Commission for Energy Regulation (CER) which is the Irish regulator for the electricity and natural gas sectors [28]. It is a collection of over 5000 Irish homes and

small/medium businesses' electricity consumption collected between 2009 and 2010 for The Smart Metering Electricity Customer Behaviour Trials (CBTs).

**Sliding window approach, input and target configuration** The datasets are divided sequentially into training, validation, and test sets. The training set is 70% of the dataset, the validation set is 10% of the dataset, and the test set is 20% of the dataset. The sliding window approach is used to extract the input and target for the batches with a stride between them equal to one. The number of time steps of the input in each batch is equal to 96 when training the linear model, the nonpersistent state RNN and the ReRNN. Otherwise, if we are training the persistent state RNN, the number of time steps of the input is equal to 432. We need to increase the window for the persistent state RNN because the first 336 time steps, corresponding to the number of time steps in a week, are used to initialize the hidden state of the RNN, while the last 96 time steps, corresponding to 2 days, are processed by the RNN Table 5.1. The horizon is also fixed to 96 time steps. The length of the CEREn-SME time series is 25727 time steps Table 5.1.

**Metrics** We use the same metrics that were used in the first experiment subsection 5.1.1.

**Models comparison** We compare the ReRNN with the baseline models described in section 3.3.

**Hardware specifications** The same as it was used for the first experiment subsection 5.1.1.

### 5.1.3. Evaluation protocol and hyperparameters search

All the results are averaged over three runs on three different seeds. The best configuration for each model was found with a search over a set of hyperparameters performed using the validation set. To ensure fairness in the evaluation, we use in the search the same hyperparameters for all the models.

For all the models, we use an equal set of learning rates and batch sizes that were found through an extensive manual search to identify a quite wide range of potential good values according to the performance on the validation set. For a second time, the search was focused on the best-performing ones on the specific dataset and model under examination.

For the RNNs (nonpersistent state RNN, persistent state RNN and ReRNN) we vary over the hidden size section 3.3. For the ReRNN we have additional parameters including the spectral radius and the leaking rate of the reservoir, density of the reservoir matrix,

the dimension and number of the hidden layers, the possibility to use the Euler reservoir subsection 3.2.4 and, eventually, the parameter gamma of the Euler layer.

Since the hyperparameter space for the ReRNN is large, we focused our search on a carefully selected set of hyperparameters that are expected to be effective for the task. Given the time constraints, this approach allowed us to efficiently optimize the model. A search over a broader set of hyperparameters could provide additional insights into the ReRNN model's potential. We believe that the search performed is sufficient to demonstrate the ReRNN's effectiveness, although better metrics' values might be achievable with different hyperparameter choices.

## 5.2. Results

In this section, we verify if it is possible to have the same capacity to memorize long-term dependencies of persistent state RNNs using the ReRNN and we collect the test metrics of the models on the datasets. At the same time, we assess the performance in terms of GPU utilization, memory consumption, and speed of the ReRNN in comparison to the baselines on the datasets. We perform an ablation study to assess the contribution of the components of the ReRNN to the final performance.

### Results on the datasets

In Table 5.2 we report the results of the experiments on the three datasets. The performances of the models are evaluated using the MAE, MSE, MAPE and MRE metrics. In order to have a better view of the performances of the models, we report a plot of the mean and the standard deviation over three different seeds of all the values of the test metrics for each model on the three datasets in Figure 5.2–5.3–5.4, and with a focus on the test MAE in Figure 5.5–5.6–5.7.

We report a plot showing the MAE at all the time steps of the horizon for all the models in Figure 5.10–5.9–5.8. This plot is used to compare the different behaviors of the models at different time steps.

In Table 5.3 we list the hyperparameters used for the experiments. The choice of the hyperparameters is the result of a search over a set of hyperparameters for each model and was chosen to minimize the MAE loss on the validation set. We fix a batch size for all the models on the same dataset, and also in this case, the choice of the batch size is the result of a search over a set of batch sizes.

In Table 5.4 we report the performances of the models on the synthetic dataset in terms

of GPU memory consumption, GPU usage and the number of batches per second in the training phase Figure 5.11.

In Table 5.5–5.6 we report the result in terms of metrics and efficiency performance of an ablation study performed to access the contribution of the various components of the ReRNN to the final performance Figure 5.13–5.12.

## 5.3. Result analysis

### 5.3.1. Multi step-ahead forecasting performances

Table 5.2 shows the mean and standard deviation of the test metrics over the three runs for the models. The plots in Figure 5.2–5.3–5.4 help to visualize the findings, and the plots in Figure 5.5–5.6–5.7 have a focus on the MAE metric, that is the one minimized during the validation phase.
As expected, consistently across all the models, the test metrics have higher values for the linear models, followed by the nonpersistent state RNN and the persistent state RNN. Indeed the linear layer does not perform well in this setting and the nonpersistent state RNN does not keep track of the dependencies across the windows. Our implementation of the persistent state RNN, therefore, is able to outperform the prediction quality of the nonpersistent state RNN thanks to the initialization of its hidden state with the encoding of the time steps preceding the input window. how to extract the important information of the time steps that process to initialize the hidden state. Although the persistent state RNN performs better than the nonpersistent state RNN, it may struggle to encode into a compressed representation the long history of the time series.

The ReRNN model outperforms the linear model, the persistent state RNN, and the nonpersistent state RNN, showing that the reservoir effectively keeps track of the past dependencies and helps the RNN, trained on the last part of the time series, to make more accurate predictions than having a random initial hidden state or a hidden state initialized as is done for the persistent state RNN.

The ESN model, having a large representation of the history of the time series, can keep track of the past dependencies. However, mapping the representation created by the reservoir to the target with a linear layer is less effective than using the ReRNN. The ESN on the synthetic dataset, indeed, has a lower MSE than the persistent state RNN and higher than ReRNN, while it has a higher MAE than both the persistent state RNN and the ESN.

Considering equal window sizes, the ReRNN model outperforms all the other models in all the settings considered. This result is in line with our expectations.

We decided to add to the comparison the full backpropagation through time recurrent neural network (FullBPTT RNN), which is an RNN trained on a bigger window than the one considered for the other models. It receives the same number of time steps as the persistent state RNN, but it performs full backpropagation through time, while the persistent state RNN performs truncated backpropagation through time. In the case of the sliding window approach, it trains on a window as large as the number of time steps that the persistent state RNN uses to initialize the hidden state plus the number of time steps to process the input. In the synthetic dataset, the window is as large as the whole time series.

The FullBPTT RNN model utilizes the full window, thus it could be able to achieve lower loss compared to the ReRNN model. These advantages also have drawbacks in decreased speed and increased memory consumption, together with the loss of the possibility of using mini-batch training that we applied consistently in all the other models. Indeed, in case of memory or computational constraints, the use of FullBPTT RNN could be impractical.

In the case of the synthetic dataset, the FullBPTT RNN model outperforms the ReRNN model in terms of forecasting accuracy, showing the existence of a margin of improvement for the ReRNN model. This suggests that the ReRNN model might need a larger representation or a more in-depth fine-tuning of the hyperparameters. However, increasing further the hidden size decreases the performance of the model in terms of speed and memory consumption. Finding a good size for the hidden layer of the reservoir is a trade-off between the performances and the resources needed to train the model.

In the case of the Metr-La dataset, the ReRNN model outperforms the FullBPTT RNN model, showing that in this setting, our approach outperforms a fully-trainable RNN with a much bigger window as input. One possible explanation might be that Fully-TrainedRNN, being trained on a larger window, is more difficult to train and may overfit the training data. Moreover, we ensured to provide the FullBPTT RNN with a window that we considered large enough to capture some significant patterns in the data. For example, we chose to add 24 hours in the MetrLa dataset and 9 days in the CEREn-SME dataset subsection 5.1.2.

Lastly, the FullBPTT RNN model outperforms the ReRNN model in the CEREn-SME dataset, but the performances are very close so that choosing among the two models becomes a matter of choice between the resources needed to train the model and the need for a more accurate prediction.

Now, let's consider each metric in detail. MAE and MSE are scale-dependent metrics, so they are not used to compare the performances across different datasets. Due to the square factor, the MSE is more sensitive to big errors and outliers than the MAE. The MAE and MSE create the same ranking of the models inside each dataset, having as a winner the ReRNN model in the Metr-La dataset and the FullBPTT RNN in the CEREn-SME and synthetic datasets. Moreover, the ReRNN and the FullBPTT RNN are significantly better than the other models, also considering the standard deviation of the metrics.

MAPE and MRE measure how big the errors are compared to the actual values. Both the MAPE and MRE are problematic metrics for datasets with very small values since the division by a small number can lead to very high values, or when the dataset includes a mix of positive and negative values since they cannot maintain a scale to reflect the error magnitude relative to the actual values.

This is the case for the synthetic dataset, where the input after the normalization can be also negative and for the CEREn-SME dataset where the time series can assume very small values. We report the MAPE and MRE for completeness, but we focus on the MAE and MSE to evaluate the models' performances.

On the Metr-La dataset, the MAPE has the highest values for the linear model, nonpersistent state RNN and persistent state RNN and decreases, as expected, for the FullBPTT RNN and for the ReRNN, for which it reaches the lowest value. The MRE is an alternative measure of an absolute error, and for the two real-world datasets, it assumes values for which there is an analogous discussion to the one made for the MAPE.

Since the linear regression used for the ESN is solved using the Ordinary Least Squares (OLS) method, the relative standard deviations between the runs are very low. The linear regression with OLS for the real-world dataset was not performed due to constraints.

| Dataset | Model | MAE | | MSE | | MRE | | MAPE(%) | |
|---|---|---|---|---|---|---|---|---|---|
| | | Mean | Std | Mean | Std | Mean | Std | Mean | Std |
| synthetic | Linear | 0.1328 | 0.0001 | 0.0297 | 0.0000 | 59.0154 | 0.0303 | 310.0975 | 4.5779 |
| | Nonpersistent state RNN | 0.1091 | 0.0002 | 0.0216 | 0.0001 | 48.4634 | 0.0876 | 354.4901 | 6.7968 |
| | Persistent state RNN | 0.0990 | 0.0033 | 0.0175 | 0.0010 | 43.9807 | 1.4683 | 320.1618 | 7.2467 |
| | ESN | 0.0991 | 0.0000 | 0.0158 | 0.0000 | 46.3428 | 0.0000 | 365.5543 | 0.0000 |
| | ReRNN | 0.0752 | 0.0018 | 0.0111 | 0.0005 | 33.4077 | 0.08030 | 277.2603 | 4.3886 |
| | FullBPTT RNN | **0.0352** | 0.0070 | **0.0033** | 0.0010 | **15.6472** | 3.0984 | **148.7451** | 25.8458 |
| MetrLa | Linear | 9.4317 | 0.0325 | 296.7985 | 0.3476 | 0.1754 | 0.0009 | 19.7536 | 0.3194 |
| | Nonpersistent state RNN | 7.2722 | 0.0925 | 248.9153 | 7.1220 | 0.1353 | 0.0018 | 18.0426 | 0.1271 |
| | Persistent state RNN | 5.6781 | 0.0887 | 173.9713 | 3.6671 | 0.1056 | 0.0018 | 12.7854 | 0.5896 |
| | ReRNN | **4.1626** | 0.0505 | **100.0896** | 0.7821 | **0.0773** | 0.0010 | **9.5033** | 0.1683 |
| | FullBPTT RNN | 5.1189 | 0.0577 | 151.3853 | 2.4033 | 0.0952 | 0.0013 | 11.2917 | 0.4764 |
| CEREn-SME | Linear | 0.7880 | 0.0095 | 3.0250 | 0.00855 | 0.3770 | 0.0028 | 163.0744 | 3.8996 |
| | Nonpersistent state RNN | 0.6411 | 0.0087 | 1.9930 | 0.0656 | 0.3069 | 0.0031 | 103.5224 | 5.8567 |
| | Persistent state RNN | 0.5511 | 0.0048 | 1.4943 | 0.0258 | 0.2651 | 0.0039 | 87.9796 | 0.8515 |
| | ReRNN | 0.5134 | 0.0087 | 1.3347 | 0.0511 | 0.2449 | 0.0021 | 71.9698 | 1.7502 |
| | FullBPTT RNN | **0.4837** | 0.0021 | **1.2056** | 0.0049 | **0.2345** | 0.0006 | **70.6963** | 2.4231 |

Table 5.2: Performance metrics for the datasets.
We report the test metrics averaged on the horizon length for all the datasets. For each metric, we report the mean and standard deviation of the runs performed with three different seeds. The numbers in bold are the ones with the best metric scores, while the underlined numbers indicate the second-best scores.

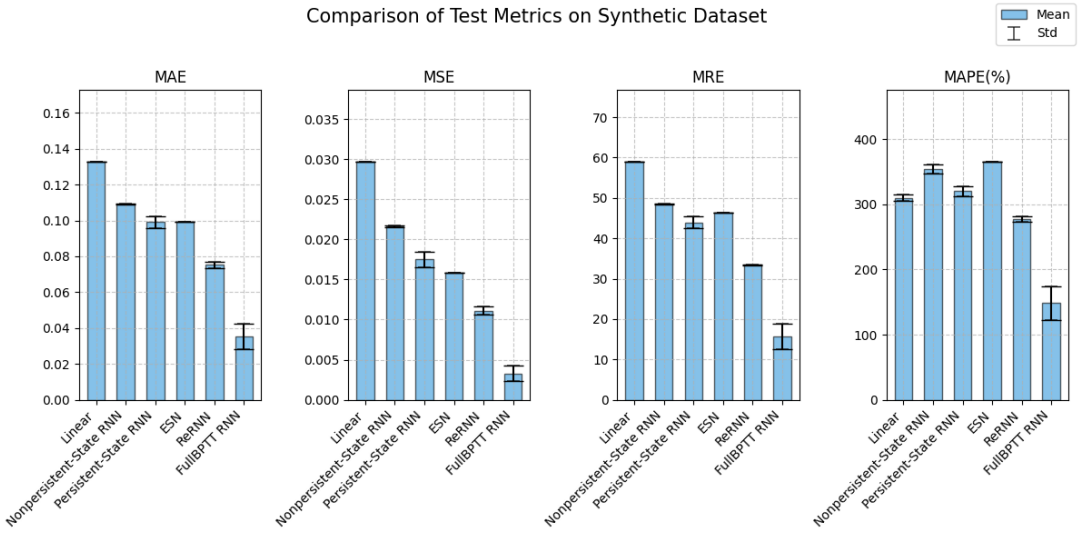



Figure 5.2: Plot of the mean and standard deviation of the test MAE, MSE, MRE and MAPE over the horizon for the models on the synthetic dataset.

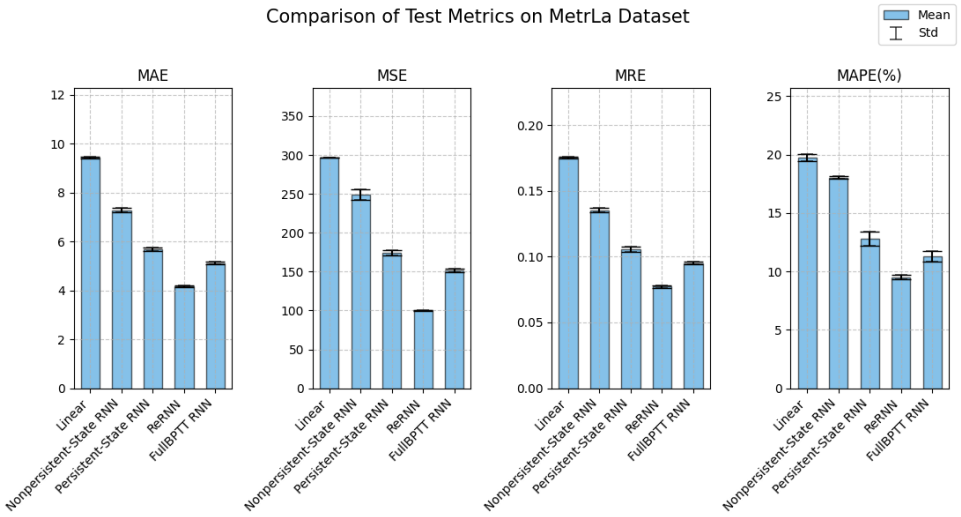



Figure 5.3: Plot of the mean and standard deviation of the test MAE, MSE, MRE and MAPE over the horizon for the models on the MetrLa dataset.

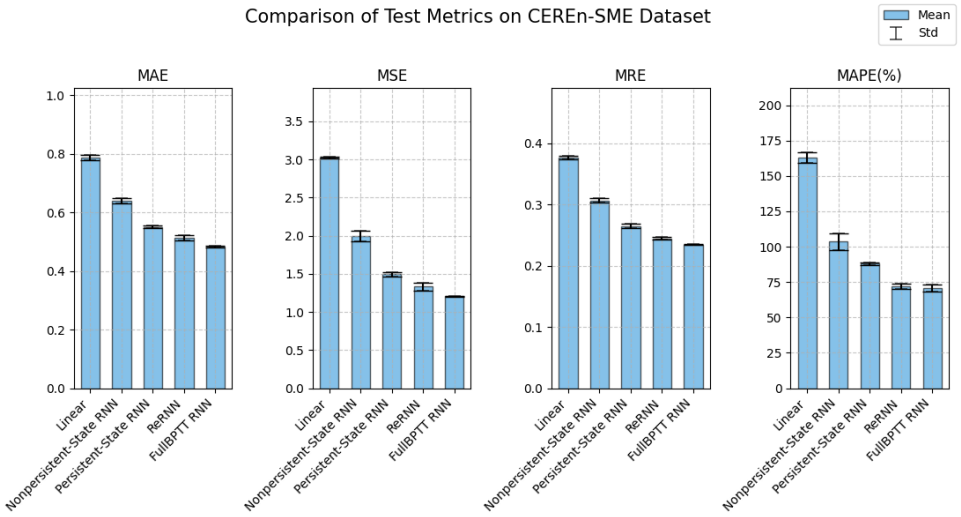



Figure 5.4: Plot of the mean and standard deviation of the test MAE, MSE, MRE and MAPE over the horizon for the models on the CEREn-SME dataset.

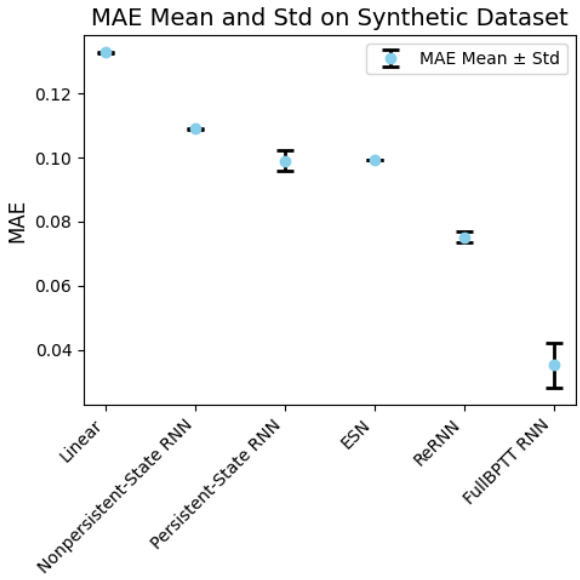



Figure 5.5: Plot of the mean and standard deviation of the test MAE metric over the horizon for the models on the synthetic dataset.

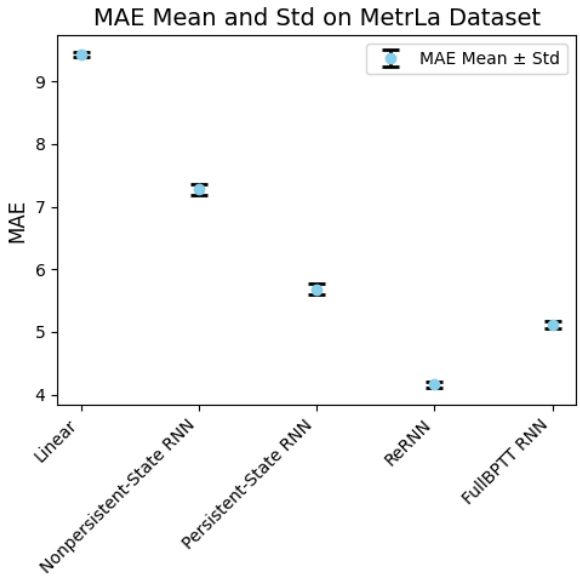



Figure 5.6: Plot of the mean and standard deviation of the test MAE metric over the horizon for the models on the MetrLa dataset.

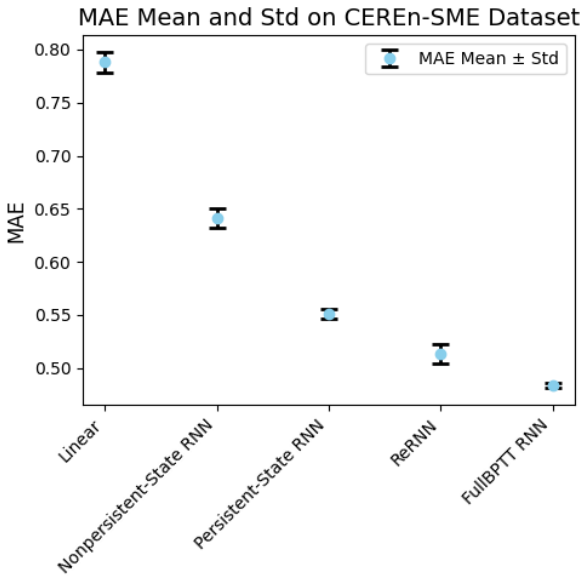



Figure 5.7: Plot of the mean and standard deviation of the test MAE metric over the horizon for the models on the CEREn-SME dataset.

### 5.3.2. Performance at specific time steps of the horizon

We plot the MAE at all the time steps of the horizon to have an insight into the models' different behaviors and their ability to predict further into the future.

In the case of the synthetic dataset (Figure 5.8), the line of the MAE test over the horizon of the FullBPTT RNN is almost straight, increasing progressively with the horizon in a linear way.

The ReRNN has almost the same behavior, having a bit faster increase in the first time steps of the horizon.

In the case of the persistent state RNN, we see a rapid increase in the first time steps of the horizon, then the line becomes almost straight. This behavior shows a rapidly increasing difficulty in predicting further into the future in the first quarter of the horizon.

For the nonpersistent state RNN and the linear model, the two highest MAE values are reached at the end of the horizon (as in the case of the other models), and before the halfway of the horizon. This can be due to the fact that these models see only the last 100 time steps of the input, and they may be blind to some short-term patterns. Hence, they may need a longer window.

Overall, in the synthetic dataset, the FullBPTT RNN and the ReRNN show the most

linear behavior along the horizon.

In the case of the Metr-La dataset Figure 5.9, the ReRNN, after a step at the beginning of the horizon, presents a flat line increasing just at the last time steps of the horizon. This suggests that the ReRNN is able to predict time steps far into the future with the same metric performance as the time steps near the present.

The FullBPTT RNN and the persistent state RNN have similar behavior, but the line is slowly increasing with the horizon, and the one of persistent state RNN is a bit steeper.

The nonpersistent state RNN and the linear model have a less flat behavior, showing they struggle to predict the time steps further in the future.

In the case of the CEREn-SME dataset Figure 5.10, it can be seen how, after a little step at the beginning of the horizon, the FullBPTT RNN and the ReRNN have an almost flat line, suggesting that the models are able to predict time steps far ahead in the future as well as the time steps near the present.

In the case of the persistent state RNN, the line is almost flat, but it has a little step also in the middle of the horizon, showing that the model predicts the second half of the horizon with a bit more difficulty than the first half.

This step at the middle of the horizon is more pronounced in the case of the nonpersistent state RNN and even more in the case of the linear model. This may be due to the fact that the data within the window is not sufficient to predict longer-term patterns, but only shorter-term patterns.

### 5.3.3. Hyperparameters

In Table 5.3 are reported the most important hyperparameters used for the models.

As we can see, across all the experiments, the best hidden size for the reservoir is high. Indeed, since the reservoir is not trained, it needs a big representation to capture all the dynamics of the input [60]. Moreover, we need to choose appropriate hyperparameters to design the reservoir so that it captures some specific dynamics of the input. In our case we want the reservoir to remember the past.

The experiments across all the datasets confirm this observation since the best leaking rate found is always a small number. Indeed, as explained in subsection 3.3.4, the lower

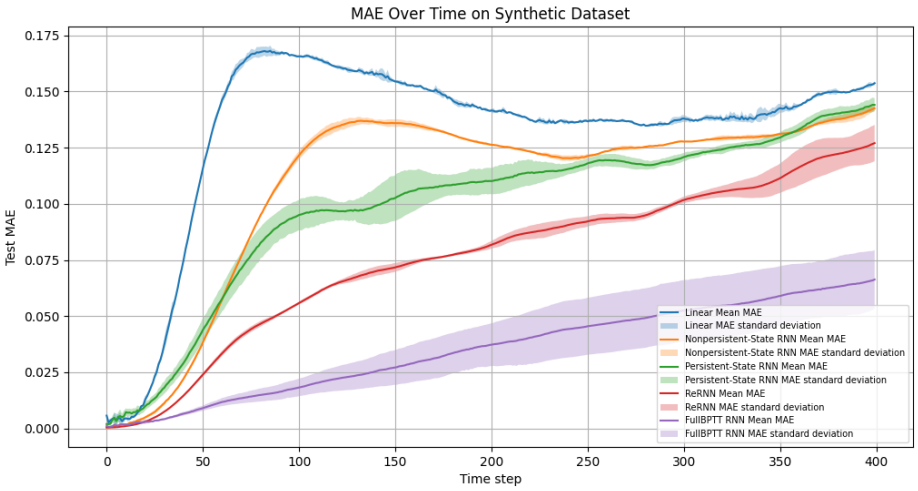



Figure 5.8: Plot of the mean and standard deviation of the test MAE at each time step (400 in total) of the horizon for the models on the synthetic dataset.

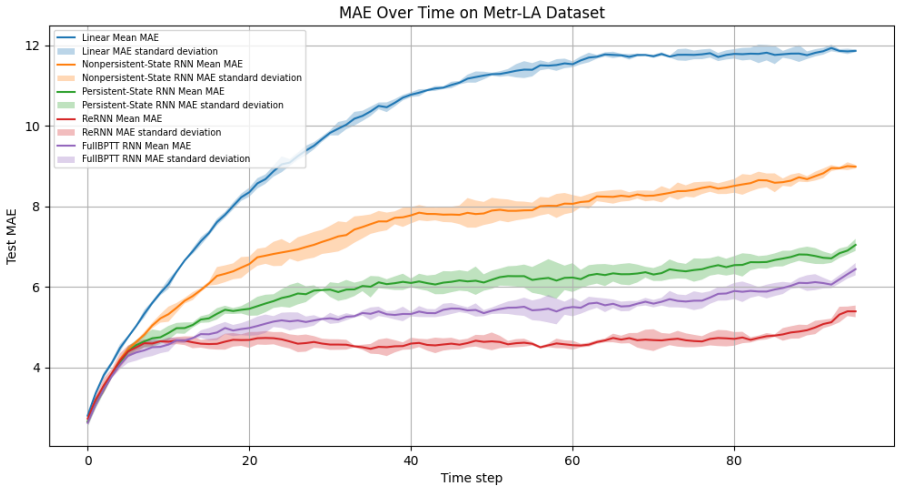



Figure 5.9: Plot of the mean and standard deviation of the test MAE at each time step (96 in total) of the horizon for the models on the MetrLa dataset.

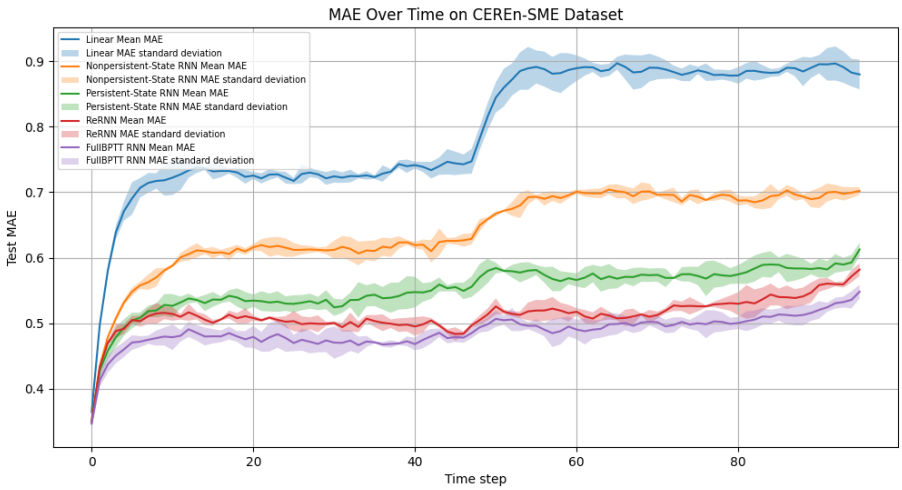



Figure 5.10: Plot of the mean and standard deviation of the test MAE at each time step (96 in total) of the horizon for the models on the CEREn-SME dataset.

the leaking rate the more the reservoir remembers the past. Moreover, we observe that the spectral radius is always close to one, and we know from the theory that the closer the spectral radius to one the more the reservoir has chaotic dynamics (see subsection 3.3.4).

The leaking rate in the synthetic dataset is even smaller than in the other datasets. This can be due to the fact that the input window is longer and, therefore, the reservoir needs to remember the past information for longer.

The spectral radius reported in Table 5.3 are always smaller than one. This is the requirement for the reservoir to have the echo state property [60]. However, during our experiments for the ESN on the synthetic dataset, we noticed a slight improvement in the performances when the spectral radius was set to 1.1. This behavior suggests that the reservoir can capture the dynamics of the data better when the dynamics are chaotic, which is already known from the theory subsection 3.3.4.

In the table, we report the ESN on the synthetic dataset with the best validation loss (the one with spectral radius 1.1), and we don't report the exact values when the spectral radius is set to 0.999 (that would make all the parameters used for the ESN equal to the one used for the ReRNN) because the difference is not significant.

The experiments show a consistent improvement when using the Euler State Network subsection 3.2.4 in both the ESN and ReRNN, across all the datasets. This confirms

the need for the datasets used to have a reservoir that is stable and with a non-fading memory that would otherwise deteriorate the reservoir's ability to capture the long-range dependencies in the data.

The learning rate for the ESN is not reported since it was fitted with a linear regression model and it was not trained with a gradient descent algorithm as the other models.

| Dataset | Model | Batch Size | Learning Rate | Hidden Size RNN | Hidden Size reservoir | Leaking Rate | Spectral Radius | Density | Input Scaling | Gamma (Euler) |
|---|---|---|---|---|---|---|---|---|---|---|
| **synthetic** | Linear | 128 | 0.005 | - | - | - | - | - | - | - |
| | Nonpersistent state RNN | | 0.0005 | 256 | - | | - | - | - | - |
| | Persistent state RNN | | 0.0005 | 256 | - | | - | - | - | - |
| | ESN | | - | - | 1024 | 0.02 | 1.1 | 0.2 | 0.5 | 0.02 |
| | ReRNN | | 0.001 | 256 | 1024 | 0.02 | 0.999 | 0.2 | 0.5 | 0.2 |
| | FullBPTT RNN | | 0.0005 | 256 | - | - | - | - | - | - |
| **MetrLa** | Linear | 256 | 0.0001 | - | - | - | - | - | - | - |
| | Nonpersistent state RNN | | 0.001 | 256 | - | | - | - | - | - |
| | Persistent state RNN | | 0.0005 | 256 | - | | - | - | - | - |
| | ReRNN | | 0.0001 | 256 | 1024 | 0.1 | 0.999 | 0.2 | 0.5 | 0.02 |
| | FullBPTT RNN | | 0.0005 | 256 | - | - | - | - | - | - |
| **CERsMe** | Linear | 512 | 0.0005 | - | - | - | - | - | - | - |
| | Nonpersistent state RNN | | 0.001 | 256 | - | - | - | - | - | - |
| | Persistent state RNN | | 0.001 | 256 | - | - | - | - | - | - |
| | ReRNN | | 0.0005 | 256 | 1024 | 0.1 | 0.999 | 0.2 | 0.5 | 0.02 |
| | FullBPTT RNN | | 0.001 | 256 | - | - | - | - | - | - |

Table 5.3: Hyperparameters used in the experiments that were identified through a search that aimed to find the best-performing ones on the validation step for each model.

### 5.3.4. Memory and computational efficiency

We need to compare the performances of the models, with a focus on the persistent state RNN and the ReRNN, because they both have different components (apart from the RNN) that can introduce delays during the training phase.

On one side, the persistent state RNN has a delay due to the processing of the time steps preceding the window with a detached computation. On the other side, the ReRNN has an additional linear (or MLP) layer used to reduce the dimensionality of the representation given by the reservoir, which is large and could hence bring a delay in the training phase.

In Table 5.4 we report the GPU usage and memory utilization of the models on the synthetic dataset and the number of batches processed per second with the relative mean and standard deviation. GPU and memory usage are measured using the NVIDIA System Management Interface, through which those quantities can be monitored in real-time during the training. The number of batches processed per second is calculated by measuring the time between the start of the training and the end of the weights update in the first 30 seconds of the training with the exclusion of the first and last 5 measurements (as done by Cini et al. [24]). Since we use the linear regression for the ESN, we don't report the number of batches per second for this model. The table is visually reported in Figure 5.11.

The GPU usage is very low for the linear model (under the 10%) and for the nonpersistent state RNN (under the 50%), while it is very high for the persistent state RNN (86%). The FullBPTT RNN is the most computationally expensive model, with a GPU usage of 95%. The ReRNN model has a GPU usage slightly higher than the nonpersistent state RNN but considerably lower than the persistent state RNN.

This result is an important achievement of our experiments since it proves the superior computational efficiency of the ReRNN model on a dataset in which the ReRNN outperforms the persistent state RNN with the same window size.

We consider separately the ESN GPU usage since the optimization procedure is performed in closed form with linear regression, and hence a different training procedure with respect to the other models.

The GPU memory utilization is very low for the linear model (0.370 GB) and it is similar between the nonpersistent state RNN and the ESN (0.612 GB and 0.636 GB respectively).

| Dataset | Model | GPU usage (%) | GPU memory (GB) | Batch/S |
|---|---|---|---|---|
| synthetic dataset | linear model | 8 | 0.370 | 267 ± 55 |
| | Nonpersistent state RNN | 40 | 0.612 | 132 ± 20 |
| | Persistent state RNN | 86 | 3.012 | 33 ± 1 |
| | ESN | 100 | 0.636 | - |
| | ReRNN | 47 | 0.820 | 130 ± 19 |
| | FullBPTT RNN | 95 | 3.662 | 16 ± 0 |

Table 5.4: GPU usage, memory consumption, and batch per second (Batch/S) for Different Models on the synthetic dataset.
The nonpersistent state RNN, persistent state RNN, ReRNN and FullBPTT RNN models have the same hidden size of the RNN. ESN and ReRNN have the same hidden size of the reservoir. The higher the Batch/S, the better.

The ReRNN model has a slightly higher memory utilization (0.820 GB) but is way apart from the persistent state RNN (3.012 GB) and the FullBPTT RNN (3.662 GB).

Again, this result is in line with the requirements of memory utilization of the ReRNN model, which is considerably lower than the persistent state RNN.

Lastly, also for the batches per second, we obtain the expected results having a higher number for the linear model, followed by closed numbers for the nonpersistent state RNN and the ReRNN model, and a lower number for the persistent state RNN and the FullBPTT RNN.

In any case, apart from the detailed comparison with the other models, the main advantage of our approach is to enable subsampling in minibatches, which cannot be done in the case of FullBPTT RNN, and which is the main reason for which the computational complexity of the training procedure is reduced and for which our model becomes scalable (see paragraph A.2–A.3).

### 5.3.5. Ablation study

We perform an ablation study on the synthetic dataset to assess that the ReRNN works as expected (better than the persistent state RNN) also replacing the MLP layer that encodes the reservoir hidden state with a simple linear layer. This study is performed to ensure that the ReRNN superiority in terms of the metrics is not due to the additional complexity given by the MLP layer but to the initialization of the RNN with the reservoir representation. The results are reported in Table 5.5 and visualized in Figure 5.12. A study on the memory and computational efficiency is reported in Table 5.6 and visualized

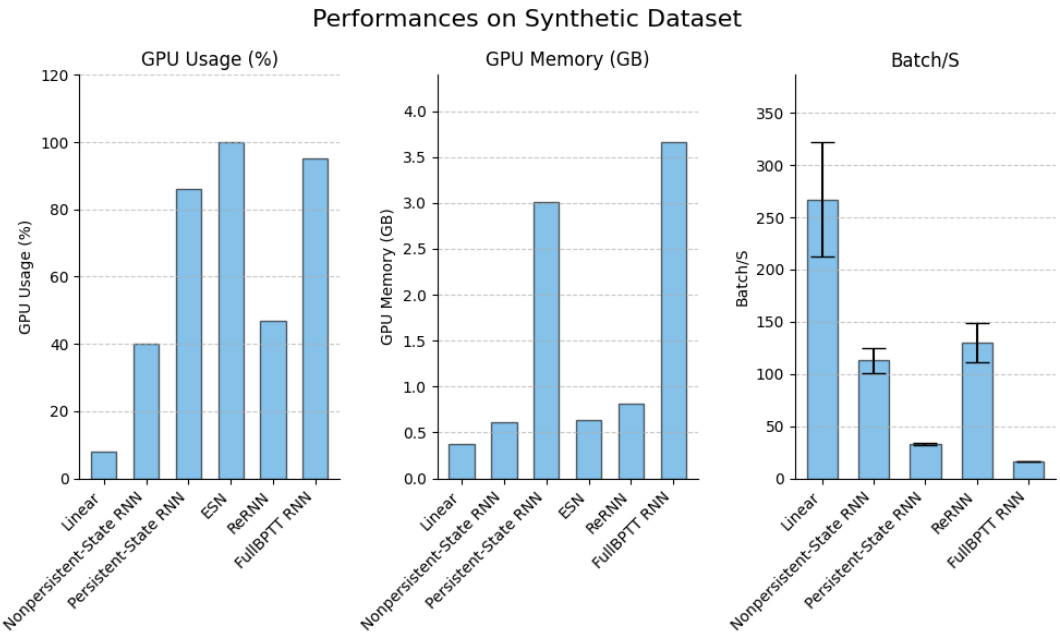



Figure 5.11: Plot of the performances of the models on the synthetic dataset.
We omit the Batch/s value for ESN because we used linear regression to train this model. The mean and standard deviation are calculated over some epochs of the training procedure.

| Dataset | Model | MAE | | MSE | | MRE | | MAPE(%) | |
|---|---|---|---|---|---|---|---|---|---|
| | | Mean | Std | Mean | Std | Mean | Std | Mean | Std |
| synthetic | ReRNN (MLP) | 0.0752 | 0.0018 | 0.0111 | 0.0005 | 33.4077 | 0.08030 | 277.2603 | 4.3886 |
| | ReRNN (Linear layer) | 0.0808 | 0.0026 | 0.0124 | 0.0008 | 35.9156 | 1.1534 | 308.9132 | 3.8364 |

Table 5.5: Ablation study: differences in GPU utilization and memory requirements between ReRNN with an MLP layer or with a Linear layer on the synthetic dataset.

in Figure 5.13. The ReRNN with the linear layer has a slight drop in performance compared to the ReRNN with the MLP layer but it still outperforms the persistent state RNN (see Figure 5.12). There is almost no change in GPU utilization, memory consumption, and speed as it can be visualized in Figure 5.13.

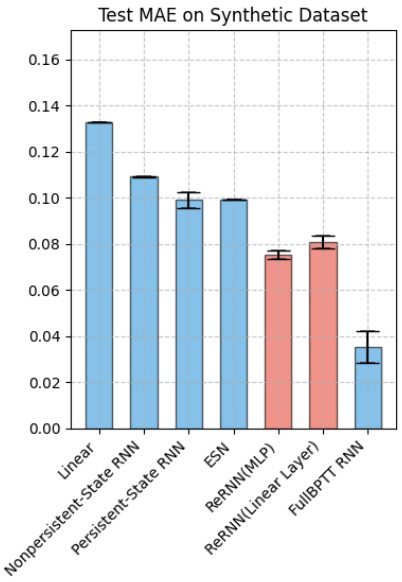



Figure 5.12: Ablation study: plot of the differences in test MAE with a focus on the ReRNN with an MLP layer or with a Linear layer (both colored in red) on the synthetic dataset.

| Dataset | Model | GPU usage (%) | GPU memory (GB) | Batch/S |
|---|---|---|---|---|
| Synthetic dataset | ReRNN (MLP) | 47 | 0.820 | 130 ± 19 |
| | ReRNN (Linear layer) | 48 | 0.760 | 131 ± 18 |

Table 5.6: Ablation study: table of the differences in GPU utilization and memory requirements between ReRNN with an MLP layer or with a Linear layer on the synthetic dataset.

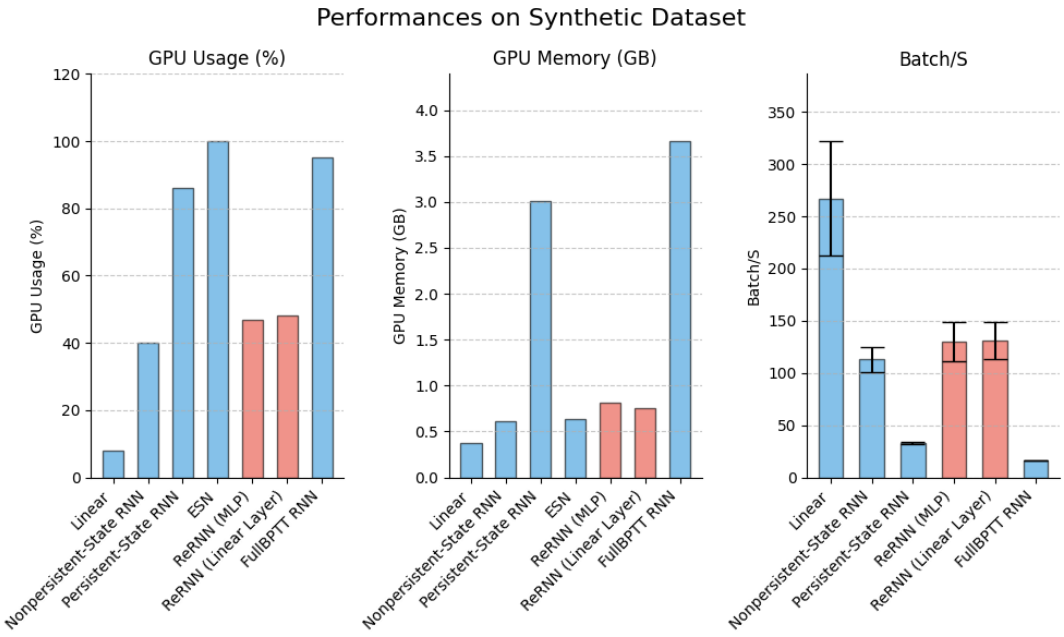



Figure 5.13: Ablation study: plot of the differences in GPU utilization and memory requirements with a focus on the ReRNN with an MLP layer or with a Linear layer (both colored in red) on the synthetic dataset.

# 6 | Conclusions

## 6.1. Implication of the thesis

In this thesis, we examined the issue with existing approaches for efficiently training RNNs. For training RNNs on long time series, it is necessary to process the whole time series in sequential order to propagate the hidden state. Moreover, the sequential processing of the time steps prevents the use of mini-batches leading to memory capacity and scalability issues during the training process.

To address this problem, we proposed ReRNN, a new approach that combines the benefits of reservoir computing to process long sequences of data with the RNNs' capacity to better adapt to the dynamics of the process generating the data on which it is trained. The ReRNN is composed of a train-free reservoir that processes the whole time series ahead of training. Its hidden state is processed with an encoder to create an appropriate initialization for an RNN. In this way, the RNN can be trained in an efficient way on a short window of the time series while accounting for the history of the time series. We tested this method on both real-world and synthetic datasets specifically created to evaluate the capacity to identify long-range dependencies in the data. We collected the test metrics of the baselines and ReRNN models and the performances in terms of GPU utilization, memory consumption, and training speed. Reported results show that our ReRNN outperforms state-of-the-art approaches in terms of forecasting accuracy while keeping memory consumption and GPU utilization low and higher speed in the training phase compared to a FullBPTT RNN.

## 6.2. Limitations

Our approach enables subsampling of the input data during training by encoding historical data in fixed-size representations in a pre-processing step. However, the pre-processing step may require a significant amount of memory if the reservoir is large.

Moreover, the transfer of the reservoir's hidden state on the GPU can be time-consuming,

as the hidden state can be large and requires significant memory and storage.

Nevertheless, our approach remains scalable and computationally efficient, since it enables subsampling in mini-batches which is particularly necessary in the context of long time series.

## 6.3. Future works

The most suitable application of this work is in the context of GNNs. Indeed, GNNs can be used to account for dependencies among time series and this approach could be helpful also in this context. Additionally, it addresses hardware constraints when dealing with distributed sensor networks by sampling the node representations in mini-batches of a size proportional to the hardware capacity. Hence, future work could focus on using the proposed approach on GNNs.

The work can be extended with the integration of techniques that enhance the stability and improve the memory of the reservoir over long time steps. For instance, incorporating delayed hidden states in the reservoir layers that make use of older updates of the hidden state instead of the most recent ones. This technique, and analogous ones, could enhance the performances in terms of metrics and achieve or outperform the FullBPTT RNN. Indeed, the empirical results suggest that, in some scenarios, the reservoir designed with a specific configuration of the hyperparameters was able to reach the same accuracy as its fully trainable counterpart. While this result has already been achieved for some datasets, we aim to establish it for all datasets used in the experiments.

# Bibliography

[1] M. Abdar, M. Samami, S. Dehghani Mahmoodabad, T. Doan, B. Mazoure, R. Hashemifesharaki, L. Liu, A. Khosravi, U. R. Acharya, V. Makarenkov, and S. Nahavandi. Uncertainty quantification in skin cancer classification using three-way decision-based bayesian deep learning. *Computers in Biology and Medicine*, 135:104418, 2021. ISSN 0010-4825. doi: https://doi.org/10.1016/j.compbiomed.2021.104418. URL `https://www.sciencedirect.com/science/article/pii/S0010482521002122`.

[2] P. H. M. Albuquerque, Y. Peng, and J. P. F. d. Silva. Making the whole greater than the sum of its parts: A literature review of ensemble methods for financial time series forecasting. *Journal of Forecasting*, 41(8):1701–1724, 2022.

[3] A. A. Ariyo, A. O. Adewumi, and C. K. Ayo. Stock price prediction using the arima model. In *2014 UKSim-AMSS 16th International Conference on Computer Modelling and Simulation*, pages 106–112, 2014. doi: 10.1109/UKSim.2014.67.

[4] S. Babinec and J. Pospíchal. Merging echo state and feedforward neural networks for time series forecasting, 09 2006.

[5] S. Bai, J. Z. Kolter, and V. Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling, 2018.

[6] J. M. Bates and C. W. J. Granger. The combination of forecasts. *Journal of the Operational Research Society*, 20(4):451–468, 1969. doi: 10.1057/jors.1969.103. URL `https://doi.org/10.1057/jors.1969.103`.

[7] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner, C. Gulcehre, F. Song, A. Ballard, J. Gilmer, G. Dahl, A. Vaswani, K. Allen, C. Nash, V. Langston, C. Dyer, N. Heess, D. Wierstra, P. Kohli, M. Botvinick, O. Vinyals, Y. Li, and R. Pascanu. Relational inductive biases, deep learning, and graph networks, 2018.

[8] Y. Bengio, P. Simard, and P. Frasconi. Learning long-term dependencies with gradi-

ent descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994. doi: 10.1109/72.279181.

[9] K. Benidis, S. S. Rangapuram, V. Flunkert, Y. Wang, D. Maddix, C. Turkmen, J. Gasthaus, M. Bohlke-Schneider, D. Salinas, L. Stella, F.-X. Aubet, L. Callot, and T. Januschowski. Deep learning for time series forecasting: Tutorial and literature survey. *ACM Comput. Surv.*, 55(6), dec 2022. ISSN 0360-0300. doi: 10.1145/3533382. URL `https://doi.org/10.1145/3533382`.

[10] N. Bertschinger and T. Natschläger. Real-time computation at the edge of chaos in recurrent neural networks. *Neural Computation*, 16:1413–1436, 2004. ISSN 1530-888X.

[11] F. M. Bianchi, S. Scardapane, A. Uncini, A. Rizzi, and A. Sadeghian. Prediction of telephone calls load using echo state network with exogenous variables. *Neural Networks*, 71:204–213, 2015. ISSN 0893-6080. doi: https://doi.org/10.1016/j.neunet.2015.08.010. URL `https://www.sciencedirect.com/science/article/pii/S0893608015001732`.

[12] F. M. Bianchi, L. Livi, and C. Alippi. Investigating echo-state networks dynamics by means of recurrence analysis. *IEEE Transactions on Neural Networks and Learning Systems*, 29(2):427–439, 2018. doi: 10.1109/TNNLS.2016.2630802.

[13] F. M. Bianchi, S. Scardapane, S. Løkse, and R. Jenssen. Bidirectional deep-readout echo state networks, 2018.

[14] F. M. Bianchi, S. Scardapane, S. Løkse, and R. Jenssen. Reservoir computing approaches for representation and classification of multivariate time series, 2020.

[15] G. E. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung. *Time series analysis: forecasting and control*. John Wiley & Sons, 2015.

[16] K. H. Brodersen, T. Schofield, A. Leff, C. S. Ong, E. Lomakina-Rumyantseva, J. Buhmann, and K. Stephan. Generative embedding for model-based classification of fmri data. *PLoS computational biology*, 7:e1002079, 06 2011. doi: 10.1371/journal.pcbi.1002079.

[17] R. G. Brown. *Statistical Forecasting for Inventory Control*. McGraw-Hill, 1959.

[18] H. Cecotti. Deep random vector functional link network for handwritten character recognition, 07 2016.

[19] H. Chen, F. Tang, P. Tino, and X. Yao. Model-based kernel for efficient time

series analysis. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '13, page 392–400, New York, NY, USA, 2013. Association for Computing Machinery. ISBN 9781450321747. doi: 10.1145/2487575.2487700. URL https://doi.org/10.1145/2487575.2487700.

[20] H. Chen, F. Tang, P. Tino, A. G. Cohn, and X. Yao. Model metric co-learning for time series classification. In *Proceedings of the 24th International Conference on Artificial Intelligence*, IJCAI'15, page 3387–3394. AAAI Press, 2015. ISBN 9781577357384.

[21] M. Chen, W. Saad, and C. Yin. Echo state networks for self-organizing resource allocation in lte-u with uplink–downlink decoupling. *Trans. Wireless. Comm.*, 16 (1):3–16, jan 2017. ISSN 1536-1276. doi: 10.1109/TWC.2016.2616400. URL https://doi.org/10.1109/TWC.2016.2616400.

[22] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In A. Moschitti, B. Pang, and W. Daelemans, editors, *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Doha, Qatar, Oct. 2014. Association for Computational Linguistics. doi: 10.3115/v1/D14-1179. URL https://aclanthology.org/D14-1179.

[23] A. Cini and I. Marisca. Torch Spatiotemporal, 3 2022. URL https://github.com/TorchSpatiotemporal/tsl.

[24] A. Cini, I. Marisca, F. M. Bianchi, and C. Alippi. Scalable spatiotemporal graph neural networks, 2023.

[25] A. Cini, I. Marisca, D. Zambon, and C. Alippi. Graph deep learning for time series forecasting, 2023.

[26] A. Cini, I. Marisca, D. Zambon, and C. Alippi. Graph deep learning for time series forecasting, 2023. URL https://arxiv.org/abs/2310.15978.

[27] R. T. Clemen. Combining forecasts: A review and annotated bibliography. *International Journal of Forecasting*, 5(4):559–583, 1989. ISSN 0169-2070. doi: https://doi.org/10.1016/0169-2070(89)90012-5. URL https://www.sciencedirect.com/science/article/pii/0169207089900125.

[28] Commission for Energy Regulation (CER). CER Smart Metering Project - Electricity Customer Behaviour Trial, 2009-2010 [dataset], 2012. URL https://www.ucd.

ie/issda/data/commissionforenergyregulationcer/. SN: 0012-00. Accessed via the Irish Social Science Data Archive - https://www.ucd.ie/issda.

[29] B. Csáji. Approximation with artificial neural networks, 06 2001.

[30] A. Daniely, R. Frostig, and Y. Singer. Toward deeper understanding of neural networks: The power of initialization and a dual view on expressivity. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016. URL https://proceedings.neurips.cc/paper_files/paper/2016/file/abea47ba24142ed16b7d8fbf2c740e0d-Paper.pdf.

[31] A. Daniely, R. Frostig, and Y. Singer. Toward deeper understanding of neural networks: The power of initialization and a dual view on expressivity, 2017.

[32] L. Du, R. Gao, P. N. Suganthan, and D. Z. Wang. Graph ensemble deep random vector functional link network for traffic forecasting. *Applied Soft Computing*, 131:109809, 2022. ISSN 1568-4946. doi: https://doi.org/10.1016/j.asoc.2022.109809. URL https://www.sciencedirect.com/science/article/pii/S1568494622008584.

[33] J. L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990. ISSN 0364-0213. doi: https://doi.org/10.1016/0364-0213(90)90002-E. URL https://www.sciencedirect.com/science/article/pii/036402139090002E.

[34] J. Frankle and M. Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks, 2019.

[35] F. Frasca, E. Rossi, D. Eynard, B. Chamberlain, M. Bronstein, and F. Monti. Sign: Scalable inception graph neural networks, 2020.

[36] C. Gallicchio. Tutorial on Deep Reservoir Neural Networks. https://sites.google.com/site/cgallicch/resources/tutorial_DRNN, 2017.

[37] C. Gallicchio. Euler state networks: Non-dissipative reservoir computing, 2023. URL https://arxiv.org/abs/2203.09382.

[38] C. Gallicchio and A. Micheli. Fast and deep graph neural networks, 2019.

[39] C. Gallicchio and S. Scardapane. Deep randomized neural networks, 2021.

[40] C. Gallicchio, J. Martín-Guerrero, A. Micheli, and E. Olivas. Randomized machine learning approaches: Recent developments and challenges, 04 2017.

[41] C. Gallicchio, A. Micheli, and L. Pedrelli. Deep reservoir computing: A critical experimental analysis. *Neurocomputing*, 268:87–99, 2017. ISSN 0925-2312. doi: https://doi.org/10.1016/j.neucom.2016.12.089. URL https://www.sciencedirect.com/science/article/pii/S0925231217307567. Advances in artificial neural networks, machine learning and computational intelligence.

[42] C. Gallicchio, A. Micheli, and L. Pedrelli. Design of deep echo state networks. *Neural networks: the official journal of the International Neural Network Society*, 108:33–47, 2018. URL https://api.semanticscholar.org/CorpusID:52075702.

[43] R. Gao, W. X. Cheng, P. N. Suganthan, and K. F. Yuen. Inpatient discharges forecasting for singapore hospitals by machine learning. *IEEE Journal of Biomedical and Health Informatics*, 26(10):4966–4975, 2022. doi: 10.1109/JBHI.2022.3172956.

[44] R. Gao, R. Li, M. Hu, P. N. Suganthan, and K. F. Yuen. Dynamic ensemble deep echo state network for significant wave height forecasting. *Applied Energy*, 329:120261, 2023. ISSN 0306-2619. doi: https://doi.org/10.1016/j.apenergy.2022.120261. URL https://www.sciencedirect.com/science/article/pii/S0306261922015185.

[45] A. Gasparin, S. Lukovic, and C. Alippi. Deep learning for time series forecasting: The electric load case, 2019.

[46] R. C. Gerum and A. Schilling. Integration of leaky-integrate-and-fire neurons in standard machine learning architectures to generate hybrid networks: A surrogate gradient approach. *Neural Computation*, 33(10):2827–2852, Sept. 2021. ISSN 1530-888X. doi: 10.1162/neco_a_01424. URL http://dx.doi.org/10.1162/neco_a_01424.

[47] R. Giryes, G. Sapiro, and A. M. Bronstein. Deep neural networks with random gaussian weights: A universal classification strategy? *IEEE Transactions on Signal Processing*, 64(13):3444–3457, 2016. doi: 10.1109/TSP.2016.2546221.

[48] Z. Gong, H. Chen, B. Yuan, and X. Yao. Multiobjective learning in the model space for time series classification. *IEEE Transactions on Cybernetics*, 49(3):918–932, 2019. doi: 10.1109/TCYB.2018.2789422.

[49] C. W. J. Granger. Investigating causal relations by econometric models and cross-spectral methods. *Econometrica*, 37(3):424–438, 1969. URL https://doi.org/10.2307/1912791. Accessed 23 July 2024.

[50] A. Graves and J. Schmidhuber. Framewise phoneme classification with bidirectional

lstm and other neural network architectures. *Neural Networks*, 18(5):602–610, 2005. ISSN 0893-6080. doi: https://doi.org/10.1016/j.neunet.2005.06.042. URL https://www.sciencedirect.com/science/article/pii/S0893608005001206. IJCNN 2005.

[51] Z. Hajiabotorabi, A. Kazemi, F. F. Samavati, and F. M. Maalek Ghaini. Improving dwt-rnn model via b-spline wavelet multiresolution to forecast a high-frequency time series. *Expert Systems with Applications*, 138:112842, 2019. ISSN 0957-4174. doi: https://doi.org/10.1016/j.eswa.2019.112842. URL https://www.sciencedirect.com/science/article/pii/S0957417419305445.

[52] W. L. Hamilton, Z. Ying, and J. Leskovec. Graph representation learning. *arXiv preprint arXiv:1709.05584*, pages 48–66, 2017.

[53] W. L. Hamilton, R. Ying, and J. Leskovec. Inductive representation learning on large graphs, 2018.

[54] J. Han, H. Liu, H. Zhu, H. Xiong, and D. Dou. Joint air quality and weather prediction based on multi-adversarial spatiotemporal networks, 2021.

[55] J. Hao and F. Liu. Improving long-term multivariate time series forecasting with a seasonal-trend decomposition-based 2-dimensional temporal convolution dense network. *Scientific Reports*, 14:1689, 2024. doi: 10.1038/s41598-024-52240-y. URL https://doi.org/10.1038/s41598-024-52240-y.

[56] S. Hochreiter and J. Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780, 11 1997. ISSN 0899-7667. doi: 10.1162/neco.1997.9.8.1735. URL https://doi.org/10.1162/neco.1997.9.8.1735.

[57] C. C. Holt. Forecasting seasonals and trends by exponentially weighted moving averages. *International Journal of Forecasting*, 20(1):5–10, 2004. ISSN 0169-2070. doi: https://doi.org/10.1016/j.ijforecast.2003.09.015. URL https://www.sciencedirect.com/science/article/pii/S0169207003001134.

[58] R. Hyndman and G. Athanasopoulos. Forecasting: principles and practice, 3rd edition, otexts: Melbourne, australia. otexts.com/fpp3. accessed on 04/2024, 2021.

[59] R. J. Hyndman. Box-jenkins modelling. http://www.robjhyndman.com/papers/BoxJenkins.pdf, 2002. PDF.

[60] H. Jaeger. The" echo state" approach to analysing and training recurrent neural networks-with an erratum note'. *Bonn, Germany: German National Research Center for Information Technology GMD Technical Report*, 148, 01 2001.

[61] H. Jaeger. Short term memory in echo state networks, 01 2002.

[62] H. Jaeger. Tutorial on training recurrent neural networks, covering bppt, rtrl, ekf and the echo state network approach. *GMD-Forschungszentrum Informationstechnik, 2002.*, 5, 01 2002.

[63] H. Jaeger. Echo state network. *Scholarpedia*, 2(9):2330, 2007. doi: 10.4249/scholarpedia.2330. revision #196567.

[64] H. V. Jagadish, J. Gehrke, A. Labrinidis, Y. Papakonstantinou, J. M. Patel, R. Ramakrishnan, and C. Shahabi. Big data and its technical challenges. *Commun. ACM*, 57(7):86–94, jul 2014. ISSN 0001-0782. doi: 10.1145/2611567. URL `https://doi.org/10.1145/2611567`.

[65] I. Jestrović, J. L. Coyle, and E. Sejdić. The effects of increased fluid viscosity on stationary characteristics of eeg signal in healthy adults. *Brain Research*, 1589: 45–53, 2014. doi: 10.1016/j.brainres.2014.09.035. Epub 2014 Sep 22.

[66] A. Jungmittag. Combination of forecasts across estimation windows: An application to air travel demand. *Journal of Forecasting*, 35(4):373–380, 2016. doi: https://doi.org/10.1002/for.2400. URL `https://onlinelibrary.wiley.com/doi/abs/10.1002/for.2400`.

[67] B. X. Kenji Kawaguchi and L. Song. Deep semi-random features for nonlinear function approximation. In *Proceedings of the 32th AAAI Conference on Artificial Intelligence (AAAI)*. AAAI Press, 2018. URL `http://lis.csail.mit.edu/pubs/kawaguchi-aaai18.pdf`.

[68] M. Khan, H. Wang, A. Riaz, et al. Bidirectional lstm-rnn-based hybrid deep learning frameworks for univariate time series classification. *Journal of Supercomputing*, 77: 7021–7045, 2021. doi: 10.1007/s11227-020-03560-z. URL `https://doi.org/10.1007/s11227-020-03560-z`.

[69] S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan, and M. Shah. Transformers in vision: A survey. *ACM Comput. Surv.*, 54(10s), sep 2022. ISSN 0360-0300. doi: 10.1145/3505244. URL `https://doi.org/10.1145/3505244`.

[70] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In *Proceedings of the 25th International Conference on Neural Information Processing Systems - Volume 1*, NIPS'12, pages 1097–1105, Red Hook, NY, USA, 2012. Curran Associates Inc.

[71] G. Lai, W.-C. Chang, Y. Yang, and H. Liu. Modeling long- and short-term temporal patterns with deep neural networks, 2018.

[72] G. Lai, W.-C. Chang, Y. Yang, and H. Liu. Modeling long- and short-term temporal patterns with deep neural networks, 2018. URL https://arxiv.org/abs/1703.07015.

[73] A. Legendre. *Nouvelles méthodes pour la détermination des orbites des comètes*. F. Didot, 1805. URL https://books.google.it/books?id=FRcOAAAAQAAJ.

[74] R. Legenstein and W. Maass. Edge of chaos and prediction of computational performance for neural circuit models. *Neural Networks*, 20(3):323–334, 2007. ISSN 0893-6080. doi: https://doi.org/10.1016/j.neunet.2007.04.017. URL https://www.sciencedirect.com/science/article/pii/S0893608007000433. Echo State Networks and Liquid State Machines.

[75] R. Legenstein and W. Maass. *What makes a dynamical system computationally powerful?*, pages 127–154. MIT Press, 1 edition, 2007.

[76] Y. Li, R. Yu, C. Shahabi, and Y. Liu. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting, 2018.

[77] Z. Li, Y. Liu, and G. Tanaka. Multi-reservoir echo state networks with hodrick–prescott filter for nonlinear time-series prediction. *Applied Soft Computing*, 135:110021, 2023. ISSN 1568-4946. doi: https://doi.org/10.1016/j.asoc.2023.110021. URL https://www.sciencedirect.com/science/article/pii/S156849462300039X.

[78] Z. Li, S. Qi, Y. Li, and Z. Xu. Revisiting long-term time series forecasting: An investigation on linear mapping, 2023.

[79] J. Lienen and E. Hüllermeier. Instance weighting through data imprecisiation. *International Journal of Approximate Reasoning*, 134:1–14, 2021. ISSN 0888-613X. doi: https://doi.org/10.1016/j.ijar.2021.04.002. URL https://www.sciencedirect.com/science/article/pii/S0888613X21000463.

[80] Y. Liu, C. Gong, L. Yang, and Y. Chen. Dstp-rnn: A dual-stage two-phase attention-based recurrent neural network for long-term and multivariate time series prediction. *Expert Systems with Applications*, 143:113082, 2020. ISSN 0957-4174. doi: https://doi.org/10.1016/j.eswa.2019.113082. URL https://www.sciencedirect.com/science/article/pii/S0957417419307997.

[81] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell. Rethinking the value of network pruning, 2019.

[82] I. E. Livieris, E. Pintelas, and P. Pintelas. A cnn–lstm model for gold price time-series forecasting. *Neural Computing and Applications*, 32:17351–17360, 2020. doi: 10.1007/s00521-020-04867-x. URL https://doi.org/10.1007/s00521-020-04867-x.

[83] M. Lukoševičius and H. Jaeger. Reservoir computing approaches to recurrent neural network training. *Computer Science Review*, 3(3):127–149, 2009. ISSN 1574-0137. doi: https://doi.org/10.1016/j.cosrev.2009.03.005. URL https://www.sciencedirect.com/science/article/pii/S1574013709000173.

[84] S. Løkse, F. M. Bianchi, and R. Jenssen. Training echo state networks with regularization through dimensionality reduction. *Cognitive Computation*, 9, 06 2017. doi: 10.1007/s12559-017-9450-z.

[85] Q. Ma, L. Shen, W. Chen, J. Wang, J. Wei, and Z. Yu. Functional echo state network for time series classification. *Information Sciences*, 373:1–20, 2016. ISSN 0020-0255. doi: https://doi.org/10.1016/j.ins.2016.08.081. URL https://www.sciencedirect.com/science/article/pii/S0020025516306661.

[86] W. Maass, T. Natschläger, and H. Markram. Real-time computing without stable states: A new framework for neural computation based on perturbations. *Neural Computation*, 14:2531–2560, 2002. URL https://api.semanticscholar.org/CorpusID:1045112.

[87] G. Manjunath and H. Jaeger. Echo State Property Linked to an Input: Exploring a Fundamental Characteristic of Recurrent Neural Networks. *Neural Computation*, 25(3):671–696, 03 2013. ISSN 0899-7667. doi: 10.1162/NECO_a_00411. URL https://doi.org/10.1162/NECO_a_00411.

[88] Y. Matsubara, Y. Sakurai, W. G. van Panhuis, and C. Faloutsos. Funnel: automatic mining of spatially coevolving epidemics. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '14, page 105–114, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450329569. doi: 10.1145/2623330.2623624. URL https://doi.org/10.1145/2623330.2623624.

[89] A. Micheli and D. Tortorella. Discrete-time dynamic graph echo state networks. *Neurocomputing*, 496:85–95, Jul 2022. doi: 10.1016/j.neucom.2022.05.001. URL https://www.sciencedirect.com/science/article/pii/S092523122200532X.

[90] R. Ni, Z. Lin, S. Wang, and G. Fanti. Mixture-of-linear-experts for long-term time series forecasting, 2024.

[91] D. Opitz and R. Maclin. Popular ensemble methods: An empirical study. *Journal of Artificial Intelligence Research*, 11:169–198, Aug. 1999. ISSN 1076-9757. doi: 10.1613/jair.614. URL `http://dx.doi.org/10.1613/jair.614`.

[92] A. Oppenheim and A. Willsky. *Signals and Systems*. Prentice Hall, 2nd edition, 1997.

[93] Y.-H. Pao, G.-H. Park, and D. J. Sobajic. Learning and generalization characteristics of the random vector functional-link net. *Neurocomputing*, 6(2):163–180, 1994. ISSN 0925-2312. doi: https://doi.org/10.1016/0925-2312(94)90053-1. URL `https://www.sciencedirect.com/science/article/pii/0925231294900531`. Backpropagation, Part IV.

[94] R. Pascanu, T. Mikolov, and Y. Bengio. On the difficulty of training recurrent neural networks, 2013.

[95] G. Perone. Comparison of arima, ets, nnar, tbats and hybrid models to forecast the second wave of covid-19 hospitalizations in italy. *European Journal of Health Economics*, 23:917–940, 2022. doi: 10.1007/s10198-021-01347-4. URL `https://doi.org/10.1007/s10198-021-01347-4`.

[96] A. Rodan and P. Tino. Minimum complexity echo state network. *IEEE Transactions on Neural Networks*, 22(1):131–144, 2011. doi: 10.1109/TNN.2010.2089641.

[97] A. Rosenfeld and J. K. Tsotsos. Intriguing properties of randomly weighted networks: Generalizing while learning next to nothing, 2018.

[98] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning internal representations by error propagation. In D. E. Rumelhart and J. L. Mcclelland, editors, *Parallel Distributed Processing: Explorations in the Microstructure of Cognition, Volume 1: Foundations*, pages 318–362. MIT Press, Cambridge, MA, 1986.

[99] H. Salehinejad, S. Sankar, J. Barfett, E. Colak, and S. Valaee. Recent advances in recurrent neural networks, 2018.

[100] J. Schmidhuber. Learning complex, extended sequences using the principle of history compression. *Neural Computation*, 4(2):234–242, 1992. doi: 10.1162/neco.1992.4.2.234.

[101] S. P. Schnaars. Long-range forecasting: From crystal ball to computer: J. scott

armstrong, 2nd ed. (wiley, new york, 1985) [uk pound]22.95 (paper), pp. 689. *International Journal of Forecasting*, 2(3):387–390, 1986. URL https://EconPapers.repec.org/RePEc:eee:intfor:v:2:y:1986:i:3:p:387-390.

[102] R. Sen, H.-F. Yu, and I. Dhillon. Think globally, act locally: A deep neural network approach to high-dimensional time series forecasting, 2019. URL https://arxiv.org/abs/1905.03806.

[103] J. Skarding, B. Gabrys, and K. Musial. Foundations and modeling of dynamic networks using dynamic graph neural networks: A survey. *IEEE Access*, 9:79143–79168, 2021. ISSN 2169-3536. doi: 10.1109/access.2021.3082932. URL http://dx.doi.org/10.1109/ACCESS.2021.3082932.

[104] M. Skowronski and J. Harris. Minimum mean squared error time series classification using an echo state network prediction model. In *Proceedings - IEEE International Symposium on Circuits and Systems*, pages 4 pp. – 3156, 06 2006. doi: 10.1109/ISCAS.2006.1693294.

[105] S. F. Stefenon, M. H. D. M. Ribeiro, A. Nied, K.-C. Yow, V. C. Mariani, L. dos Santos Coelho, and L. O. Seman. Time series forecasting using ensemble learning methods for emergency prevention in hydroelectric power plants with dam. *Electric Power Systems Research*, 202:107584, 2022. ISSN 0378-7796. doi: https://doi.org/10.1016/j.epsr.2021.107584. URL https://www.sciencedirect.com/science/article/pii/S0378779621005654.

[106] T. Strauss, W. Wustlich, and R. Labahn. Design Strategies for Weight Matrices of Echo State Networks. *Neural Computation*, 24(12):3246–3276, 12 2012. ISSN 0899-7667. doi: 10.1162/NECO_a_00374. URL https://doi.org/10.1162/NECO_a_00374.

[107] M. Su, W. Zhao, Y. Zhu, D. Zha, Y. Zhang, and P. Xu. Anomaly detection of vectorized time series on aircraft battery data. *Expert Systems with Applications*, 227:120219, 2023. ISSN 0957-4174. doi: https://doi.org/10.1016/j.eswa.2023.120219. URL https://www.sciencedirect.com/science/article/pii/S0957417423007212.

[108] S. B. Taieb, G. Bontempi, A. Atiya, and A. Sorjamaa. A review and comparison of strategies for multi-step ahead time series forecasting based on the nn5 forecasting competition, 2011.

[109] G. Tanaka, T. Yamane, J. B. Héroux, R. Nakane, N. Kanazawa, S. Takeda, H. Numata, D. Nakano, and A. Hirose. Recent advances in physical reservoir computing:

A review. *Neural Networks*, 115:100–123, July 2019. ISSN 0893-6080. doi: 10.1016/j.neunet.2019.03.005. URL http://dx.doi.org/10.1016/j.neunet.2019.03.005.

[110] Y. Tay, M. Dehghani, S. Abnar, Y. Shen, D. Bahri, P. Pham, J. Rao, L. Yang, S. Ruder, and D. Metzler. Long range arena: A benchmark for efficient transformers, 2020.

[111] The New York Times. Pentagon explores a new frontier in the world of virtual intelligence. *The New York Times*, May 30 2003. Retrieved December 19, 2013.

[112] R. van den Berg, T. N. Kipf, and M. Welling. Graph convolutional matrix completion, 2017.

[113] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin. Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

[114] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol. Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion. *J. Mach. Learn. Res.*, 11:3371–3408, dec 2010. ISSN 1532-4435.

[115] L. Wang, Z. Wang, and S. Liu. An effective multivariate time series classification approach using echo state network and adaptive differential evolution algorithm. *Expert Systems with Applications*, 43:237–249, 2016. ISSN 0957-4174. doi: https://doi.org/10.1016/j.eswa.2015.08.055. URL https://www.sciencedirect.com/science/article/pii/S0957417415006120.

[116] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon. Dynamic graph cnn for learning on point clouds, 2019.

[117] Y. Wang, R. Zou, F. Liu, L. Zhang, and Q. Liu. A review of wind speed and wind power forecasting with deep neural networks. *Applied Energy*, 304:117766, 2021. ISSN 0306-2619. doi: https://doi.org/10.1016/j.apenergy.2021.117766. URL https://www.sciencedirect.com/science/article/pii/S0306261921011053.

[118] P. Werbos. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990. doi: 10.1109/5.58337.

[119] R. J. Williams and J. Peng. An Efficient Gradient-Based Algorithm for On-Line Training of Recurrent Network Trajectories. *Neural Computation*, 2(4):490–501, 12

1990. ISSN 0899-7667. doi: 10.1162/neco.1990.2.4.490. URL https://doi.org/10.1162/neco.1990.2.4.490.

[120] P. R. Winters. Forecasting sales by exponentially weighted moving averages. *Management Science*, 6(3):324–342, 1960.

[121] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang. Graph wavenet for deep spatial-temporal graph modeling, 2019.

[122] Z. Wu, D. Zheng, S. Pan, Q. Gan, G. Long, and G. Karypis. Traversenet: Unifying space and time in message passing for traffic forecasting, 2022.

[123] I. B. Yildiz, H. Jaeger, and S. J. Kiebel. Re-visiting the echo state property. *Neural Networks*, 35:1–9, 2012. ISSN 0893-6080. doi: https://doi.org/10.1016/j.neunet.2012.07.005. URL https://www.sciencedirect.com/science/article/pii/S0893608012001852.

[124] A. Zeng, M. Chen, L. Zhang, and Q. Xu. Are transformers effective for time series forecasting? *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(9):11121–11128, Jun. 2023. doi: 10.1609/aaai.v37i9.26317. URL https://ojs.aaai.org/index.php/AAAI/article/view/26317.

[125] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, and V. Prasanna. Graphsaint: Graph sampling based inductive learning method, 2020.

[126] B. Zhang, D. J. Miller, and Y. Wang. Nonlinear system modeling with random matrices: Echo state networks revisited. *IEEE Transactions on Neural Networks and Learning Systems*, 23(1):175–182, 2012. doi: 10.1109/TNNLS.2011.2178562.

[127] W. Zheng and G. Chen. An accurate gru-based power time-series prediction approach with selective state updating and stochastic optimization. *IEEE Transactions on Cybernetics*, 52(12):13902–13914, 2022. doi: 10.1109/TCYB.2021.3121312.

[128] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting, 2021.

# A | Graph neural networks

In this section, we provide a high-level overview of GNNs (GNNs). Even if the main focus of our work is not on GNNs, our approach is particularly useful in addressing scalability issues in GNNs. The main reference for this section is [52], [24].

## A.1. A brief introduction to graph neural networks

GNNs are a class of neural networks created for handling graph-structured data.

A graph is made by a set of nodes and a set of edges that connect pairs of nodes. The adjacency matrix $\mathbf{A}$ of a graph is a matrix of size $N \times N$ where $N$ is the number of nodes in the graph and it represents the connections between nodes. The attribute matrix $\mathbf{X}$ is a matrix of size $N \times F$ where $F$ is the number of features for each node. The graph can eventually have an optional set of edges' attributes denoted by $\mathcal{E}$.

The graph is processed by an encoder made of multiple layers of non-linear transformations. The encoder processes both the nodes' features and the structure of the network. The result of the encoding is embeddings, which capture the important information about each node or subgraph in dense, lower-dimensional representations. The conventional RNNs to exchange information across nodes is a process called "Neural Message Passing" [52]. In this process, neighbor nodes in the graph iterate a process of information passing allowing each node to gather information from nodes at different distances in the graph. In doing so, each node can use the information from its neighbors to adjust its internal representation. At the end of the procedure, each node has integrated into its representation both structural and feature information of the graph.

The message-passing layer is composed of a message function $\mathrm{MsG}^l$, an aggregation function AGGR and an update function $\mathrm{UP}^l$. For each $l$-th layer and each node $i$, it is computed:

$$\boldsymbol{h}^{i,l+1} = \mathrm{UP}^l \left( \boldsymbol{h}^{i,l}, \underset{j \in \mathcal{N}(i)}{\mathrm{AGGR}} \left\{ \mathrm{MsG}^l \left( \boldsymbol{h}^{i,l}, \boldsymbol{h}^{j,l}, \boldsymbol{e}^{ji} \right) \right\} \right)$$

where $\mathcal{N}(i)$ is the set of neighbors of node $i$ and $\boldsymbol{e}^{ji}$ is the edge attribute between the two

neighbors nodes $i$ and $j$ [26].

Using graphs as inputs implies a strong relational inductive bias on the learning process of RNNs which captures the dependencies among the graph's nodes [7]. This indicates that the graph's connection and structure are essential to the learning process.

## A.2. Spatio-temporal time series forecasting

**Multiple time series** When considering time series forecasting, one time series only is often not enough to generalize well and make accurate predictions. For this reason, the use of multiple time series, each of which can be multivariate, may be needed to improve the forecasting performance. Let's consider two opposite types of solutions: local models and global models [9]. On one side, local models exploit a distinct model for each time series. However, even if they are tailored to each time series, they have the downside of being sample and computationally inefficient. On the opposite side, global models train a single model on all the time series learning parameters for the whole group and not specific for each time series. Using a single model for all the time series can be more efficient in terms of sample complexity since the sample size is bigger allowing the use of more complex models that, in turn, prevent overfitting [26]. The downside of both models is that they neglect the dependencies among the time series.

**Taking into account the dependencies** The simplest way to take into account dependencies is to consider the input as a single multivariate time series, however, this approach suffers from the curse of dimensionality and has scalability issues.

If we employ GNNs, the relationships between different time series can be used to improve the forecast. These models, known as Spatio-Temporal GNNs (STGNNs), are dynamic GNNs [103] with nodes and edge attributes that can evolve through time. The topology of the graph can also change over time, with the addition or the removal of nodes and edges. The adjacency matrix $\mathbf{A}_t \in 0, 1^{N \times N}$, as well as the optional set of edges attributes associated with non-zero entries in $\mathbf{A_t}$ denoted by

$$\mathcal{E}_t \doteq \left\{ \left\langle (i,j), \boldsymbol{e}_{\boldsymbol{t}}^{ij} \right\rangle \mid \forall i,j : \boldsymbol{A}_t[i,j] \neq 0 \right\}.$$

The forecast is performed by taking into account an inductive bias between related time series making use of a sharing architecture. This framework is based on the assumption of *"causality à la Granger"* [49], [26] which states that the uncertainty on future values of each time series can be reduced by taking into account the observations on other

time series in the collection. Therefore, to exploit the known relationships between the time series, the data is represented as a graph where each node represents a time series [121], [76] and the edges represent the relationships between them. Then, information is propagated in space through the message-passing framework.

**STGNN formulation** In STGNN the dynamics are influenced by both the sequence of time steps and the connections between vertices in the graph.

Apart from the observations $\boldsymbol{x}_t^i \in \mathbb{R}^{d_x}$ at each time step $t$ we can also take into account $\boldsymbol{u}_t^i \in \mathbb{R}^{d_u}$ exogenous variables at each time step $t$ and $\boldsymbol{v}^i \in \mathbb{R}^{d_v}$ a vector of time-independent attributes.

We obtain the tuple

$$\mathcal{G}_t \doteq \langle \boldsymbol{X}_t, \boldsymbol{U}_t, \mathcal{E}_t, \boldsymbol{V} \rangle$$

of all the information available at time $t$ [26]. An example reported in [26] about a network of traffic sensors, takes as $\boldsymbol{X}_{1:t}$ the past observations of the traffic speed, $\boldsymbol{U}_{1:t}$ as the time-of-day and day-of-week indicators plus the weather conditions, $\boldsymbol{V}$ as the information on the road where the sensors are placed, such as the road type and the number of lanes, and finally, $\mathcal{E}_t$ and $\mathcal{A}_t$ that indicate whether the sensors are connected by a road segment and the distance between them and if there are any roadworks or accidents on the road segment, they change their values.

We are interested in learning a parametric model $\mathcal{F}(\cdot;\theta)$ s.t.

$$\mathcal{F}\left(\mathcal{X}_{t-W:t}, \boldsymbol{U}_{t:t+H}; \boldsymbol{\theta}\right) = \widehat{\boldsymbol{X}}_{t:t+H} \approx E_p\left[\boldsymbol{X}_{t:t+H}\right]$$

[26] where $\mathcal{X}_t = \langle \boldsymbol{X}_t, \boldsymbol{U}_t, \boldsymbol{V} \rangle$, $\boldsymbol{X}_t \in \mathbb{R}^{N \times d_x}$ are the observations at time $t$, $\boldsymbol{U}_t \in \mathbb{R}^{N \times d_u}$ are the exogenous variables at time $t$ and $\boldsymbol{V} \in \mathbb{R}^{N \times d_v}$ are the time-independent attributes. As it can be seen in the equation above, the input of the model is the past values of $\mathcal{X}_t$ in a window of $W$ time steps and the future values of the exogenous variables $\boldsymbol{U}_{t:t+H}$ in a horizon of $H$ time steps. Then, $\boldsymbol{\theta}$ are the parameters of the model, $\widehat{\boldsymbol{X}}_{t:t+H}$ are the values predicted by the model and $E_p\left[\boldsymbol{X}_{t:t+H}\right]$ is the expected value of the future values (see Figure A.1). The parameters $\boldsymbol{\theta}$ are learned by minimizing a loss function $\ell$:

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \frac{1}{T} \sum_{t=1}^{T} \ell\left(\widehat{\boldsymbol{X}}_{t:t+H}, \boldsymbol{X}_{t:t+H}\right)$$

[26].

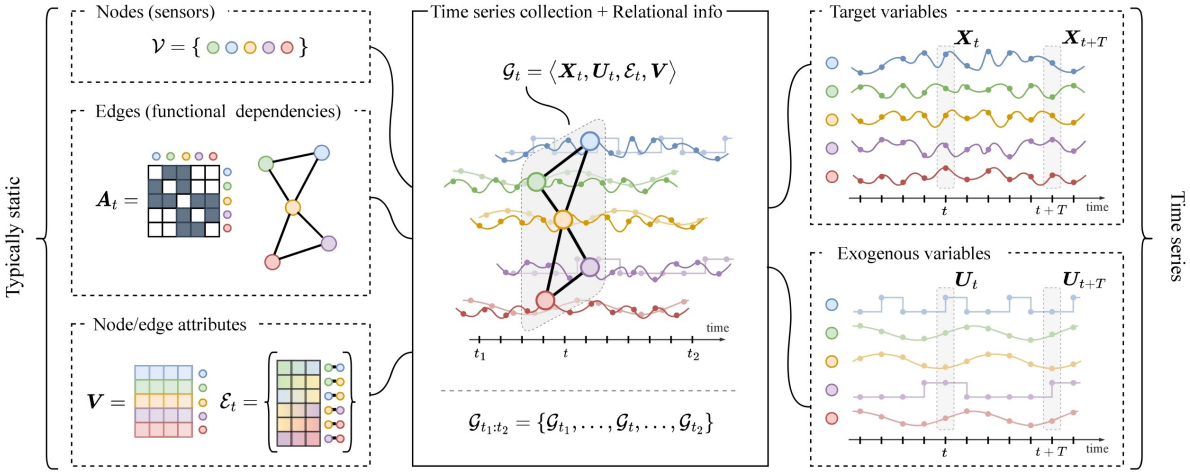



Figure A.1: Spatiotemporal Graph Neural Network.
On the left there is the set of nodes $\mathcal{V}$, the set of edges $\mathcal{A}_t$, the edge attributes $\mathcal{E}_t$, and the node attributes $\boldsymbol{V}$. On the right, there is a set of target and exogenous variables. In the middle, there is the RNN processing the time series data for each node.

*Source: [25]*

**Components of STGNNs** STGNNs is made of three main components: an encoder, a message-passing layer and a decoder. An encoding layer is used to encode the observations at each time step (e.g. an MLP), separately for each time series. The encoder is followed by a stack of spatiotemporal message-passing layers, which propagate the information through time and space. The resulting representation is mapped into a prediction using a decoder (e.g. an MLP). The message-passing blocks take into account the dependencies between the nodes and time steps. The message-passing layer is the only component propagating temporal and spatial information [26]. It is composed of a message function $\text{MsG}^l$, a permutation-invariant aggregation function AGGR and an update function $\text{UP}^l$ implemented, for example, by an RNN. A Message-Passing (MP) neural network computes for each $l$-th layer and each node $i$ at time $t$ the following equation:

$$\boldsymbol{h}_t^{i,l+1} = \text{UP}^l\left(\boldsymbol{h}_{\leq t}^{i,l}, \underset{j \in \mathcal{N}_t(i)}{\text{AGGR}}\left\{\text{MsG}^l\left(\boldsymbol{h}_{\leq t}^{i,l}, \boldsymbol{h}_{\leq t}^{j,l}, \boldsymbol{e}_{\leq t}^{ji}\right)\right\}\right)$$

where $\mathcal{N}_t(i)$ is the set of neighbors of node $i$ at time $t$ and $\boldsymbol{e}_{\leq t}^{ji}$ is the edge attribute between the two neighbors nodes $i$ and $j$ up to time $t$ [26].

**Complexity** This approach, however, introduces additional computational complexity. As stated in [24], the computational complexity of training STGNNs is challenging dis-

abling the use of large graphs or long spatiotemporal series. In contrast, the approach proposed in [24] allows for constant scalability in training, concerning the spatiotemporal dimension of the problem. This is achieved by enabling mini-batch sampling without considering the length of the sequence and the size of the graph, as will be explained in section A.3.

## A.3. Research gap: scalable spatio-temporal RNN architecture

Unlike static graph signals, where research has focused on scalability, spatiotemporal dynamics, which introduce additional complexities, haven't been explored under this aspect [24].

When dealing with the problem of computational scalability, the two dimensions that play the game are space and time. Space is the dimension corresponding to the number of time series, which in the case of STGNN can be the number of sensors, and time is the dimension associated with the number of time steps. The standard STGNNs have time and space complexity that scales with the length of the input sequence times the number of layers in the network times the number of edges in the graph: $\mathbf{O}(\mathbf{TL}|\mathcal{E}_\mathbf{t}|)$ [26] where $\mathbf{T}$ is the number of time steps, $\mathbf{L}$ is the number of layers, and $|\mathcal{E}_\mathbf{t}|$ is the number of edges in the graph at time $\mathbf{t}$ [24]. It is evident that the high complexity prevents the application of this type of model on large sensor networks.

One common approach to enhance scalability in training procedures involves subsampling graphs [53], [125]. However, applying subsampling to node-level observations as if they were iid can break dependencies, especially in dynamic graphs where dependencies are both spatial and temporal [24].

It is possible not to consider spatial propagation during the training phase by precomputing aggregated features over the space dimension as demonstrated in architectures like the one presented by [35]. However, extending this method to the spatiotemporal case is challenging, as the preprocessing step must consider both spatial and temporal dependencies.

This research gap is addressed by the scalable spatio-temporal RNN architecture proposed in [24]. In the approach proposed by [24], the encoding utilizes a training-free deep randomized RNN [60], [41] to generate high-dimensional vector embeddings to encode the history of each sequence (see Figure A.2). In addition to the time dependencies, the spatial dependencies are included in the representation using incremental powers of the

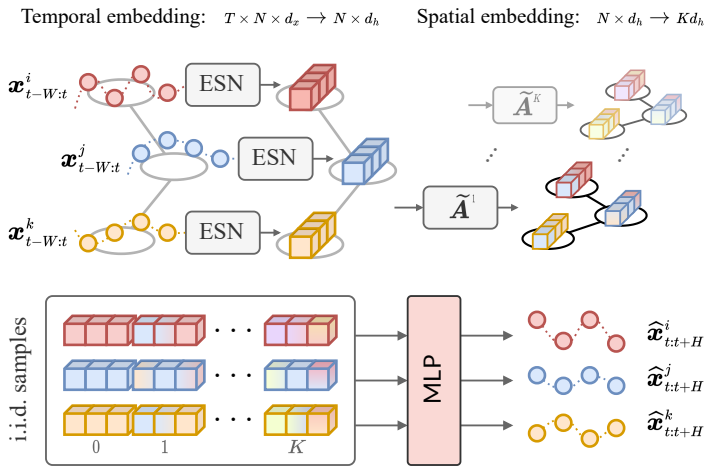



Figure A.2: Scalable spatiotemporal RNN by [24].
The encoder and decoder proposed by [24] to overcome the problem of scalability of STGNNs. The time series are processed by an ESN (encoder), then powers of the shift operator are employed to propagate the information along the spatial dimension and the resulting concatenations of the representation are fed to an MLP, the only trainable component, that outputs the predictions.

*Source: [24]*

graph adjacency matrix. The final embedding is created by concatenating representations obtained for each propagation step and includes both the spatial and temporal dynamics.

In this way, the preprocessing calculates ahead of training all the nodes' representations taking into account time and space dependencies, so that the resulting representations can be sampled as they were iid to train the readout. An advantage of this method is that the preprocessing step can be performed ahead of training and is not constrained by online training on GPUs with limited memory. Similarly to this approach, SIGN [35], performs the spatial propagation in a preprocessing step using linear diffusion matrices. In contrast to this approach, SIGN is for static graphs.

The decoder in [24], according to the target task, maps these node representations obtained as concatenation of the encodings to the output. In this way, the decoder is the only trainable component, and the cost of the training step becomes independent of the length of the input sequence and the number of edges in the graph.

The results of [24] show lower memory consumption and higher throughput compared to the state of art models. The downside of the approach is the great amount of disk storage required to pre-compute the representation and the need for a fine-tuned hyperparameter selection because the process is not end-to-end and hence is more reliant on

hyperparameter.

Previous works already described architecture that extends the deep ESN reservoir model to dynamic temporal graphs in which the vector encoding for the dynamic graph is updated at each time step without requiring training such as for DynGESN [89]. However, while DynGEN architecture is fully randomized, the architecture presented by [24] is hybrid, since the decoder is trainable. This hybrid approach allows to employ the strengths of both randomized and trainable components in different parts of the model.

### A.3.1. Trainable decoding

The decoder is the only trainable component of the architecture. Following the approach proposed in [24] since the embeddings already include the spatial and temporal relations among time steps and nodes, the decoder can process them enabling subsampling in batches that fit in the available hardware, in parallel to be scalable to larger networks and efficient.

The only memory constraint is in the pre-processing step, which may require a significant amount of memory if the reservoir is large.

# B | ESN technical specifications

Here, we briefly touch on the Lyapunov Exponent, which is a useful measure for the stability of the reservoir in Echo State Networks.

## B.1. Lyapunov exponents

The spectral radius does not guarantee the echo state property and it is not an accurate measure of reservoir stability. The Lyapunov Exponents are a more accurate measure of the stability of the reservoir. When a system is stable, for a given time series and its slightly perturbed version, the system's trajectory should not deviate significantly, because the perturbation effects should reduce over time. The Lyapunov Exponents are the eigenvalues of the Jacobian matrix. If the neuron activation function is a sigmoidal function, the Jacobian matrix at time k can be expressed as:

$$\mathbf{J}(h[k]) = \begin{bmatrix} 1-(h_1[k])^2 & 0 & \dots & 0 \\ 0 & 1-(h_2[k])^2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1-(h_{N_r}[k])^2 \end{bmatrix} \mathbf{W}_r^r$$

where $\mathbf{W}_r^r$ is the recurrent connections matrix, and $h_l[k]$ is the activation of the $l$ th neuron, with $l = 1, 2, \dots, N_r$ .

$\lambda$ is then computed as

$$\lambda = \max_{n=1,\dots,N_r} \frac{1}{K} \sum_{k=1}^{K} \log\left(r_n[k]\right)$$

where $r_n[k]$ is the absolute value of the $n$ th eigenvalue of the Jacobian matrix at time k and $K$ is the number of time steps in the trajectory [12]. The sign of $\lambda$ indicates the stability of the reservoir. If $\lambda$ is greater than zero, the system has a chaotic behavior [12].

# Acknowledgements

I am grateful to my co-advisors, Andrea and Ivan, for their invaluable knowledge and experience that they shared with me. Their guidance enabled me to create a thorough and well-founded work that addresses current interests in the field.

My sincere thanks go to Professor Alippi and Professor Manuel Roveri for the opportunity provided and for their support.